

pico

Agent项目

pico

项目简历描述&代码&原始文档下载

怎么学

Pico 安装与模型使用

[临时] right code key

背景与目标

00-全局总览

01-代码仓库上下文设计

02-运行时主循环与执行编排设计

03-工具的接入与调用设计

04-提示词形态与缓存复用设计

05-结构化的会话记忆

06-上下文瘦身与输出管理设计

07-会话状态、运行工件与恢复机制设计

08-评测框架与实验方法设计

【重要】90-针对项目描述的逐字面试话术

【更新中】91-面试题

92-claude code相关

99-面经

0417字节Agent开发一面

AI Coding常见面试题

项目简历描述&代码&原始文档下载

version 2.0 优化记忆模块，支撑面试深度讲

Pico：本地代码智能体 Harness

核心技术：Python、Agent Harness、Tool Calling、Context Management、Checkpoint / Resume、Layered Memory、Run Trace

项目描述：

面向代码仓库长链路任务开发本地代码 agent harness，围绕模型接入、工具调用、上下文管理、任务恢复、结构化记忆、运行审计和评测闭环做系统化设计，重点解决多轮任务里 prompt 膨胀、重复读文件、状态丢失、工具副作用不可控和结果难复盘的问题。

核心职责与贡献：

1. **Agent Harness 架构设计**：负责本地代码 agent 的整体设计与开发，统一模型接入、工具执行、会话状态、checkpoint 恢复和运行工件落盘流程，形成可复盘的执行链路；支持 **2 类模型后端、7 类工具和 3 类运行工件**。
2. **长上下文治理**：设计分层上下文管理与预算裁剪机制，在 **12 组长上下文配置里**，将平均 prompt 长度从 **7082** 压到 **5664**，平均压缩率 **16.19%**，最高压缩率 **33.28%**，同时保证当前请求不被裁坏。
3. **结构化记忆系统**：针对多轮任务里 agent 反复读同一文件、重复确认已知事实的问题，把任务摘要、文件摘要、过程笔记和相关记忆召回做了分层；在 **12 个记忆依赖任务里**，follow-up 阶段的重复读文件次数从 **60 次** 降到 **0 次**，且不再需要额外工具调用去重新确认已经拿到的事实。
4. **任务恢复机制**：设计 checkpoint / resume 机制，让 agent 在上下文超预算、中断恢复和 workspace 漂移场景下恢复任务状态，而不是重读整段聊天历史；覆盖 **10 个恢复场景**，workspace 漂移识别率 **100%**，且没有出现误信旧状态继续执行的情况。
5. **工具安全与运行治理**：构建标准化工具调用与安全边界，覆盖参数校验、工作区隔离、高风险审批、重复调用拦截、敏感信息脱敏和 partial success 识别；在固定回归任务中保持 **100% 通过率、100% 预算内完成率和 100% verifier 通过率**。
6. **评测与审计闭环**：将评测拆成 harness regression、上下文治理、记忆收益和恢复正确性几层，分别验证运行时合同稳定性、模块收益和恢复边界，避免把模型能力、系统能力和运行观测混成一个总分；形成固定 benchmark、对照实验和运行工件聚合三类评测路径。

version 1.0 初始发布版本

Pico：本地代码智能体Harness

核心技术：

Python、Agent Harness、Tool Calling、Context Engineering、Memory Retrieval、Prompt Cache

项目描述：

面向代码仓库长链路任务开发本地代码 Agent Harness，覆盖模型接入、工具调用、上下文管理、结构化记忆、运行审计与评测闭环，重点解决多轮任务中的上下文膨胀、重复读取、工具误调用和结果不可复现问题。

核心职责与贡献：

- 1. Agent Harness 架构设计：**负责本地代码 Agent 的整体设计与开发，统一模型接入、工具执行、会话状态和运行工件落盘流程，形成可回放的任务执行链路；支持 2 类模型后端、7 类工具和 3 类运行工件。
- 2. 长上下文治理：**针对代码仓库长链路任务中的 prompt 膨胀问题，设计分层上下文管理与预算裁剪机制；在 12 组真实长上下文配置上，将平均 prompt 长度从 6964 压缩到 5418，平均压缩率 18.01%，最高压缩率 35.63%。
- 3. 结构化记忆系统：**将任务摘要、文件摘要和会话笔记分层管理，并结合 freshness 校验和相关记忆召回减少重复读取；在 12 个真实记忆依赖任务中，将重复读文件次数从 8 次降到 3 次，平均工具步数从 0.67 降到 0.25，任务正确率从 66.7% 提升到 100%。
- 4. 工具安全与运行治理：**构建标准化工具调用与安全边界，覆盖参数校验、工作区隔离、高风险审批、只读委派、重复调用拦截和敏感信息脱敏；在 11 个真实治理场景中，结构化记录 3 次路径逃逸拦截、5 次无效参数拒绝和 2 次重复调用拦截。
- 5. 评测与审计闭环：**建立固定 benchmark 与运行审计体系，支持 pass_rate / attempts / tool_steps / failure category / trace 的自动汇总与回归对比；当前覆盖 6 个标准化任务和 86 条自动化测试。
- 6. 模型后端效果评估：**搭建 GPT / Claude provider 对照实验框架，在 GPT 后端的 6 个固定任务上取得 83.33% 的 pass rate，平均 attempts=3.00、平均 tool_steps=2.00，支撑不同模型后端的效果与成本评估。

代码仓库

<https://gitee.com/htxofficial/pico/tree/main>

运行部分截图

```

○ ● ● ■ pico — python - uv run pico — 127x35
martinlos@192 pico % uv run pico

=====+=====
              /\____/\
             (  o o  )
              ^
             /\
            /\  \
           /|   \|\\
          pico
        local coding agent
      calm shell, ready for work
=====+=====

| WORKSPACE  /Users/martinlos/code/pico
| MODEL      gpt-5.4
| APPROVAL   ask
| BRANCH     main
| SESSION    20260409-212105-b5a2de
|=====+=====

pico> 看下这个代码仓库是在干什么

这个仓库是在做一个本地代码助手 `pico`。

- 它是面向“当前代码仓库”工作的轻量 coding agent，不是普通聊天工具
- 运行形态是命令行工具，包名/CLI 都叫 `pico`，也支持 `python -m pico`
- 主要用途是：读仓库、分析代码、排查问题、修改文件、执行命令、修测试
- 它会把会话和运行产物持久化到本地 `.pico/` 目录，方便连续工作
- 支持多种模型后端：Ollama、OpenAI 兼容 Responses API、Anthropic 兼容 Messages API

从 README 看，核心定位可以理解为：
“一个直接在终端里、围绕本地仓库持续工作的 coding agent”。

如果你愿意，我下一步可以继续帮你看两件事之一：
1. 梳理这个仓库的代码结构和核心模块
2. 说明它一次完整运行的流程（CLI → 会话 → 工具调用 → 持久化）

pico> █
```

```
pico — python - uv run pico — 127*34
```

pico> pico有哪些harness的设计，读代码之后告诉我。

`pico` 里比较明确的 “harness” 设计，核心是把模型包在一层受控运行时里，而不是直接裸调模型。

- 运行时 harness: `pico/runtime.py:1` 把 pico 定义成模型外的控制循环，负责组 prompt、解析模型输出、校验并执行工具、写 trace、更新工作记忆、决定何时停止。这是总控 harness。
- 工具 harness: `pico/tools.py:1` 是能力白名单层，定义工具 spec、参数 schema、风险标记和执行方式。也就是说模型不能随便做事，只能走这里注册过的工具。
- 安全/审批 harness: 从 `pico/tools.py:14` 可以看到工具分 `risky` 与否；README 也强调“受约束的工具”执行。本质上这是一个 safety harness，把高风险动作单独标注并交给外层策略控制。
- 工作区 harness: `pico/workspace.py` 和 `pico/tools.py:12` 这一层负责仓库边界、忽略路径、输出裁剪等，让 agent 始终围绕当前 repo 工作，而不是无限制访问环境。
- 上下文 harness: `pico/context\_manager.py` 提供上下文管理，把当前任务、近期文件、摘要等整理进 prompt，避免每轮都把整个仓库塞给模型。
- 状态持久化 harness: `pico/run\_store.py`、`pico/task\_state.py`、`pico/memory.py` 组成会话/任务状态层，用来保存运行记录、任务状态和本地记忆，这对应 README 里提到的 `.pico/` 本地状态目录。
- 模型适配 harness: `pico/models.py` 是后端适配层，说明 pico 不是把业务逻辑写死在某个模型 SDK 上，而是通过统一接口切换/封装模型提供者。
- CLI harness: `pico/cli.py` 是用户入口，把命令行输入接到 runtime，再把结果、工具调用和状态管理串起来，相当于“终端交互壳”。

如果用更工程化的话讲，这个仓库里的 harness 设计大致是：

CLI -> Runtime loop -> Context assembly -> Model adapter -> Tool gateway -> Workspace/safety guard -> State/memory persistence

目前我已经直接读到并能明确确认的是：

- `pico/runtime.py`
- `pico/tools.py`

其余几个层次是结合仓库结构、README 和运行时引用关系判断的。

如果你要，我下一步可以继续逐个读 `pico/models.py`、`pico/context\_manager.py`、`pico/run\_store.py`、`pico/task\_state.py`，给你整理成一张更准确的“harness 架构图 + 调用链路”。

pico> █

文档Markdown下载

01-代码仓库上下文设计.md 02-运行时主循环与执行编排设计.md 03-工具的接入与调用设计.md
04-提示词形态与缓存复用设计.md 05-结构化的会话记忆.md
06-上下文瘦身与输出管理设计.md 07-会话状态、运行工件与恢复机制设计.md
08-评测框架与实验方法设计.md 背景与目标.md 项目简历描述&代码.md

怎么学

先从简历描述和追问切入，带着问题去读模块文档和代码，用 AI 帮你补问题清单，再反复把理解压成自己的面试话术；学到后面，最好亲手改一两个点，这样这个项目才真正变成你的。

1. 先看简历描述。

先看这个项目在简历里是怎么讲的，然后假设你自己是面试官，想一想你会往下追问什么。

- a. 比如：这个项目到底解决了什么问题？为什么叫 harness，不是普通 agent？上下文、记忆、工具、安全、评测这些模块为什么要这么设计。
- b. 如果条件允许，最好先把项目跑一次

2. 带着问题去看对应模块文档，过程中一定要结合代码。

看模块文档 -> 对照核心代码 -> 顺手看一点测试。

每看完一章，强制自己输出三件事：

这个模块解决什么问题；

它是怎么做的；

为什么这样做，而不是另一种做法。

最后把这些内容整理成自己的面试话术，可以参考对应的章节。

3. 如果不会提问，或者提问能力比较弱，就让 AI 帮你补问题清单。

AI 的作用是帮你放大大理解，不是替你理解。

你可以让 AI 帮你做这些事：

针对某个模块生成 5 到 10 个面试追问；

帮你判断哪些问题是高频问题，哪些是进阶问题；

帮你把一章内容压成 30 秒、1 分钟、3 分钟三种讲法。

等等……核心的方法论就是培养你提问的维度和深度。

4. 反复执行第 2 步，直到你能把项目主链路讲顺。

测试下你能不能不用看资料和自己的面试话术，把这个项目怎么跑起来讲清楚。

比如至少要能顺着讲出一条主链路：

用户请求进来之后，CLI 怎么组装 agent，runtime 怎么进入主循环，context 怎么拼 prompt，模型怎么返回结果，工具怎么执行，记忆怎么更新，最后 trace 之类的怎么落盘。

到了这一步，再用费曼学习法，假设你在教别人，看看自己会怎么讲。

讲不顺的地方，基本就是你还没真的吃透的地方。

5. 过程中有问题就来群里问我，进阶的话一定要自己改一点东西。

最好的进阶方式，是选一个你最熟的模块，做一点自己的改造。哪怕只是：

改一下上下文裁剪策略；

补一个记忆召回规则；

加一个工具安全校验；

改一个 benchmark 或实验口径。

Q1：我完全没有基础，是不是要先看八股？

不用先看八股，直接看项目更有效，build first。

因为你完全没基础的时候，先去啃一大堆八股，很多东西其实进不了脑子。先跟着项目走，边做边碰问题，你会更清楚每个知识点到底是拿来解决什么的，这时候再回头查，吸收会快很多。

但有个前提，不能闷头做，一定要带着问题去学。比如这里为什么这么设计，这个报错是怎么来的，这个方案为什么不用另一种。你一直带着这些问题去补知识，学到的东西才会慢慢连起来。

Pico 安装与模型使用

这推荐直接用默认配置，也就是 `openai` provider 配 `Right Code`。

这条链路在 `pico` 里默认就是通的，而且走的是带缓存的实现。

Right Code 注册地址：

<https://www.right.codes/register?aff=e1617692>

注：right code是我用的比较多的一个中转站，大家可以根据自己需求配置

1. 安装 Pico

`pico` 需要 Python 3.10+。

如果你用 `uv`，在项目根目录执行：

```
1 uv sync
```

如果你已经有自己的 Python 环境，也可以直接装成可编辑模式：

```
1 pip install -e .
```

装好以后，下面几个入口都能用：

```
1 uv run pico
2 python -m pico
3 pico
```

2. 最推荐的用法：默认 OpenAI 兼容接口 + Right Code

`pico` 的默认 provider 是 `openai`，默认 base URL 是：

```
1 https://www.right.codes/codex/v1
```

也就是说，多数情况下你其实不用再手动指定 `--provider openai`，也不用手动写 `--base-url`，只要把 API Key 配好就能直接跑。

最小配置就是：

```
1 export OPENAI_API_KEY="你的 Right Code Key"
2 uv run pico
```

如果你想显式写全，也可以这样：

```
1 export OPENAI_API_BASE="https://www.right.codes/codex/v1"
2 export OPENAI_API_KEY="你的 Right Code Key"
3 export OPENAI_MODEL="gpt-5.4"
4 uv run pico --provider openai
```

- `pico` 默认就是 `openai` provider。
- `openai` 这条链路会走 `prompt cache`，也是现在更推荐的一条。

3. 为什么推荐用带缓存的 provider

`pico` 每一轮都会带上稳定前缀，比如工具说明、工作区信息、会话结构这些内容。只要后端支持缓存，这部分前缀就不用每次都完整重算一遍。

在当前实现里：

- `openai` 兼容接口支持这套缓存参数
- 只要 base URL 命中 `openai.com` 或 `right.codes`，`pico` 就会自动带上缓存字段
- 运行时默认会发送 `prompt_cache_key`
- 缓存保留策略默认是 `in_memory`

所以如果你主要是日常写代码、追问、接着上一轮继续干，推荐优先用：

```
1 uv run pico
```

或者：

```
1 uv run pico --provider openai
```

别自己切到没有缓存的 provider，除非你就是想测别的后端。

4. Right Code 的推荐配置

如果你是第一次配，直接按下面来就行：

```
1 export OPENAI_API_KEY="你的 Right Code Key"
2 export OPENAI_API_BASE="https://www.right.codes/codex/v1"
3 export OPENAI_MODEL="gpt-5.4"
4 uv run pico
```

如果想长期使用，可以把这几行放进你的 shell 配置，比如 `~/.zshrc`。

推荐注册入口：

<https://www.right.codes/register?aff=e1617692>

5. 常见启动方式

在当前仓库里启动交互模式：

```
1 uv run pico
```

指定一个别的项目目录：

```
1 uv run pico --cwd /path/to/repo
```

直接跑一次性任务：

```
1 uv run pico "inspect the test failures and propose a fix"
```

继续上一次会话：

```
1 uv run pico --resume latest
```

6. 其他 provider 怎么用

Ollama

适合本地模型，但不走 `pico` 现在这套 prompt cache 语义。

```
1 ollama serve
2 ollama pull qwen3.5:4b
3 uv run pico --provider ollama --model qwen3.5:4b
```

Anthropic 兼容接口

`pico` 也支持 Anthropic 兼容接口，默认地址是：

```
1 https://www.right.codes/claude/v1
```

配置方式：

```
1 export ANTHROPIC_API_BASE="https://www.right.codes/claude/v1"
2 export ANTHROPIC_API_KEY="你的 Right Code Key"
3 export ANTHROPIC_MODEL="claude-sonnet-4-6"
4 uv run pico --provider anthropic
```

这一条能用，但要注意，当前 `anthropic` 这条链路在 `pico` 里没有接 prompt cache 复用。如果你更看重缓存和连续使用成本，还是优先用默认的 `openai` provider。

还有个细节，`anthropic` provider 会优先读这些环境变量：

1. `ANTHROPIC_API_KEY`
2. `RIGHT_CODES_API_KEY`
3. `OPENAI_API_KEY`

但 `openai` provider 只直接读取 `OPENAI_API_KEY`。所以你如果走默认推荐配置，最稳的是直接设置 `OPENAI_API_KEY`。

8. 常见问题

Q1：为什么我什么都没配，`pico` 还是在走 Right Code？

因为 `pico` 默认 provider 是 `openai`，默认 base URL 也是 `https://www.right.codes/co`
`dex/v1`。

Q2：我已经有别家的 OpenAI 兼容接口，能接吗？

可以，改掉 `OPENAI_API_BASE` 就行。但如果那个后端不支持缓存语义，`pico` 虽然也能跑，体验和成本就不一定有 Right Code 这条默认链路好。

[临时] right code key

因为最近right code拉闸注册，给大家一些key，没有注册的直接用就好了（ps：我近期会更新pico接入一些国内的模型，比如qwen 3.6这样的，有能力的小伙伴也可以帮我实现一下提个pr～）

以下都是我自掏腰包的，每个5美元的额度，希望大家按需取用，一个人用了评论一下已经用了这个key，给大家节省点时间，感谢。

1. sk-66cd848448d74139ae08ee1e746e2dea
2. sk-71845fb77f2b40f4b859a8823f3f2d1f
3. sk-caa638b638f847cead42f6b12e2057e4
4. sk-8ea86b9cd9d746f793f38b425afec39e
5. sk-06ea56b7c87c41aba650975da4cb40e6
6. sk-17abc00a0f3849e89015549207c71e3b
7. sk-97122839837e4b5f9e3b56e9b6d481cb
8. sk-61ef2419afb84284b2f16d045d94290e
9. sk-2d2f6943601e4d968e973022a384db51
10. sk-ea361bdc5f4e46b5905b7f489cd2c30b

建议测试的时候用gpt5.4 量大管饱～

背景与目标

我们的目标：如何编写一个code agent让LLM发挥更大的威力

大家可以思考一下下面这个问题怎么解决

如果你用GPT4o，但是想让你在**解决问题的时候**达到GPT5家族的能力

很多人可能会觉得这是天方夜谭

这对大多数人来说，确实是这样的，模型能力基本就等于了他日常的使用能力，但在

最近LLM领域的进步，不仅仅靠模型本身变强，更在于我们**怎么用**它们。

在很多真实的落地场景里，模型外围的配套系统——比如工具调用、上下文管理和记忆功能——发挥的作用一点都不比模型本身小。这就解释了，为什么像Claude Code或Codex这样的系统，用起来感觉比你在普通聊天界面里直接跟它们背后的模型对话要强得多（感兴趣的可以去了解下post training）

这个项目最核心的地方就是通过编写一个轻量化的code agent来展示你在面试时候对agent和harness能力面。

那接下来就让我补充一点前置知识，来一点点回答：如何编写一个code agent让LLM发挥更大的威力

从codex/claude code说起

最近（2026/04）的后端/agent面试，逃不开的话题就是claude code和codex，很多面试官会考察你

1. 如何使用
2. 有没有了解里面的一下架构设计，甚至把一些思想落地到自己的agent开发中

像claude code和codex，它们本质上都是agent化的编程工具。它们把大语言模型包裹在一个应用层（也就是我们说的 Agent harness）里，从而在处理编程任务时更加顺手，表现也更好。

code agent是专门为软件开发打造的，在这之中，重要的可不仅仅是你选了哪个模型，更在于外围的配套系统——包括代码仓库的上下文、工具的设计、提示词缓存的稳定性、记忆能力，以及能够应对长时间连续工作的连贯性。（不知道大家有没有在claude code和codex中分别用同一个模

型，体验一下同一模型在不同code agent下的任务表现，应该就能发现，其实模型强是我们需要的，但一个好的code agent设计更是我们当前所急需的)

弄清楚这个区别很重要。因为当我们聊起大语言模型（LLM）时，大家往往会把模型本身、模型的推理行为和智能体产品混为一谈。

所以在探讨code agent到细节之前，我们先花点时间，简单梳理一下大语言模型、推理模型和智能体这三个概念之间的区别。

大语言模型、推理模型与智能体

大语言模型（LLM） 是核心，本质上它就是一个不断预测“下一个词”的模型。

推理模型（Reasoning Model） 其实也是 LLM，只不过它经过了特殊的训练或提示词引导，会在生成答案时投入更多的算力（即增加推理时间的算力消耗，**test-time compute**），用来做中间步骤的推理、自我验证，或者在多个候选答案中搜索最佳结果。

智能体（Agent） 则是盖在模型上面的一层，你可以把它理解为一个围绕模型运转的control loop。

通常情况是这样的：你给出一个目标，Agent层（或者说 Harness）就会替模型做决定：**接下来要检查什么？该调用哪个工具？怎么更新当前的状态？什么时候算是成果并且可以停下来？**

举个例子：

- LLM 像一个普通但合格的厨师，日常炒几个家常菜没问题，出菜也快。
- 推理模型像一个更强的大厨，做复杂菜、宴席菜、更讲究火候和步骤的菜会更稳，但也更耗时间、食材和成本。
- Agent harness 则不是厨师本身，而是整个后厨的调度系统：点单、备菜、分工、计时、查菜谱、调用烤箱和料理机、最后摆盘出菜，都是它在协调。

你当然可以让一个厨师单独做菜，就像直接调用 LLM 或推理模型。但如果要稳定地做一整桌菜，甚至根据客人的要求临场调整、补一道菜、检查有没有漏单，那就不是厨师够不够强这么简单了，而是要看后厨系统是不是成熟。所以，LLM 和推理模型决定谁来做菜、做菜能力有多强；Agent harness 决定这顿饭能不能有条理地做出来。

换句话说，**Agent**就是一个在特定环境里不断循环调用模型的系统。

总结一下：

- **大语言模型 (LLM):** 最原始的基础模型。
- **推理模型 (Reasoning model):** 优化过的 LLM，专门为了输出中间推理过程（也就是我们常说

的思维链 Chain of Thought) 和增强自我验证能力。【deepseek r1在2025年火的就是这个原因】

- *智能体 (Agent)*: 一个包含了“模型 + 工具 + 记忆 + 环境反馈”的循环系统。
- *Agent harness (智能体运行框架)*: 围绕智能体搭建的软件脚手架, 负责管理上下文、工具调用、提示词、状态和控制流。
- *Coding harness (编程运行框架)*: Agent harness 的“特化版”, 专门针对软件工程量身定制, 负责管理代码上下文、开发工具、代码执行和迭代反馈。

就像上面列出的, 在讨论智能体和编程工具时, 我们常会碰到这两个词: **Agent harness (智能体运行框架)** 和 **Coding harness (编程运行框架)**。Coding harness 是帮助模型高效编写和修改代码的软件脚手架。而 Agent harness 的适用面更广, 不仅限于编程 (比如 OpenClaw)。Codex 和 Claude Code 都可以算作 Coding harness。

总而言之: 更好的 LLM 能为推理模型打下更坚实的基础 (当然还需要额外的训练), 而优秀的 Harness 则能把推理模型的潜力压榨到极致。

当然, LLM 和推理模型光靠自己 (不用任何 Harness) 也能解决一些编程问题。

但是, 真实的写代码可不仅仅是大模型最擅长的预测下一个词。

开发工作中有很大一部分精力耗费在看代码、搜文档、找对应的方法名或函数名、diff、跑测试、排查报错, 以及在脑子里把所有这些信息串联起来。(这是个耗费心神的脑力活, 也是为什么大家在专注敲代码时很讨厌被打断, 以及程序员为什么都秃头orz)

看到这里, 你可以开始看后面的01-08的章节, 一步步思考, 怎么设计一个好的code agent

00-全局总览

Pico 全局架构导览：从一条请求到整个系统

这篇文档把大家把 `pico` 整个项目讲成一个能一口气读下来的系统。

如果你是第一次接触 `pico`，读完以后应该能回答这些问题：

- `pico` 到底是什么，不是什么
- 一条请求从终端进来以后，系统怎么往前推
- 为什么它不只是“模型 + 工具”这么简单
- `memory`、`checkpoint`、`resume`、`.pico/` 这些状态到底各干什么
- 每个核心模块在代码里是什么位置

如果你已经做过一轮开发，现在是回来复习，这篇文档可以帮你梳理一遍整体的架构

前言

`pico` 是接近一个面向代码仓库任务的本地 coding harness。模型当然重要，但模型只是其中一层。真正让它能连续工作的，是模型外面的几件事一起成立：

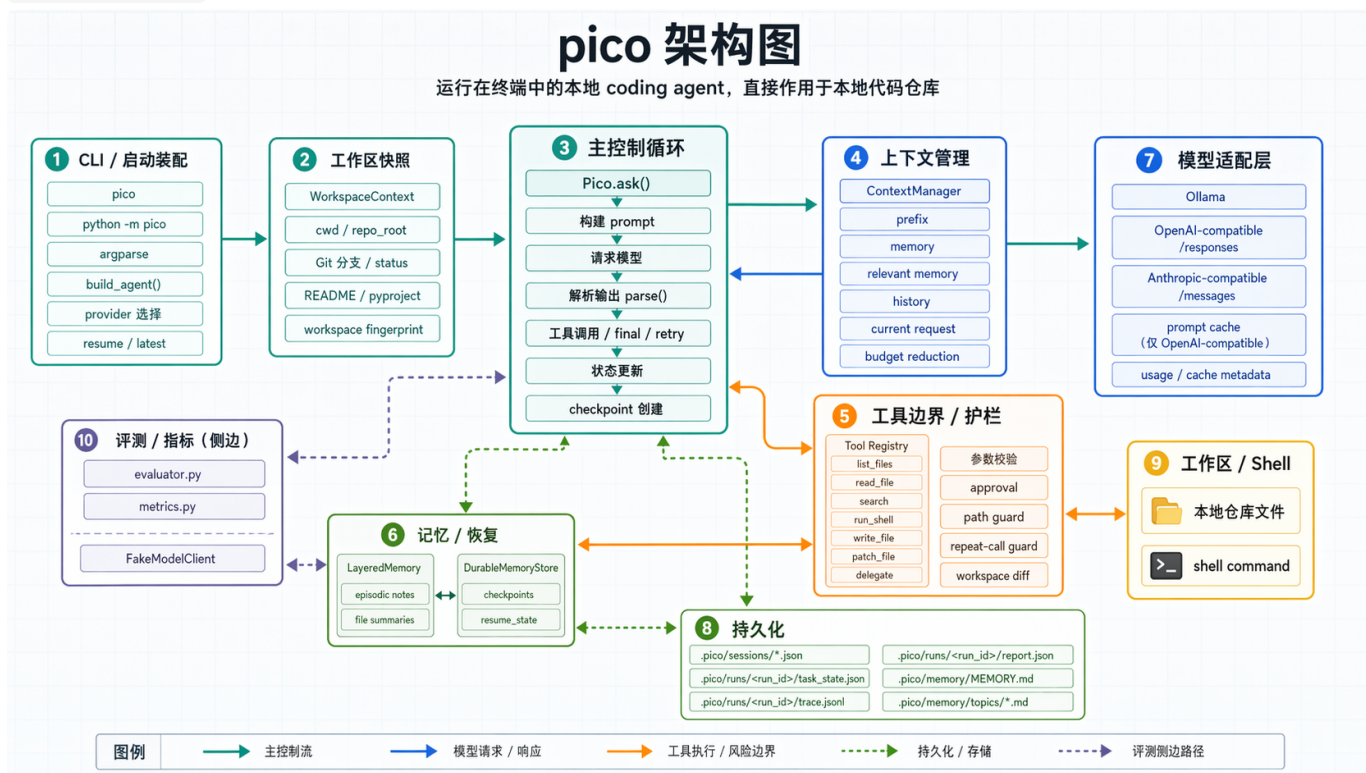
- 启动时先把工作区、模型后端和会话状态装配好
- 运行时有一条清楚的主控制循环
- 工具不直接执行，先经过统一边界（类似网关）
- 上下文不是乱塞进 prompt，是按 section 和预算组织
- 记忆、checkpoint、恢复和运行工件会一起把状态留住
- 评测和指标是验证这套系统有没有跑稳的证据链

`pico` 做的事，是把“模型在真实代码仓库里持续工作”这件事，从一次一答的聊天，变成一条有状态、有边界、能恢复、能复盘、能评测的执行链。

一、先看全貌



图片加载失败



一开始不用急着整个架构都塞进脑子里去想，可以先从这三条主线去开始看：

第一条是主控制流。它从 **CLI / 启动装配** 开始，进入 **Pico.ask()** 的主控制循环，再往两边接上 **上下文管理**、**模型适配层** 和 **工具边界 / 护栏**。这个部分回答的是，一条请求进来以后，系统怎么一步步往前推。

第二条是状态面。它把 **记忆 / 恢复** 和 **持久化** 放在一起。这里最关键的是这些状态各自解决不同问题。session 让系统还能继续干，run artifacts 让你知道刚才到底发生了什么，durable memory 则负责把长期还成立的事实留在会话外面。

第三条是证据面。**评测 / 指标** 这一块不是主请求路径的一部分，但它很重要。没有这层，很多“这个版本更好了”的说法都只能靠体感。有了固定 benchmark、FakeModelClient、metrics 和工件汇总，这套运行时才不只是能跑，还能被比较、被回看。

把这张总图压成更短的抽象，可以先记成下面三层：

层	作用	对应模块
控制面	决定系统怎么跑	cli.py、runtime.py、context_manager.py、tools.py、models.py
状态面	决定系统记住什么、恢复什么	memory.py、run_store.py、task_state.py、.pico/

证据面	决定系统怎么被验证、比较、复盘	<code>evaluator.py</code> 、 <code>metrics.py</code> 、 <code>trace.jsonl</code> 、 <code>report.json</code>
-----	-----------------	---

二、一条请求怎么跑完整个 Pico

顺着一条请求带大家梳理一遍

假设你在终端里输入：

```
1 uv run pico "inspect the test failures and propose a fix"
```

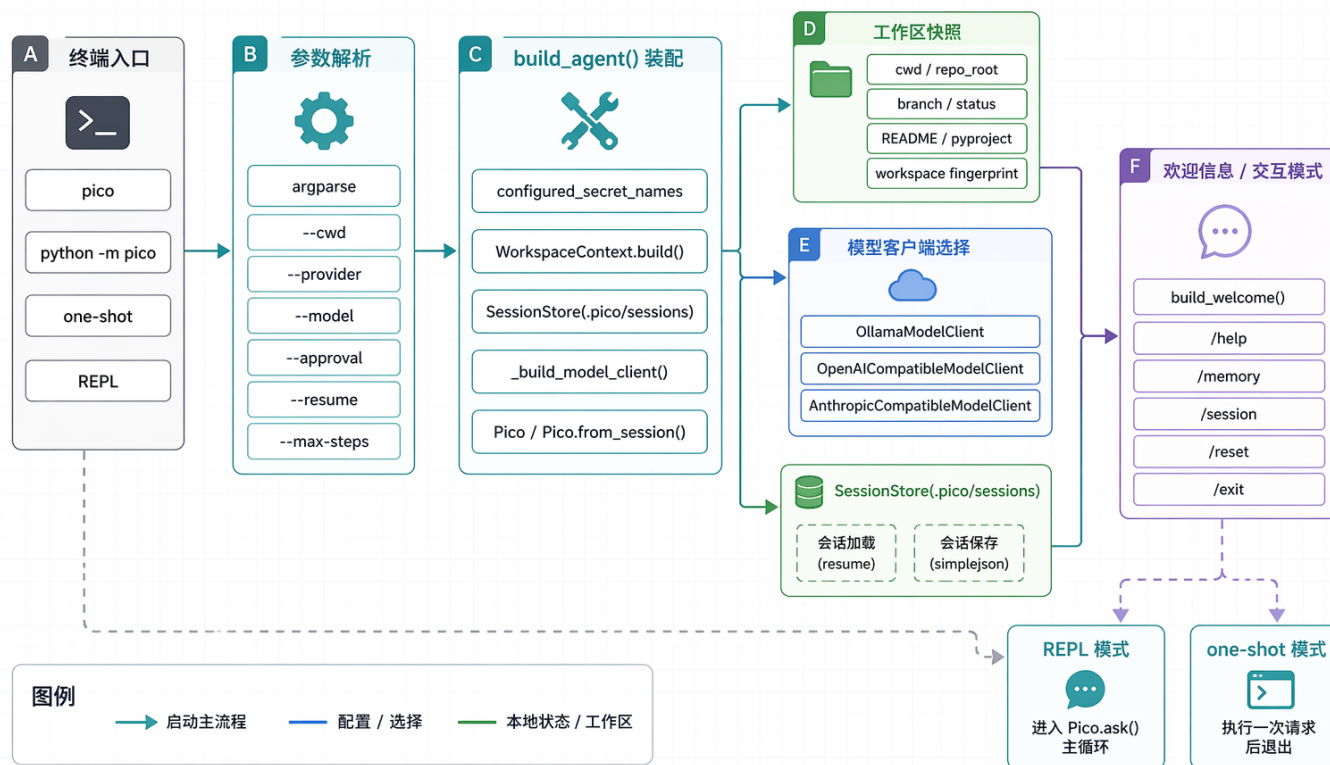
系统大概会经历下面这条线：

1. CLI 解析参数，决定工作目录、provider、model、approval、resume 模式。
2. `build_agent()` 把 `WorkspaceContext`、`SessionStore` 和具体 `ModelClient` 装好。
3. 如果是交互模式，就进入 REPL；如果是 one-shot，就直接把请求交给 `Pico.ask()`。
4. `Pico.ask()` 为这次请求创建 run、写第一版 `task_state.json`，再开始主循环。
5. 每一轮都会重新构建 prompt，把 prefix、memory、history 和 current request 拼起来。
6. 模型返回结果以后，runtime 先 `parse()`，判断这是工具调用、最终答案，还是一次需要重试的无效输出。
7. 如果是工具调用，就走 `run_tool()`，在统一边界里校验、审批、执行、比对工作区变化，再把结果写回状态。
8. 这轮发生过的事情会进入 session history、working memory、trace、checkpoint，必要时还会写 durable memory。
9. 直到模型给出 `<final>`，或者命中 step limit / retry limit，这次 run 才结束。

三、启动装配：先把运行现场搭起来

pico 模块图：启动装配

CLI 入口如何装配工作区、模型客户端与会话状态



启动装配主要负责，用户怎么启动 `pico`，最后为什么会落成一个已经能工作的 `Pico` 实例。

这里最关键的文件是 `pico/cli.py`。它把几个真正影响运行时的对象装配出来：

- 当前工作区的快照，也就是 `WorkspaceContext`
- 当前用哪个模型后端，也就是 `_build_model_client()`
- 当前 session 是新开还是恢复，也就是 `SessionStore`
- 当前 approval policy、max steps、resume 模式这些运行时约束

`pico` 的启动不是简单的先起个 REPL 再说，它会先把运行现场装好，再决定是 one-shot 还是 REPL。这让后面的行为更统一。REPL 和 one-shot 的区别只是交互方式不同，真正的运行时骨架是一套。

下面这段代码能直接看出这个装配点长什么样：

```

1 def build_agent(args):
2     configured_secret_names = _configured_secret_names(args)
3     workspace = WorkspaceContext.build(args.cwd)
4     store = SessionStore(workspace.repo_root + "/.pico/sessions")
5     model = _build_model_client(args)
6     session_id = args.resume
7     if session_id == "latest":
8         session_id = store.latest()

```

CLI 负责把用户怎么启动翻译成runtime 拿到什么对象图。

从这里往后，主循环不再关心命令行细节。

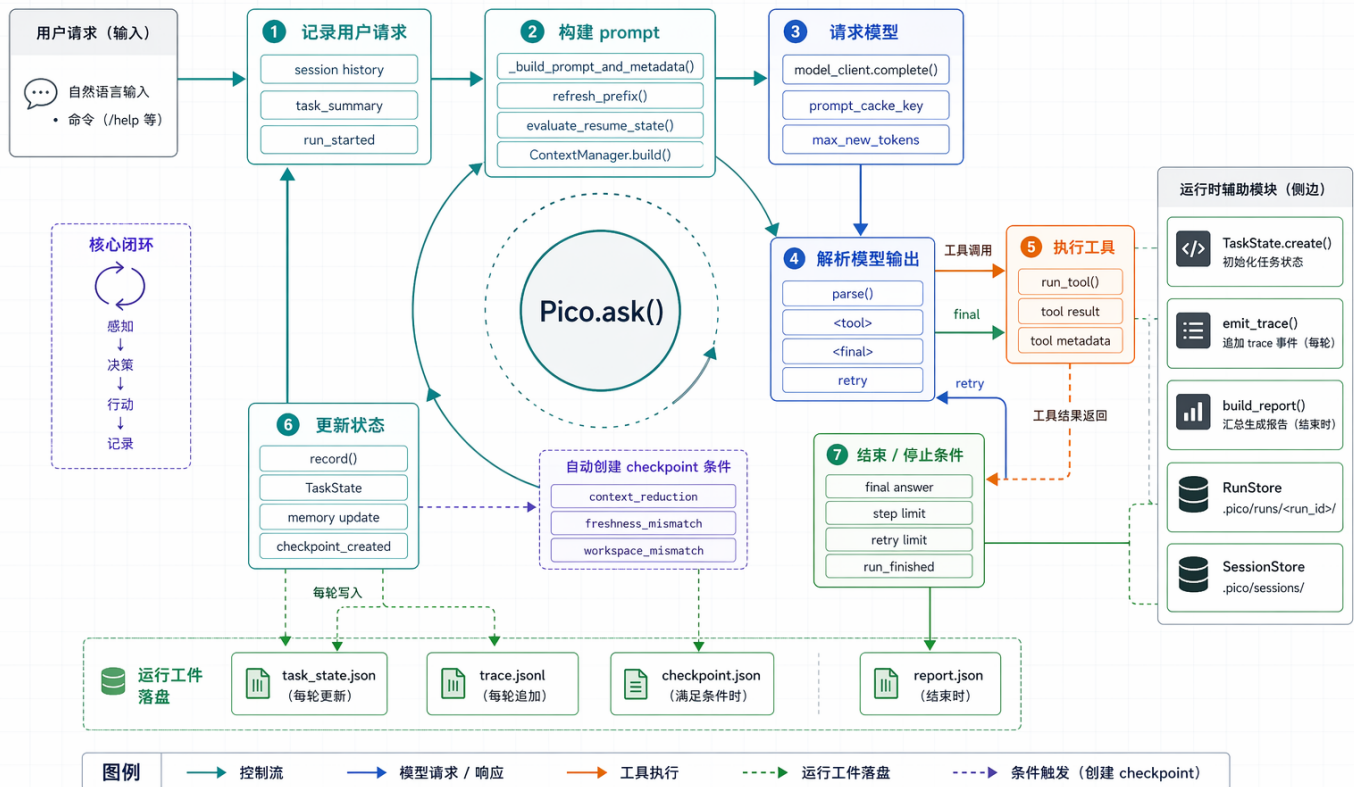
四、主控制循环：Pico.ask() 才是系统心脏



图片加载失败

pico 模块图：运行时主控制循环

Pico.ask() 如何在模型、工具、状态与落盘之间形成闭环



pico 真正的中心在 pico/runtime.py 的 Pico.ask()。

如果只看这一个函数，可以把它理解成四个动作不断循环：

- 感知：重新整理当前状态，构建这一轮 prompt
- 决策：请求模型，拿回一轮输出
- 行动：如果模型要调工具，就执行工具
- 记录：把这轮发生的事写回状态和工件

这哥agent loop值得大家反复去可能啊：

```
1 while tool_steps < self.max_steps and attempts < max_attempts:
2     attempts += 1
3     task_state.record_attempt()
4     prompt, prompt_metadata = self._build_prompt_and_metadata(user_message)
5     raw = self.model_client.complete(prompt, self.max_new_tokens, ...)
6     kind, payload = self.parse(raw)
```

主循环最容易被忽略的点有三个。

第一，`prompt` 不是在请求开始时构建一次就算了，而是每轮都重建。因为工作区、memory、history、checkpoint 都可能变，上一轮的工具结果也会直接影响下一轮 prompt。

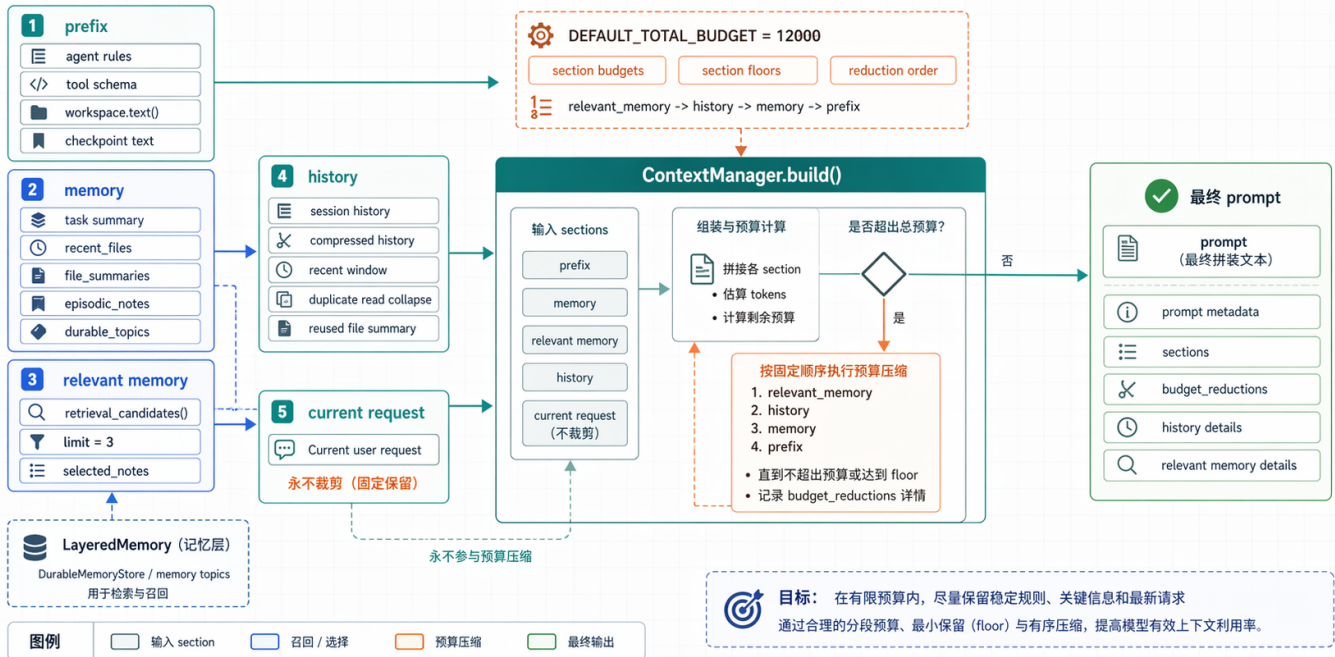
第二，模型输出不会直接进入执行阶段。中间有一个很关键的 `parse()`。这层负责把模型返回的文本接到控制流里，判断它到底是 `<tool>`、`<final>`，还是一次 malformed output，需要进入 retry。

第三，`Pico.ask()` 从第一轮开始就同时在做两件事，一边推进任务，一边生产证据。`task_state.json`、`trace.jsonl`、`report.json` 不是跑完以后补写的装饰，它们本来就是这条主循环的一部分。

五、上下文管理：不是把信息都塞进 prompt

pico 模块图：上下文管理与 Prompt 预算

ContextManager 如何组合 prefix、memory、history 与 current request



图片加载失败

很多 agent 一开始都栽在同一个地方，前几轮还好，跑到后面 prompt 越来越胖，历史越来越乱，真正该保留的东西和该丢掉的东西混在一起。

pico 的处理方式在 `pico/context_manager.py` 里。它不是粗暴截断，而是先把 prompt 拆成几段，再分别管：

- prefix
- memory
- relevant memory
- history
- current request

这几个 section 里，current request 的地位最高，默认不裁。因为最新请求一旦失真，后面所有优化都没有意义。

pico 对 prompt 做了预算控制。ContextManager 不只知道“总预算是 12000 chars”，还知道各 section 的初始预算、最低 floor，以及超预算以后先裁谁。当前默认顺序是：

```
1 relevant_memory -> history -> memory -> prefix
```

这套设计的价值，不在于 prompt 一定会更短，而在于系统知道自己在删什么。删的是旧历史，还是相关记忆，还是稳定前缀，这三件事的后果完全不一样。把这些 section 拆开以后，prompt 组装这件事就不再是黑盒。

从代码上看， `ContextManager.build()` 的输入是状态，输出不是单纯一个字符串，而是：

- 最终 prompt
- prompt metadata

这份 metadata 后面会进入 trace 和 report，所以你不只知道这一轮 prompt 长什么样，还知道它为什么长成这样。

六、工具边界：模型不能直接碰外部世界



对 code agent 来说，危险的地方在于调用工具之前有没有统一边界。

`pico` 这一层在 `pico/tools.py` 和 `pico/runtime.py` 里一起完成。`tools.py` 定义工具白名单和 schema，`runtime.py` 的 `run_tool()` 负责把所有工具执行都收口到一个边界里。

当前工具列表不算多：

- `list_files`
- `read_file`
- `search`
- `run_shell`

- `write_file`
- `patch_file`
- `delegate`

`run_tool()` 现在至少会做这些检查：

- 工具是不是已注册
- 参数是不是合法
- 路径有没有逃出工作区
- 是不是刚刚连续发起过完全相同的调用
- 高风险工具在当前 approval policy 下能不能执行

执行前后，runtime 还会对工作区做快照，比较这次动作到底改了什么。这样工具结果不只是一段返回文本，还会带上：

- `affected_paths`
- `workspace_changed`
- `tool_status`
- `tool_error_code`

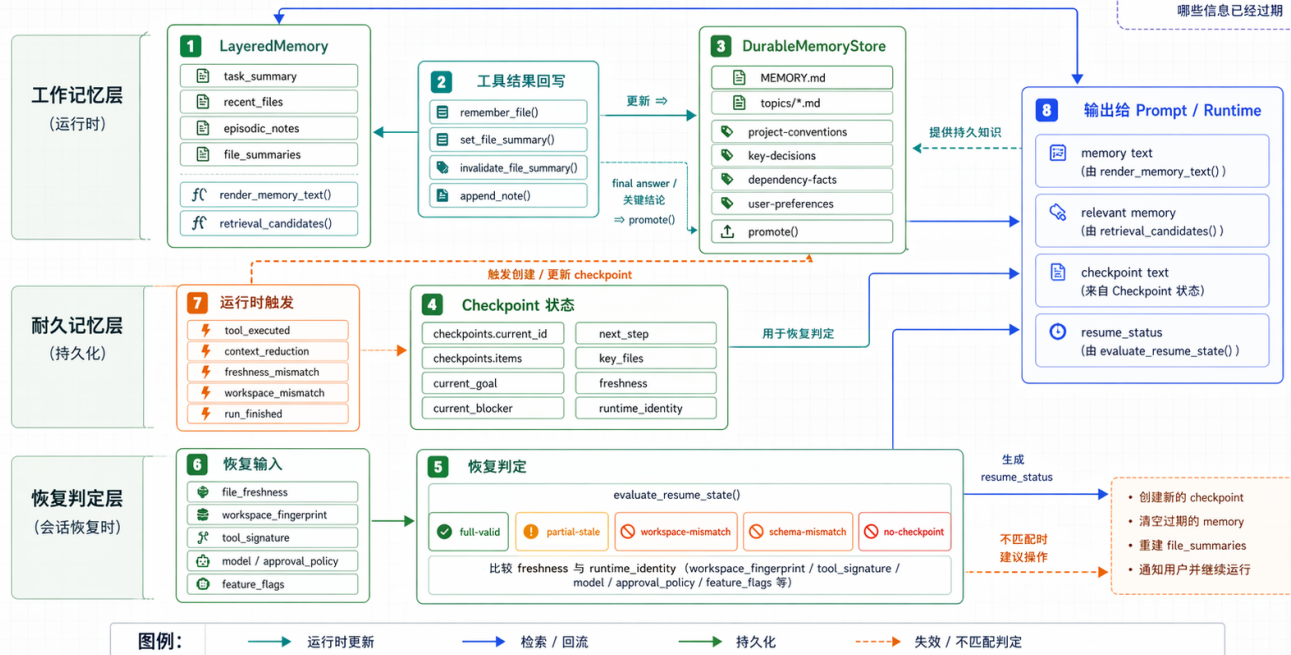
这一层的设计思想是模型不能直接碰外部世界，所有动作都必须经过统一边界。没有这个边界，后面的 memory、trace、benchmark 都会变得不可信，因为你已经不知道系统到底做过什么。

七、记忆、Checkpoint 与恢复：系统怎么持续工作

pico 模块图：记忆、Checkpoint 与恢复

LayeredMemory、DurableMemoryStore 与 resume_state 如何支持持续工作

目标：
既保留短期工作上下文，
也能在会话恢复时判断
哪些信息已经过期



图片加载失败

pico 现在的记忆不是单层结构，而是至少分成三部分：

- working memory，也就是 LayeredMemory
- durable memory，也就是 DurableMemoryStore
- checkpoint / resume_state，也就是会话恢复判断

LayeredMemory 负责当前工作面，里面主要是这些东西：

- task_summary
- recent_files
- file_summaries
- episodic_notes

它解决的是，模型是不是还在重复做刚做过的事，下一轮 prompt 还能不能拿回刚刚确认过的小结论。

DurableMemoryStore 负责长期仍然成立的事实。它不再继续塞在 session JSON 里，而是单独落到 .pico/memory/。这部分不是所有输出都自动写进去，而是要经过 promotion gate。比如项目约定、关键设计决策、依赖事实、用户偏好，这些才适合长期沉淀。

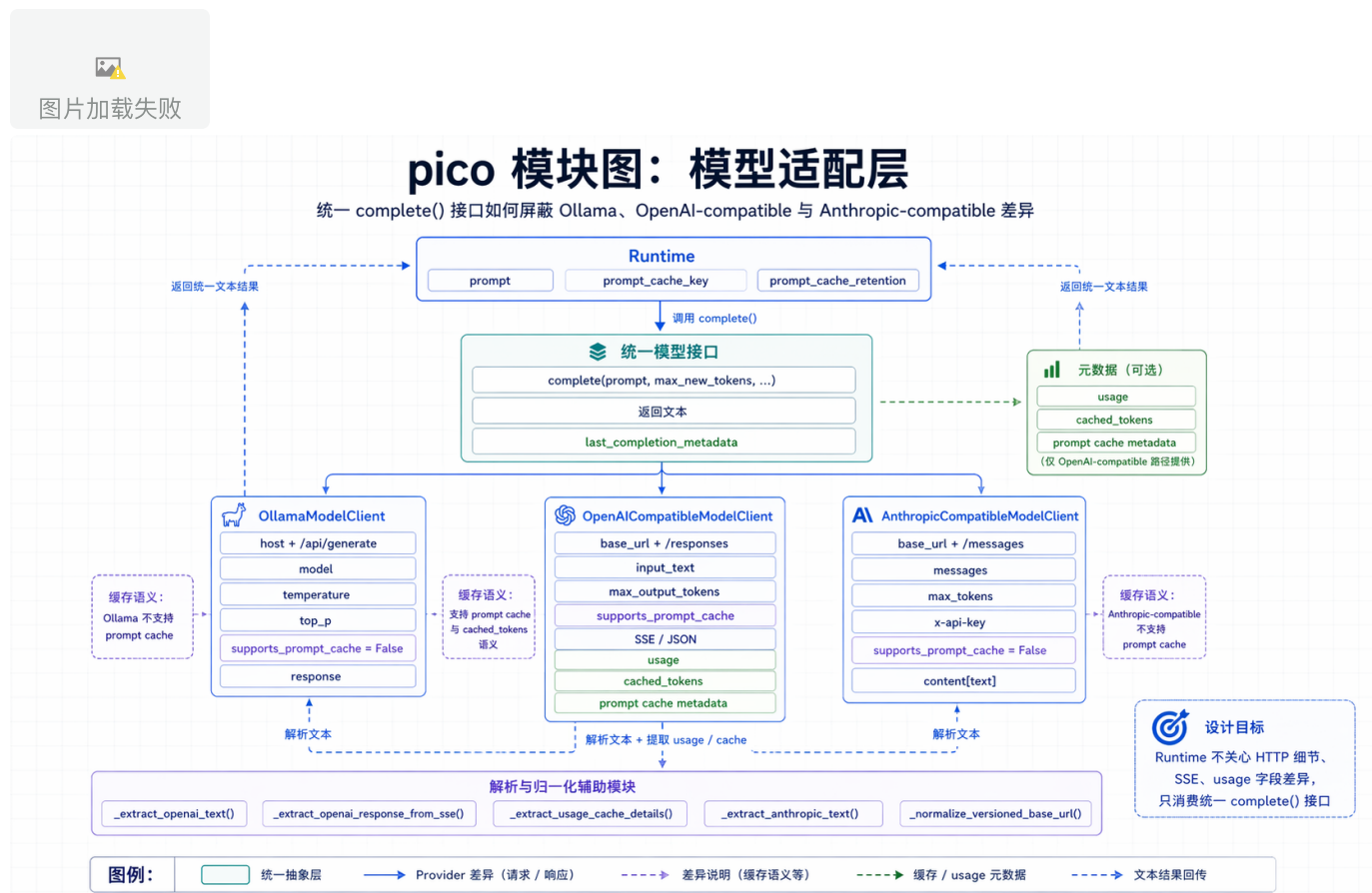
checkpoint 和 resume_state 解决的是另一个问题，这些旧状态现在还靠不靠谱。恢复的时候，runtime 会比较：

- 文件 freshness

- workspace fingerprint
- tool signature
- model / approval policy / feature flags

所以 `resume` 不是简单把旧 JSON 读回来，而是读回来以后，再判断哪些状态还是有效的。这是 `pico` 这套恢复机制最关键的地方。

八、模型适配层：Runtime 只想面对一个小接口



从 runtime 视角看，它对模型层的要求其实很克制。它不想知道 HTTP 细节，不想知道某个 provider 走 JSON 还是 SSE，也不想知道 usage 字段长什么样。它只想做这几件事：

- 给一段 prompt
- 指定 `max_new_tokens`
- 可选带上 prompt cache hint
- 拿回一段文本
- 顺手拿到少量 completion metadata

`pico/models.py` 的作用这么关键把三条 provider 路径都收成了统一的 `complete()` 行为：

- `OllamaModelClient`
- `OpenAICompatibleModelClient`
- `AnthropicCompatibleModelClient`

其中 OpenAI-compatible 这条线最复杂，因为它不只是普通 JSON，还要处理 `/responses`、SSE 事件流、usage 结构抽取和 prompt cache metadata。Anthropic-compatible 走 `/messages`，Ollama 走本地 `/api/generate`，两条线都简单一些。

如果大家要接新的模型，在这加就行

九、持久化与评测：能跑不够，还要能回看、能比较



pico 到这里已经不只是一个能调工具的 agent 了。它还得回答这些问题：

- 跑到一半中断了，下一次怎么接着来
- 跑完以后结果不对，怎么回看过程
- 两个版本谁更好，怎么稳定比较

这部分现在主要由三个模块负责：

- `SessionStore`

- `RunStore`
- `evaluator.py / metrics.py`

`SessionStore` 管的是以后还能不能继续干。它把会话级状态保存在 `.pico/sessions/*.json`。

`RunStore` 管的是刚才到底发生了什么。每次 `ask()` 都会新开一个 `run` 目录，里面至少有这三份工作：

- `task_state.json`
- `trace.jsonl`
- `report.json`

这三份工作解决的问题也不同。`task_state` 是运行中的快照，`trace` 是逐事件时间线，`report` 是收口后的聚合摘要。

评测侧边系统在 `pico/evaluator.py` 和 `pico/metrics.py` 里。它们不是主请求路径的一部分，但它们让 `pico` 从能跑走到了能稳定比较。固定 `benchmark`、`FakeModelClient`、`fixture repo`、`verifier`、`summary rows` 这些东西，最后一起回答的是同一个问题，你怎么知道这次版本真的变好了。

十、`.pico/` 在磁盘上到底长什么样

系统跑起来以后，状态最终会落在本地 `.pico/` 目录里。把这个目录想明白，很多抽象会一下子落地。

```

1  .pico/
2  |— sessions/
3     |— <session_id>.json
4  |— runs/
5     |— <run_id>/
6         |— task_state.json
7         |— trace.jsonl
8         |— report.json
9  |— memory/
10     |— MEMORY.md
11     |— topics/
12         |— project-conventions.md
13         |— key-decisions.md
14         |— dependency-facts.md
15         |— user-preferences.md

```

这棵树里，最容易混淆的是三件事。

`sessions/` 存的是会话级状态，它面向的是继续工作。

`runs/` 存的是单次请求的运行工件，它面向的是回放和审计。

`memory/` 存的是 durable memory，它面向的是跨会话仍然成立的长期事实。

十一、关键代码入口：想回到源码，先从这里看

如果你已经有了pico的系统抽象，下一步建议你回到几个关键文件验证这个抽象是不是对的。

可以先按下面这个顺序读：

文件	角色	为什么先看它
<code>pico/cli.py</code>	启动装配层	能看清 <code>pico</code> 怎么从命令行落成一個运行时对象图
<code>pico/runtime.py</code>	主控制循环	整个系统心脏， <code>Pico.ask()</code> 、 <code>run_tool()</code> 、 <code>checkpoint</code> 、 <code>report</code> 都在这里
<code>pico/context_manager.py</code>	prompt 组装层	看清 <code>section</code> 、预算和裁剪顺序
<code>pico/tools.py</code>	工具定义层	看白名单、 <code>schema</code> 和基本执行逻辑
<code>pico/memory.py</code>	记忆内核	<code>working memory</code> 、 <code>durable memory</code> 、 <code>retrieval</code> 都在这里
<code>pico/models.py</code>	模型适配层	看 <code>provider</code> 差异是怎么被吃掉的
<code>pico/run_store.py</code>	运行工件落盘	看 <code>run artifact</code> 为什么会分三类
<code>pico/task_state.py</code>	运行状态快照	看一轮请求到底被记录成什么
<code>pico/evaluator.py</code>	benchmark harness	看 <code>benchmark</code> 任务、 <code>fixture</code> 、 <code>verifier</code> 怎么接进 <code>Pico</code>
<code>pico/metrics.py</code>	指标聚合	看 <code>report</code> 最后怎么变成 <code>summary rows</code> 和 <code>pass rate</code>

如果你只剩 15 分钟，优先看四个文件：

- `pico/runtime.py`
- `pico/context_manager.py`
- `pico/memory.py`
- `pico/models.py`

这四个文件基本已经把 `pico` 的系统骨架撑起来了。

复习版：10 分钟把 Pico 过一遍

如果你以后想快速回顾，按这个顺序复习就够了：

1. 先看中文总图，重新把主控制流、状态面、证据面放回脑子里。
2. 再看启动装配图和运行时主循环图，确认一条请求是怎么跑起来的。
3. 然后看上下文管理图和工具边界图，记住 prompt 和工具都不是黑盒。
4. 再看记忆 / 恢复图，确认 working memory、durable memory、checkpoint 的分工。
5. 最后看模型适配层和持久化 / 评测图，把 provider 差异和 run artifacts 接回整个系统。

如果你能顺着这条线把系统讲出来，说明 `pico` 你已经理解的很透彻了。

01-代码仓库上下文设计

背景

这部分内容解决一个问题：code agent 进到代码仓库以后，首先应该拿到什么上下文。

在使用code agent到时候用户经常只会给一句很短的话，比如修一下测试、加个接口、看下这里为什么挂了。

对人来说，这种话还能靠经验脑补上下文。

对模型不行。它如果连当前目录、仓库根目录、分支、未提交改动、关键项目文档都不知道，很容易就出问题。

Pico 在这里的判断很直接，先给模型一份便宜、稳定、足够支撑首步判断的仓库基线，然后再让它自己通过工具去查细节。这样第一步能站稳，后面也还有继续探索的空间。

这篇要解决什么问题

问题可以压成一句话：

在不预加载整仓代码的前提下，怎么让模型先知道自己现在身处什么仓库。

这里有两个约束：

- 全仓预加载很贵，prompt 很快会被吃满
- 只给用户原话又太少，模型连从哪开始读都不知道

所以仓库上下文这一层要做的事很明确，先回答这几个最影响首步判断的问题：

- 我现在在哪个目录
- 这里是不是 Git 仓库
- 当前在哪个分支
- 工作区是不是 dirty
- 最近改过什么
- 项目里有没有会直接改变行动方式的文档

这层信息够了，模型就能决定下一步是先读 **README**、先看测试、先搜入口，还是先确认分支和改动范围。

先给模型什么

当前 `Pico` 给的是一份小的工作区摘要，也可以把它理解成仓库基线。

它包含这些字段：

字段	来源	为什么要带
<code>cwd</code>	启动目录	告诉模型当前工作位置
<code>repo_root</code>	<code>git rev-parse --show-toplevel</code>	确定仓库边界和工具作用范围
<code>branch</code>	<code>git branch --show-current</code>	判断当前工作分支
<code>default_branch</code>	<code>refs/remotes/origin/HEAD</code>	方便理解主线分支和偏移语境
<code>status</code>	<code>git status --short</code>	判断有没有未提交改动
<code>recent_commits</code>	<code>git log --oneline -5</code>	给近期变更一点背景
<code>project_docs</code>	白名单文档扫描	把高价值项目文档先带进上下文

这张表看起来很普通，但它抓的都是第一步最值钱的信息。

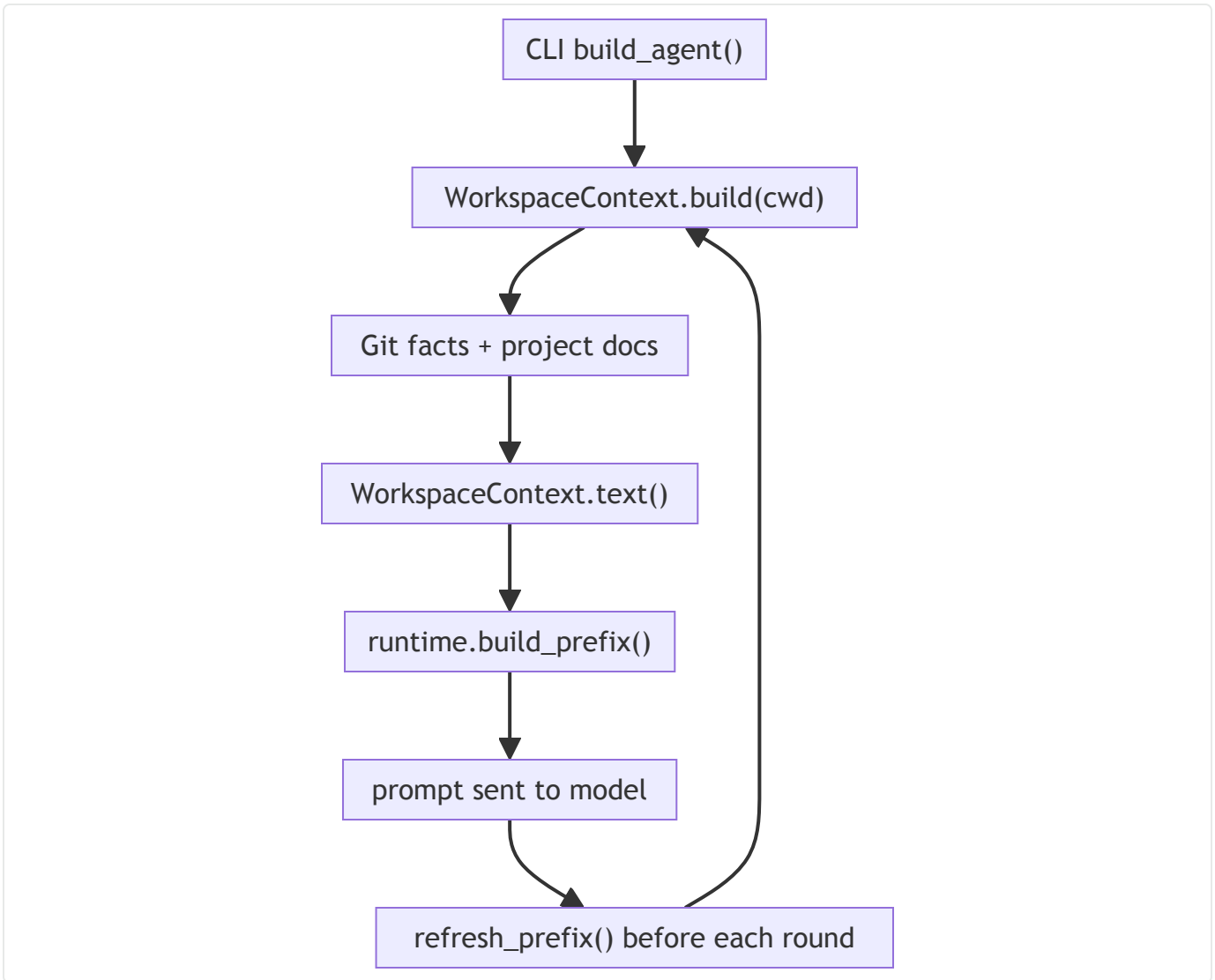
模型刚进仓库时，知道得太少会乱走，知道得太多会浪费预算。

`Pico` 选取了一个折中的方案。

代码里怎么做

入口在 `pico/cli.py` 。 `build_agent()` 会先跑一次 `WorkspaceContext.build(args.cwd)` ，把这份仓库基线收集出来，再把它交给 runtime。

核心路径可以看成下面这条线：



真正采集逻辑在 `pico/workspace.py`，有两部分最关键。

第一部分是白名单文档。当前只看四类文件：

```
1 DOC_NAMES = ("AGENTS.md", "README.md", "pyproject.toml", "package.json")
```

第二部分是采集方式。它会同时看 `repo_root` 和 `cwd`，这样从子目录启动时，也能顺手把本地文档带上。

```

1 for base in (repo_root, cwd):
2     for name in DOC_NAMES:
3         path = base / name
4         if not path.exists():
5             continue
6         key = str(path.relative_to(repo_root))
7         if key in docs:
8             continue
9         docs[key] = clip(path.read_text(encoding="utf-8", errors="replace")
, 1200)

```

这里有三个设计点值得注意：

- 扫描范围很小，只看白名单，不扫整仓
- 扫描位置同时覆盖 `repo_root` 和 `cwd`
- 每份文档都会截断，避免某一份长文档直接把 prompt 空间吃掉

这样设计的核心思想是先把最可能改变行动路径的事实送进去。

为什么这几类文档对模型来说最重要

当前白名单里的四类文件，作用都很直接。

- `AGENTS.md` 通常会规定工作方式、测试方式、提交约束
- `README.md` 经常会交代项目结构、启动命令和目录入口
- `pyproject.toml` 很容易暴露 Python 项目结构和工具链
- `package.json` 能快速说明这是不是 Node 项目、脚本入口在哪

这些文件不一定全面，但很适合当第一眼材料。

模型看完以后，至少知道自己应该往哪边查，而不是在仓库里盲找。

这份仓库基线怎么进 prompt

采集完以后，`WorkspaceContext.text()` 会把这些字段渲染成一段稳定文本。runtime 再把它放进 prefix。

`pico/runtime.py` 里的 `build_prefix()`，会把三类内容拼在一起：

- agent 的行为规则
- 工具清单和调用格式
- 工作区摘要，也就是 `self.workspace.text()`

这样模型在第一次输出前，已经拿到了这几个最低限度的判断依据：

- 当前仓库根目录在哪
- 自己处在哪个分支
- 工作区有没有未提交改动
- 最近发生过哪些提交
- 哪些项目文档值得先看

对面试来说，这里有个很好讲的点。很多人会把 prompt 理解成一整块文本，但在 agent runtime 里，prefix 更像工作手册。仓库基线能放进 prefix，是因为它相对稳定，而且每轮都可能还会继续有用。

它什么时候刷新

`Pico` 启动时会先构建一次 `WorkspaceContext`。后面每轮 prompt 组装前，`runtime._build_prompt_and_metadata()` 还会先调用 `refresh_prefix()`。

`refresh_prefix()` 做了两件事：

1. 重新执行 `WorkspaceContext.build(self.root)`，拿到最新仓库状态
2. 用 `fingerprint()` 比较新旧工作区摘要，只有在仓库状态真的变了时，才重建 prefix 文本

所以更准确的说法是：

- 工作区状态会在每轮 prompt 前重新采样
- prefix 文本不会无脑重建，只有 fingerprint 变了才重建

这个设计挺像一个轻量缓存。它保住了实时性，也控制住了每轮都重拼 prefix 的额外开销。

为什么不直接预加载整个仓库

这个问题面试里很常见，因为直觉上大家都觉得知道越多越好。

真放到 runtime 里，事情没那么简单。

第一，成本不划算

很多任务第一步根本不需要看代码细节，只需要知道项目类型、仓库状态和关键文档。这个阶段把整仓内容塞进去，花掉的是上下文预算，换来的不一定是更高成功率。

第二，行为不稳定

白名单摘要很稳定。你大概知道这轮 prompt 会进什么，长度也比较可控。整仓预加载会让 prompt 内容随着仓库变化大幅波动，排查问题时很难知道到底是哪部分信息把模型带偏了。

第三，探索责任会变模糊

`Pico` 的思路是先给方向，再让模型自己走工具链。这样路径边界、读取动作和副作用都还在 runtime 的治理里。平台如果先替模型把整仓读完，模型可见的信息会更多，但这不等于第一步判断会更准，反而可能因为上下文变长和噪声增加，让真正重要的线索被稀释。

所以这里的取舍很明确，仓库上下文这一层先做导航，不做知识库。

当前设计的边界

这层设计已经够用，但边界也很明确。

还没有目录树视图

当前工作区摘要里没有目录结构，也没有热点模块地图。

模型第一次决定从哪读起，更多还是靠文档和工具自己补。

文档发现偏保守

当前只看 `AGENTS.md`、`README.md`、`pyproject.toml`、`package.json`。这对很多项目够用，但像 `Makefile`、`justfile`、`pytest.ini`、`Cargo.toml`、`go.mod` 这些高价值入口文件还没纳入。后续还可以考虑这点。

Git 视角还比较轻

现在有分支、状态和最近 commit

但还没有改动规模、冲突状态、stash、upstream 偏移这种更细的仓库态信息。

这些边界不影响第一版成立，但它们会直接影响 agent 进复杂仓库时的第一步判断速度。

如果继续往下做，优先补什么（面试时候经常问）

我会优先补三件事。

一，补轻量目录摘要

不要整仓索引，先补一份受预算约束的目录视图，比如根目录前两层、测试目录、脚本目录、主要语言分布。这会明显改善模型第一次选入口的速度。

二，扩文档白名单

把 `Makefile`、`justfile`、`pytest.ini`、`tox.ini`、`Cargo.toml`、`go.mod` 这类文件加进候选集，效果通常比继续加自然语言说明更直接。

三，补 Git diff 摘要

只靠 `git status --short` 还不够。哪怕只是加一层改动文件数、增删行规模、改动集中目录，模型接续当前分支工作的判断都会更快。

面试里怎么讲这一块设计

代码 agent 进仓库后的第一步，是先拿一份仓库基线。Pico 这里收的都是第一步最值钱的稳定事实，比如 `cwd`、`repo root`、`branch`、`dirty status`、`recent commits` 和少量关键项目文档。它们会被渲染进 `prefix`，模型一开始就知道自己在哪、仓库当前是什么状态、应该优先看哪些入口。后面每轮 `prompt` 前还会重新采样工作区状态，用 `fingerprint` 控制 `prefix` 是否重建。首步判断的稳定性和上下文成本，都能一起管住。

总结

Pico 这层仓库上下文设计，核心是让模型先站在一个正确的位置上开始行动。

工作区摘要解决的是第一步的定位和起点判断。它很小，但足够告诉模型当前仓库是什么状态、哪些入口最值得先看。后面的文件阅读、字符串搜索、测试执行和记忆回写，都是在这层基线之上继续往前走。

02-运行时主循环与执行编排设计

背景

如果只看 `tools.py`、`memory.py`、`context_manager.py` 这些模块，很容易把项目理解成几个零散能力的组合。真实运行时不是这么工作的。用户给一句话之后，系统要先建状态、再组 prompt、再调模型、再接工具、再落工件、最后决定什么时候停。把这些动作串起来的，就是主循环。

对面试来说，这一层很关键。因为很多人能讲清工具、能讲清 memory、也能讲清 prompt，但一问到一句请求进来以后系统到底怎么往前推，就开始散。

这篇的任务，就是把这条执行线讲清楚。

这篇要解决什么问题

问题可以压成一句话：

一条用户请求，怎么被 runtime 推进成一次有边界、有状态、可回放的 agent 执行。

这里面有五个子问题：

- 这轮请求什么时候算真正开始
- 模型一轮一轮是怎么被调起来的
- 模型说要调工具以后，runtime 怎么接住
- 执行过程中哪些状态会持续更新
- 什么情况下继续跑，什么情况下停

从一句话到一次 run

`Pico` 对一次请求的处理单位，是一轮独立运行。

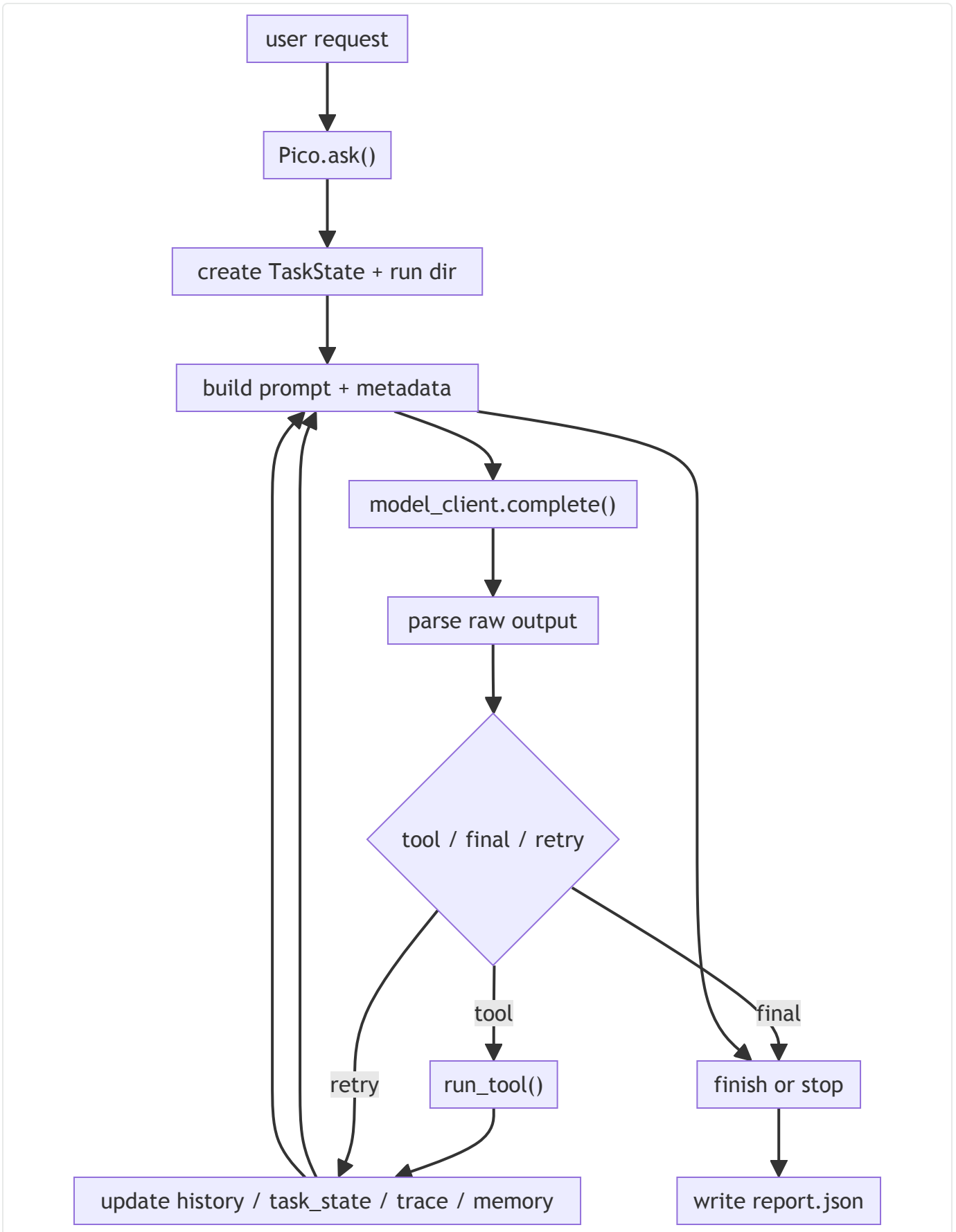
CLI 在 `pico/cli.py` 里先把现场装好，包括：

- 用哪个 provider
- 用哪个 model
- 当前工作区是什么
- session 是新开还是恢复
- 高风险工具怎么审批

- 最多允许跑几步

这些东西定完以后，运行权就交给 `agent.ask(user_message)` 。

从 runtime 的视角看，一次请求会走下面这条线：



这张图需要注意两个地方。

第一，`ask()` 不只是调模型，它是在驱动一条完整执行链。

第二，工具执行并没有把控制流带出 runtime，结果还会重新回到状态里，再进入下一轮。

ask() 一开始先做什么

`pico/runtime.py` 里的 `ask()`，开头先立运行现场，再进入推理和执行。

它会按顺序做这些事：

1. 把当前请求写进 `task_summary`
2. 把用户消息记进 session history
3. 创建 `TaskState`
4. 在 `.pico/runs/<run_id>/` 下创建本轮 run 目录
5. 立刻写出第一版 `task_state.json`
6. 追加一条 `run_started` trace

代码骨架大概就是这样：

```
1 def ask(self, user_message):
2     self.memory.set_task_summary(user_message)
3     self.record({"role": "user", "content": user_message, "created_at": now()})
4
5     task_state = TaskState.create(
6         run_id=self.new_run_id(),
7         task_id=self.new_task_id(),
8         user_request=user_message,
9     )
10    self.current_task_state = task_state
11    self.current_run_dir = self.run_store.start_run(task_state)
12    self.emit_trace(task_state, "run_started", {...})
```

这个设计把一次请求和一次 run 绑定死了。

后面你查问题、做实验、回放 trace，都不需要再从一整段 session history 里猜边界。

主循环到底在循环什么（重要）

`ask()` 的 while 循环可以压成四个阶段：

- 感知：把当前状态重新整理成 prompt
- 决策：调模型，拿回一轮输出

- **落地**：如果模型要调工具，就执行工具
- **记录**：把这轮发生的事写回状态和工作

核心骨架可以看这一段：

```

1  while tool_steps < self.max_steps and attempts < max_attempts:
2      attempts += 1
3      task_state.record_attempt()
4      self.run_store.write_task_state(task_state)
5
6      prompt, prompt_metadata = self._build_prompt_and_metadata(user_message
7      )
8      raw = self.model_client.complete(prompt, self.max_new_tokens, ...)
9      kind, payload = self.parse(raw)
10
11     if kind == "tool":
12         task_state.record_tool(name)
13         result = self.run_tool(name, args)
14         self.record({...})
15         self.emit_trace(task_state, "tool_executed", {...})
16         continue
17
18     if kind == "retry":
19         self.record({...})
20         continue
21
22     task_state.finish_success(final)
23     self.run_store.write_report(task_state, self.build_report(task_state))
24     return final

```

这里有两个状态字段要注意：

- `attempts` 统计模型被调了多少轮
- `tool_steps` 统计真正进入执行阶段的工具调用次数

一个任务 `attempts` 很高但 `tool_steps` 很低，通常是模型输出格式不好或者一直在 `retry`。

`tool_steps` 很高，说明它真的在执行很多动作。

为什么 prompt 每轮都要重建

runtime 每轮都会重新走 `_build_prompt_and_metadata()`。

主要原因是，下面几个东西每一轮都可能变：

- history

- memory
- workspace 状态
- 工具数和 prefix 状态
- 当前请求对应的相关上下文

所以 prompt 在 runtime 里不是一份固定文本，更像当前执行状态的投影。 `_build_prompt_and_metadata()` 除了组 prompt，还会顺手带出一份 metadata，里面会记：

- prefix 长度
- workspace 长度
- history 长度
- tool 数量
- prefix hash
- workspace fingerprint
- prompt cache key

这些元数据对主循环很重要，后面 trace、report、实验聚合都要靠它们解释这一轮 prompt 到底发生了什么。

模型输出怎么进入控制流

模型返回的是文本，runtime 需要的是动作。中间这一步由 `parse()` 来做。

`parse()` 会把一轮模型输出压成三类结果：

- `tool`
- `final`
- `retry`

它支持两类工具调用格式：

- `<tool>...</tool>` 里包 JSON
- XML 风格 `<tool name="...">...</tool>`

核心逻辑可以直接看这一段：

```

1  ▾ if "<tool>" in row:
2      payload = json.loads(body)
3      return "tool", payload
4
5  ▾ if "<tool" in row:
6      payload = Pico.parse_xml_tool(row)
7  ▾   if payload is not None:
8      return "tool", payload
9
10 ▾ if "<final>" in row:
11     return "final", final
12
13 ▾ if row.strip():
14     return "final", row.strip()
15
16     return "retry", Pico.retry_notice(...)

```

模型说了什么不重要，runtime 关心的是接下来该走哪条分支。

工具调用怎么重新接回主循环

当 `parse()` 返回 `tool` 之后，主循环不会直接跳到具体工具函数，而是统一先走 `run_tool()`。

`run_tool()` 负责把执行前后的关键动作串起来：

1. 检查工具是不是已注册
2. 校验参数是否合法
3. 判断是不是连续重复调用
4. 高风险工具走审批
5. 真正执行工具
6. 把结果裁剪后返回
7. 更新 memory 里的局部事实

工具执行没有脱离主循环。工具结果回来以后，会继续写回这些状态：

- session history
- `TaskState`
- trace
- report 使用的最后一轮 metadata
- memory 里的轻量事实

这样 runtime 才能在下一轮继续推进，而不是每次工具调用都像一条临时岔路。

TaskState 为什么是主循环的中心状态

`pico/task_state.py` 里的 `TaskState` 很值得单独讲，因为它把这轮运行的中心状态收在了一个地方。

它主要记录这些字段：

- `run_id`
- `task_id`
- `status`
- `tool_steps`
- `attempts`
- `last_tool`
- `stop_reason`
- `final_answer`

这里最值钱的设计是 `status` 和 `stop_reason` 分开存。

`status` 回答的是这轮现在是什么状态，比如

`running`、`completed`、`stopped`、`failed`。

`stop_reason` 回答的是它为什么停，比如 `final_answer_returned`、`step_limit_reached`、`retry_limit_reached`。

这两个维度分开以后，后面做 benchmark 和失败归因时，就不会把所有停机都混成一个模糊的失败。

RunStore 为什么要跟主循环绑在一起

如果主循环只返回一个最终答案，很多工程问题都没法解释。

`Pico` 这里把工件落盘直接绑进主循环，靠的是 `RunStore`。

每次 run 至少会写三类工件：

- `task_state.json`
- `trace.jsonl`
- `report.json`

这三类工件分工很清楚。

- `task_state.json` 负责看当前这轮跑到哪了

- `trace.jsonl` 负责看过程里发生了什么
- `report.json` 负责看最后结果和关键指标

这也是为什么 `ask()` 看上去比普通 demo agent 重一点。它不是只求跑通，还要把每一步都留证据。

循环什么时候停

当前主循环的停止条件很清楚，主要有三类。

一，模型给出最终答案

这时 `task_state.finish_success(final)`，状态会进入 `completed`，然后写 report，函数直接返回。

二，达到工具步数上限

这时 runtime 会返回一条停止说明，并把 `stop_reason` 记成 `step_limit_reached`。

三，达到重试上限

这一般意味着模型连续多轮输出了坏格式、空响应或者没法进入有效动作。

runtime 会把它记成 `retry_limit_reached`。

在面试的时候你可以这么说：runtime 关心的不只是成功路径，也关心怎么有边界地停下来。

当前设计的trade-off

这一版 runtime 很克制，它减少了不少复杂度。

减少了 planner 和多级调度

当前主循环就是单循环，没有额外 planner，也没有复杂 phase machine。

这样做的好处是调试简单，控制流清楚，trace 也更容易看。

减少了花哨编排

工具调用、模型调用、状态回写都还在一个明确的总调度器里。

复杂度集中，坏处是 `ask()` 会显得比较重，好处是主链路很完整。

换来了可回放和可解释

`prompt_metadata`、`TaskState`、`trace`、`report` 这些东西都让代码更厚，但这部分厚度换来了 runtime 级别的解释能力。你可以说清楚这轮为什么停、调了几次模型、用了几步工具、最后留下了什么证据。

对一个本地 coding harness 来说，这个取舍是合理的。主循环先做稳，后面别的能力才有地方挂。

对于runtime，还有什么能改进的地方，如果继续开发的话

我会优先补三件事。

一，细化失败类别

当前已经区分 final、retry、tool，也有 step limit 和 retry limit。

再往下做，最值得补的是把模型空响应、解析失败、后端异常拆得更开，这样 report 和评测会更干净。

二，补更显式的阶段字段

现在 trace 是事件序列，可读性已经够用。

如果以后想做可视化或复杂评测，runtime 可以再加一层统一的 phase 标记。

三，补父子 run 关系

现在 `delegate` 已经能工作，但在主循环里还是被当成一种特殊工具。

后面如果子 agent 变多，父子 run 的关系值得被单独建模。

面试里怎么讲runtime相关的内容

Pico 的 runtime 主循环负责把一条用户请求推进成一次独立 run。`ask()` 开头先建

`TaskState`、run 目录和第一条 trace，然后每轮都重新组 prompt、调模型、解析输出。如果模型要调工具，统一先过 `run_tool()`，工具结果回来以后再写回 history、task state、trace 和

memory，继续下一轮。停机条件也很清楚，要么拿到 final answer，要么命中 step limit 或 retry limit。这样一轮请求从开始到结束都会留下工件和状态，不会只剩一句最后答案。

总结

Pico 的运行时主循环，做的是把一句用户请求推进成一次有状态、有边界、可回放的执行。

这条主线把 prompt、model、tool、state、artifact 全部接到了同一个控制流里。前面的仓库基线、后面的工具治理、记忆系统、缓存和审计，最后都要挂在这条线上，整个 agent 才像一个真正能持续工作的 runtime。

03-工具的接入与调用设计

背景

工具层是 **Pico** 里很容易被误解的一层。

很多人一说 agent 会用工具，注意力就会跑到能力扩展上。能读文件、能搜字符串、能跑 shell、能改文件，听起来当然比纯聊天更强。在工程里，**更麻烦的是这些动作怎么被约束住**。

模型一旦拿到文件写入和 shell 执行能力，问题就不再只是答得对不对了。更实际的问题是，它会不会在错误路径上真的动手，会不会反复做无效调用，会不会越过工作区边界，会不会把不该带进去的环境信息带进命令执行。

对于工具的重点不放在“Pico 接了哪些工具”，重点放在**工具层为什么必须做成一条受控执行链**。

解决什么问题

问题可以压成一句话：

模型已经有动作能力了，runtime 为什么还要把这些动作锁进一条有边界的执行链。

这里真正要回答的是四件事：

- 模型到底能做什么动作
- 这些动作是怎么暴露给模型的
- 一条工具调用在执行前会被拦哪些东西
- 为什么这些护栏比“多接几个工具”更重要

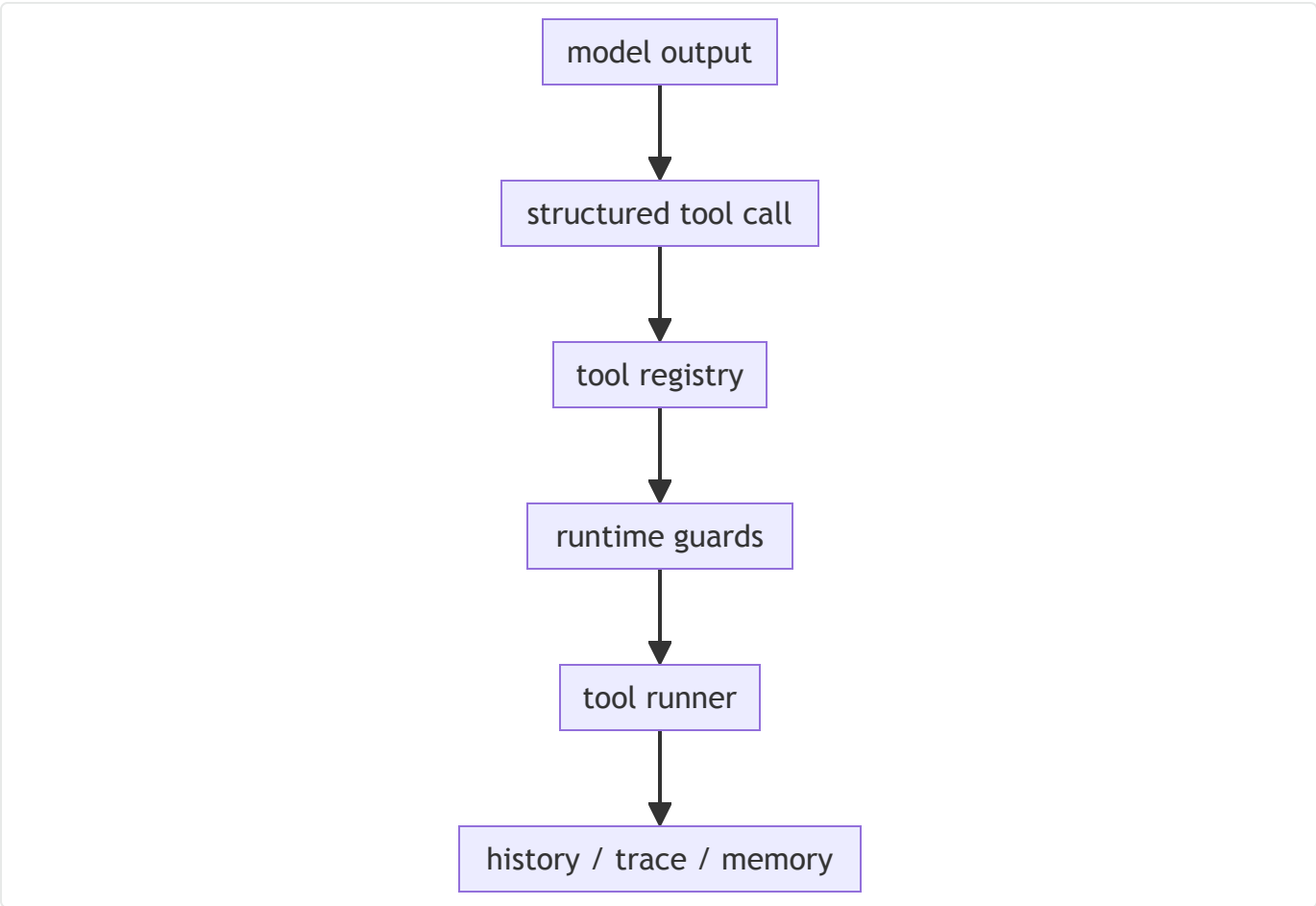
如果这几件事讲不清楚，工具层就很容易被看成一张工具清单。**Pico** 在这里做的是信任边界。

工具层先解决什么问题（面试重要）

从 runtime 视角看，工具层要先解决三个问题。

- **模型不能随便发明动作，动作集合得是显式的**
- **模型不能只表达“我想做点什么”，它得把工具名和参数交代清楚**
- **runtime 不能看见动作就直接执行，得先过安检**

这三步连起来，才有后面的“模型会用工具”。



这张图可以这样理解，模型先把动作说清楚，runtime 再判断这个动作是不是已知的、参数是不是合法、风险是不是可接受。前面都过了，才会真的执行。

工具到底是怎么暴露给模型的

工具定义在 `pico/tools.py` 。

`Pico` 没有走动态发现，也没有开放一个任意注册的工具市场。

当前版本就是一组显式白名单：

工具	作用	风险级别
<code>list_files</code>	列目录	低风险
<code>read_file</code>	读文件	低风险
<code>search</code>	搜字符串	低风险
<code>run_shell</code>	跑 shell	高风险
<code>write_file</code>	写文件	高风险

patch_file	精确替换文本	高风险
------------	--------	-----

除此之外，还有一个条件注册的 `delegate`。当前 agent 深度还没超过上限时，它才会出现在工具表里。

注册表代码本身很直白：

```

1 ▾ BASE_TOOL_SPECS = {
2     "list_files": {...},
3     "read_file": {...},
4     "search": {...},
5     "run_shell": {...},
6     "write_file": {...},
7     "patch_file": {...},
8 }
9
10 ▾ if agent.depth < agent.max_depth:
11     tools["delegate"] = {**DELEGATE_TOOL_SPEC, "run": partial(tool_delegate, agent)}
```

这里有两个点要注意。

第一，模型看到的是一组预定义动作。

第二，`delegate` 这种子 agent 能力也没有默认放开，它本身也是受限工具。

每个工具除了名字，还有什么

工具在 `Pico` 里不只是一个函数名。每个工具至少带三样东西：

- `schema`
- `risky`
- `description`

比如：

```

1 ▾ "run_shell": {
2     "schema": {"command": "str", "timeout": "int=20"},
3     "risky": True,
4     "description": "Run a shell command in the repo root.",
5 }
```

这个结构很重要，因为 runtime 后面所有约束都要靠它接上。

- `schema` 让模型知道参数长什么样

- `risky` 决定它要不要走审批
- `description` 决定这个动作怎么被提示给模型

所以工具注册表本身已经不只是普通配置，它直接定义了模型动作能力的边界。

一条工具调用在执行前会拦什么

真正的执行闸口在 `pico/runtime.py` 的 `run_tool()`。

这条链路顺序非常固定：

1. 看工具是不是存在
2. 跑 `validate_tool()`
3. 拦连续重复调用
4. 高风险工具走审批
5. 真正执行工具
6. 裁剪结果并写回 memory

核心代码就是这一段：

```
1  tool = self.tools.get(name)
2  if tool is None:
3      return f"error: unknown tool '{name}'"
4
5  self.validate_tool(name, args)
6
7  if self.repeated_tool_call(name, args):
8      return "error: repeated identical tool call ..."
9
10 if tool["risky"] and not self.approve(name, args):
11     return f"error: approval denied for {name}"
12
13 result = clip(tool["run"](args))
14 self.update_memory_after_tool(name, args, result)
15 return result
```

这个顺序很重要，因为它决定了系统是先拦后跑。

未知工具、非法参数、重复调用、审批拒绝，这些情况都在真正执行之前就被挡住了。

参数校验到底在拦什么

参数校验逻辑在 `pico/tools.py` 的 `validate_tool()`。

这层不只是检查字段齐不齐，它会把工具输入压到一个可解释范围里。

比如：

- `list_files` 的目标必须是目录
- `read_file` 的目标必须是文件，行号范围也要合法
- `search` 的 `pattern` 不能为空
- `run_shell` 的 `command` 不能为空，`timeout` 要在 `1~120` 秒之间
- `write_file` 不能把目录当文件写
- `patch_file` 的 `old_text` 必须出现且只能出现一次
- `delegate` 的 `task` 不能为空

这里最能看出设计取舍的是 `patch_file`。它走的是精确替换：

```
1 if name == "patch_file":
2     path = agent.path(args["path"])
3     old_text = str(args.get("old_text", ""))
4     count = text.count(old_text)
5     if count != 1:
6         raise ValueError(f"old_text must occur exactly once, found {count}")
    )
```

这个约束确实更保守，但失败原因会很清楚。对 agent 来说，这一点很重要。runtime 能早点把错误变成可解释的输入问题，后面就不容易演化成难追的副作用。

路径为什么要钉死在工作区里

工具层最关键的护栏之一，是路径边界。

这层逻辑在 `pico/runtime.py` 的 `path()`。流程很简单：

1. 相对路径先挂到仓库根目录下
2. 再做 `resolve()`
3. 最后用 `commonpath` 判断解析后的真实路径还在不在工作区内

只要越界，直接报错：

```
1 raise ValueError(f"path escapes workspace: {raw_path}")
```

这一步挡住的，不只是 `../outside.txt` 这种直白逃逸，也包括通过符号链接跳出去的情况。

这一层边界很硬，因为一旦模型能读写任意路径，很多看起来只是“文件工具”的动作，实际上已经开始碰宿主机了。

run_shell 现在收了哪些边界

`run_shell` 是工具层里最容易失控的动作，所以它现在属于基础约束已经有了，但离强沙箱还有距离。

当前至少有这几层约束：

- 工作目录固定在仓库根目录
- 超时时间限制在 `1~120` 秒
- shell 环境变量走 `allowlist`，不整包继承父进程环境
- 输出会被裁剪后再回给模型

对应代码在 `tool_run_shell()`：

```
1  result = subprocess.run(  
2      command,  
3      cwd=agent.root,  
4      shell=True,  
5      capture_output=True,  
6      text=True,  
7      timeout=timeout,  
8      env=agent.shell_env(),  
9  )
```

这里最实际的作用有两个。

第一，模型默认活动范围还是当前仓库。

第二，像 API key、token 这种环境变量，不会因为跑 shell 就顺手全带进去。

这不等于完整隔离，但已经能挡掉很多常见失控场景。

delegate 为什么做成只读调查

`delegate` 是工具层里一个很有代表性的设计。

它本质上是一个受限子任务。代码里对子 agent 的约束很明确：

- `approval_policy="never"`
- `read_only=True`
- `max_steps` 更小

- 深度超过上限时，连 `delegate` 都不再暴露

对应实现就在 `tool_delegate()`：

```
1 child = Pico(
2     ...,
3     approval_policy="never",
4     max_steps=int(args.get("max_steps", 3)),
5     depth=agent.depth + 1,
6     max_depth=agent.max_depth,
7     read_only=True,
8 )
```

所以 `delegate` 在 `Pico` 里就是调查工具，不承担放权执行。

为什么还要拦重复调用

`run_tool()` 里还有一条很容易被忽略的护栏：连续重复调用拦截。

逻辑很简单，最近两次工具事件如果和当前调用完全一样，直接拒绝：

```
1 return f"error: repeated identical tool call for {name}; choose a different tool or return a final answer"
```

这条规则拦的是一种很常见的坏循环。模型拿到一个不满意的结果，又没有任何新信息，就把同样的调用再发一遍。这样的调用通常不会带来新东西，只会白白消耗步骤和 token。

执行完以后，这些结果会留下什么

工具执行完以后至少会留下几样东西：

- 工具结果会写进 session history
- `tool_executed` 事件会写进 trace
- 最后一轮工具状态会进入 report 相关指标
- 对读写文件类工具，memory 会更新最近路径和局部摘要

这一点很关键，因为工具层执行完以后，结果还要继续服务下一轮推理和后面的审计。

这层约束现在是怎么被验证的

工具层这块代码，仓库里已经有对应验证。

- 路径逃逸和 symlink 越界在 `tests/test_safety_invariants.py`
- 审批拒绝在 `tests/test_safety_invariants.py`
- `run_shell` 的 `allowlist` 环境变量在 `tests/test_safety_invariants.py`
- `delegate` 深度限制和只读子 agent 在 `tests/test_safety_invariants.py`、`tests/test_pico.py`
- `patch_file` 精确替换和重复调用拦截在 `tests/test_pico.py`

实验侧也有一层锚点。 `pico/metrics.py` 里现在有 11 个真实治理场景，对应的主数字是：

- 3 次路径逃逸拦截
- 5 次无效参数拒绝
- 2 次重复调用拦截

当前设计减少了什么，换来了什么

减少了自由度

工具不多，参数校验很严， `patch_file` 也走精确替换。这会牺牲一部分灵活性。

减少了重型安全设施

当前没有完整 OS 级沙箱，也没有更细粒度的目录 ACL。它更像一套运行时治理和工作区约束。

换来了清楚的边界

工具能做什么、什么时候会被拒绝、为什么会被拒绝、执行后会留下什么，这些事情现在都能说清楚。

对一个本地 coding harness 来说，这个取舍是合理的。先把动作边界做实，比先追求能力面更重要。

如果之后要继续修改，优先补什么

我会优先补三件事。

一，把风险模型做得更细

当前 `risky` 还是布尔值。后面如果工具种类继续长，读、写、执行、网络这些动作最好拆成更细的权限层级。

二，把 shell 边界再做厚一点

现在已经有 root、timeout、allowlist env 这些基础约束。

真要接更复杂的实仓库任务，资源限制和更强沙箱还是会变重要。

PS: sandbox对于code agent还是很重要的，但作为一个实习/秋招项目，我没有扩展开增加大家的学习成本，感兴趣的可以了解下，把sandbox也加上。

三，把工具结果做得更结构化

当前大部分工具结果还是文本，这对调试很友好。后面如果 trace 聚合和实验再往前走，结构化结果会更方便。

面试里怎么讲这一层

Pico 的工具层可以讲成一条受控执行链。工具先在 `tools.py` 里显式注册，每个工具都带 schema、风险标记和说明。模型发起调用以后，runtime 不会直接执行，而是统一先过 `run_tool()`，按顺序检查工具是否存在、参数是否合法、是不是重复调用、高风险动作要不要审批。路径默认被限制在工作区里，`run_shell` 也只在 repo root 下执行，并且环境变量走 allowlist。这样模型的动作始终落在的一组明确、可校验、可拒绝、可审计的边界里。

总结

Pico 的工具层做的事情很直接，把模型的动作能力变成一条受控执行链。

工具定义动作边界，中间这层拦非法输入和高风险动作，下面一层负责把执行结果重新接回 history、trace 和 memory。这样模型拿到工具以后，系统还能保持清楚的边界和可解释的行为。对一个本地、轻量、可检查的 harness 来说，这一层比继续加工具更重要。

04-提示词形态与缓存复用设计

背景

Pico 到了这一层，讨论的重点已经从 prompt 怎么拼，走到了 prompt 里哪些东西值得被当成稳定部分来管理。

如果每一轮都把系统规则、工具说明、工作区摘要、记忆、历史和当前请求混在一起，整体功能当然能跑起来。问题也很直接，prompt 会越来越长，重复内容越来越多，后端即使支持 cache，也很难真的命中。

写代码这类长链路任务有个很明显的特点。**每一轮真的在变的，通常是最近历史、当前记忆、用户刚提的新请求。相对稳定的，反而是另一批东西，比如系统规则、工具表面、工作区基线。**

所以这一篇文章我主要给大家讲清楚，这部分稳定内容为什么要被单独拎出来，以及 runtime 是怎么围绕这部分稳定前缀做复用和失效判断的。

这篇要解决什么问题

问题可以压成一句话：

为什么 prompt 里有一部分要被当成稳定前缀来管理，而不是每轮都和动态内容混在一起。

这里真正要回答的是四件事：

- 哪些内容算稳定前缀
- 这份前缀在代码里长什么样
- 工作区或工具表面变了以后，它怎么失效
- 后端支持 cache 时，这份前缀怎么接到真实调用和统计里

为什么要先把稳定段和变化段分开（重要）

从 runtime 视角看，prompt 里的内容并不是同一种东西。

变化很频繁的内容主要是这些：

- 当前用户请求
- 最近几轮历史
- 会话里刚积累出来的短期记忆

相对稳定的内容主要是这些：

- 系统规则
- 工具清单和调用格式
- 工作区摘要
- 工具表面对应的能力边界

这两类内容混在一起当然也能工作，但混在一起之后，runtime 很难回答这几个问题：

- 这轮 prompt 里哪一部分其实和上一轮一样
- 哪一部分真的值得复用
- 哪一部分一变就应该立刻失效

所以 **Pico** 这里先做了一次拆分。稳定段先单独收成 **PromptPrefix**，动态段还是按轮重建。这样后面不管是做 cache，还是做前缀失效判断，runtime 都有抓手。

稳定前缀在代码里长什么样

稳定前缀定义在 `pico/runtime.py` 的 **PromptPrefix** 和 `build_prefix()`。

PromptPrefix 不是一段普通字符串，它至少带这几个字段：

字段	作用
<code>text</code>	真正写进 prompt 的稳定前缀文本
<code>hash</code>	这份前缀文本本身的签名
<code>workspace_fingerprint</code>	当前工作区摘要的签名
<code>tool_signature</code>	当前工具表面的签名
<code>built_at</code>	这份前缀生成时间

代码可以直接看这一段：

```
1  return PromptPrefix(  
2      text=text,  
3      hash=hashlib.sha256(text.encode("utf-8")).hexdigest(),  
4      workspace_fingerprint=self.workspace.fingerprint(),  
5      tool_signature=self.tool_signature(),  
6      built_at=now(),  
7  )
```

这里要注意两件事。

第一，`hash` 对的是完整前缀文本，不只是 `workspace`。

第二，`workspace_fingerprint` 和 `tool_signature` 被单独留了出来，后面判断前缀为什么变时，`runtime` 能说得 clearer。

稳定前缀里到底装了什么

`build_prefix()` 当前会把三类东西拼进前缀：

- 系统规则
- 工具清单和调用示例
- `WorkspaceContext.text()` 生成的工作区摘要

你可以把它理解成 `agent` 的工作手册。它会告诉模型：

- 自己是谁
- 工具怎么调用
- 当前仓库是什么状态

这部分代码在 `build_prefix()` 里很直观：

```
1  text = textwrap.dedent(  
2      f"""\n  
3      You are pico, a small local coding agent working inside a local repository.  
4  
5      Rules:  
6      ...  
7  
8      Tools:  
9      {tool_text}  
10  
11     Valid response examples:  
12     {examples}  
13  
14     {self.workspace.text()}  
15     """)  
16 ).strip()
```

前缀什么时候会重建

真正把这层设计做实的地方，在 `refresh_prefix()`。

`Pico` 不是在初始化时建一份 `prefix`，然后一路用到会话结束。每轮 `prompt` 组装前，`runtime` 都会先跑一次：

- `WorkspaceContext.build(self.root)`
- 再拿新的 `workspace.fingerprint()` 和旧值比较

这一步如果发现工作区变了，`runtime` 会更新 `self.workspace`，然后再决定要不要重建 `prefix`。

代码主线就是这样：

```
1 refreshed_workspace = WorkspaceContext.build(self.root)
2 refreshed_workspace_fingerprint = refreshed_workspace.fingerprint()
3 workspace_changed = force or refreshed_workspace_fingerprint != previous_workspace_fingerprint
4
5 if workspace_changed:
6     self.workspace = refreshed_workspace
7
8 prefix_state = self.build_prefix() if workspace_changed or force or previous_hash is None else self.prefix_state
9 prefix_changed = force or previous_hash != prefix_state.hash
```

这里的关键点是每轮都先重新判断。

判断完发现没变，就继续复用原前缀。判断完发现变了，就立刻重建。

哪些变化会让前缀失效

这层判断不是一个黑箱的设计，它跟 `WorkspaceContext.fingerprint()` 和工具表面直接相关。

当前这几类变化会影响前缀：

- 当前分支变化
- `git status` 变化
- 最近提交变化
- `README.md`、`AGENTS.md` 这些白名单文档变化
- 工具表面变化

这几类变化的共同点很明确，它们都会改变模型对当前仓库的理解。前缀继续沿用旧值，方向就可能歪。

对应地，下面这些变化不会直接让前缀失效：

- `history` 变化

- memory 变化
- 当前用户请求变化

这些内容本来就是动态段，应该按轮重建，不应该混到前缀失效逻辑里。

完整 prompt 这时候是怎么拼出来的

前缀单独管理以后，完整 prompt 还是要每轮重建。

这一层还是由 `ContextManager.build(user_message)` 来做。当前顺序是：

1. `prefix`
2. `memory`
3. `relevant_memory`
4. `history`
5. `current_request`

所以真正的形态可以这样理解，runtime 内部已经知道：

- 哪一块是稳定段
- 哪一块是动态段
- 前缀现在的签名是什么
- 这轮是不是还在用同一份前缀

也正因为这样，`prompt_cache_key` 直接取 `prefix_hash`，而不是整段 prompt 的哈希。

cache 这条链路是怎么接到模型调用里的

`Pico.ask()` 在真正调模型之前，会先看当前 client 支不支持 prompt cache。

如果支持，就把：

- `prompt_cache_key = prefix_hash`
- `prompt_cache_retention = "in_memory"`

传给 `complete()`。

代码就是这一段：

```

1  prompt_cache_key = None
2  prompt_cache_retention = None
3  if getattr(self.model_client, "supports_prompt_cache", False):
4      prompt_cache_key = prompt_metadata.get("prompt_cache_key")
5      prompt_cache_retention = "in_memory"
6
7  raw = self.model_client.complete(
8      prompt,
9      self.max_new_tokens,
10     prompt_cache_key=prompt_cache_key,
11     prompt_cache_retention=prompt_cache_retention,
12 )

```

这一段说明 prompt cache 在 `Pico` 里不是只停在设计图上，它已经进入真实调用链路了。

哪些后端会真的走这条链

当前后端支持情况在 `pico/models.py` 里。

`OpenAICompatibleModelClient` 会根据 base URL 判断自己是不是支持这条 cache 语义：

```

1  self.supports_prompt_cache = any(host in self.base_url for host in ("openai.com", "right.codes"))

```

而 `OllamaModelClient` 明确是：

```

1  self.supports_prompt_cache = False

```

这个判断很保守，但方向是对的。cache 最怕接口看起来统一，实际没有语义。后端根本不支持时，runtime 也不应该硬传一个表面统一的伪参数。

响应回来以后，cache 信息怎么被记下来

这一层也很重要，因为没有观测，就很难知道 cache 到底有没有起作用。

`OpenAICompatibleModelClient` 会从响应里的 `usage` 结构抽这些字段：

- `input_tokens`
- `output_tokens`
- `total_tokens`
- `cached_tokens`
- `cache_hit`

抽取逻辑在 `_extract_usage_cache_details()` :

```
1 return {
2     "input_tokens": input_tokens,
3     "output_tokens": output_tokens,
4     "total_tokens": usage.get("total_tokens"),
5     "cached_tokens": cached_tokens,
6     "cache_hit": cached_tokens > 0,
7 }
```

然后 `complete()` 会把这些元数据写进 `self.last_completion_metadata` 。

回到 `Pico.ask()` 之后, runtime 再把它并到本轮的 `prompt_metadata` 里。

最后这些信息会继续进入:

- `self.last_prompt_metadata`
- `trace.jsonl`
- `report.json`

所以现在 runtime 不只是会传 cache key, 它还能回答:

- 这轮后端支不支持 cache
- 这轮有没有命中
- 命中了多少 token

目前没做什么

这一层也有边界, 而且这些边界最好主动讲清楚。

- 当前没有 provider-agnostic 的统一 cache 策略层
- `prompt_cache_retention` 还是固定写死为 `"in_memory"`
- prefix 还没有拆成更细的子块分别管理
- Ollama 当前没有接这条 cache 语义
- 现在的 cache 统计还没有进一步换算成更直观的节省成本

这些都不是 bug, 是下一步可以继续做的空间。

如果后续要继续改进, 怎么做

一, 把 `prompt_cache_retention` 做成策略项

现在固定成 `"in_memory"`，先跑通链路没问题。后面如果任务更长、后端更多，这一层最好能根据场景调整。

二，把 prefix 再拆细一点

现在规则、工具说明、工作区摘要还是合成一整块前缀。后面如果想更清楚地看是谁导致失效，拆细会更方便。

三，把 cache 收益写得更直观点

现在已经能拿到 `cached_tokens`、`cache_hit_rate`、`prefix_reuse_rate`。后面可以继续把它们转成更容易读的收益指标。

面试里怎么讲

Pico 这里先把 prompt 里的稳定部分单独拎出来管理。系统规则、工具说明和工作区摘要会先组成一份 `PromptPrefix`，这份前缀有自己的文本签名、工作区指纹和工具签名。每轮请求前，runtime 会先判断这份前缀是不是还有效；如果工作区没变，就继续复用；如果分支、状态、项目文档或者工具表面变了，就立刻重建。然后在支持的后端上，再把这份前缀的 hash 作为 `prompt_cache_key` 发给模型，把响应里的 `cached_tokens` 和 `cache_hit` 收回来。这样稳定部分会被认真复用，动态部分还是按轮重建。

总结

`Pico` 的这一层设计，核心是把 prompt 里的稳定段单独拎出来管理。

前缀先被做成一个有签名、可复用、可失效判断的对象；动态段还是按轮重建；支持的后端再把前缀 hash 接成真实 cache 复用链路。这样 prompt 不只是能拼出来，还能被 runtime 更稳定地管理和观测。

05-结构化的会话记忆

背景

Pico 到了记忆这一层，真正要解决的不是“像人一样记住很多事”，而是一个更具体的问题，模型是不是还在重复做刚做过的事情。

代码仓库任务里，这个问题很常见。模型上一轮刚读过一个文件，下一轮又去读一遍；刚确认过一个事实，过两轮又忘了；或者 prompt 里明明塞了很多旧内容，但真正该留下的那一小部分事实并没有被稳定接住。

所以对于code agent的设计，不做大记忆系统，也不做会话知识库。

我们的核心设计思考是 **Pico** 怎么用一个很轻的结构，把跨轮还真有用的东西留下来，减少重复读取和重复劳动。

这篇要解决什么问题

问题可以压成一句话：

在多轮代码任务里，哪些事实值得跨轮留下来，而且留下来以后要怎么保证它们还能继续可信。

这里真正要回答的是四件事：

- **Pico** 现在把哪些东西记进 memory
- 这些内容是怎么和工具执行链绑在一起的
- 旧摘要怎么失效，避免把过期结论继续带下去
- 这套轻量记忆为什么真的减少了重复读取

这些问题回答不清楚的话，记忆层就很容易变成一个模糊概念，看起来高级，但说不清到底解决了什么。

Pico 的记忆不是知识库

Pico 记忆的目标很明确：

- 先减少重复读取
- 先保住跨轮还会继续用到的局部事实
- 先让记忆保持可解释

它没有一上来走向量检索，也没有把所有历史都变成一个大召回池。

目前的设计更像一个轻量工作记忆，主要围绕当前任务、最近文件、文件摘要和几条跨轮笔记来组织。

记忆在代码里长什么样

记忆结构定义在 `pico/memory.py` 的 `default_memory_state()`。

当前默认结构长这样：

```
1 return {
2     "working": {
3         "task_summary": "",
4         "recent_files": [],
5     },
6     "episodic_notes": [],
7     "file_summaries": {},
8     "task": "",
9     "files": [],
10    "notes": [],
11    "next_note_index": 0,
12 }
```

真正承担记忆职责的，主要是这四块：

- `working.task_summary`
- `working.recent_files`
- `file_summaries`
- `episodic_notes`

这几个字段拆开以后，一眼就能看出 `Pico` 的取舍。它没有把会话压成一段大摘要，做的就是任务、文件、摘要、笔记分开。

这四层分别在记什么

1. `task_summary`

这一层最直接，就是当前任务的短摘要。

每次 `ask(user_message)` 开始时，runtime 都会先把用户请求写进这里：

```
1 self.memory.set_task_summary(user_message)
```

它的作用很简单，哪怕后面 history 被裁剪了，模型也还有一个很短的锚点，知道这一轮到底在干什么。

2. recent_files

这一层记录最近接触过的文件路径，而且会去重，只保留固定上限。

对应代码在 `remember_file()`：

```
1 files = [item for item in state["working"]["recent_files"] if item != path]
2 files.append(path)
3 state["working"]["recent_files"] = files[-WORKING_FILE_LIMIT:]
```

它解决的问题也很具体，下一轮继续思考时，模型至少知道自己刚碰过哪些文件，不需要又从头猜一遍。

3. file_summaries

这一层是 `Pico` 记忆里最有用的一层。

`read_file` 的结果不会原样塞进 memory，而是先走 `summarize_read_result()`，压成一个很短的摘要，再按路径写进 `file_summaries`。

```
1 summary = memorylib.summarize_read_result(result)
2 self.memory.set_file_summary(canonical_path, summary)
```

这意味着记忆里留下的不是整段文件内容，而是一句足够提醒下一轮“刚才读到了什么”的短结论。

4. episodic_notes

这一层更像跨轮笔记。

当前实现会把一些值得跨轮带走的短结论放进这里，然后在组 prompt 时，按当前请求做轻量召回。

它不是 embedding 检索，规则很透明：

- tag 命中
- 关键词重叠
- 时间新旧
- 插入顺序

对应代码在 `retrieval_candidates()`：

```

1 exact_tag_match = int(bool(query_tokens & note_tags))
2 keyword_overlap = len(query_tokens & note_tokens)
3 ...
4 ranked.sort(key=lambda item: item[0], reverse=True)
5 return [note for _, note in ranked[:limit]]

```

这套召回很轻，但它有个优点，结果为什么是这样，基本能解释清楚。

旧摘要怎么失效

这一层是 `Pico` 记忆设计里很关键的地方。

如果文件摘要不能感知文件是不是已经变了，记忆就会开始把旧结论当真。对代码 agent 来说，这比“没记住”更危险。

所以 `file_summaries` 里每条摘要都带 `freshness`。本质上就是文件内容哈希。

```

1 state["file_summaries"][path] = {
2     "summary": summary,
3     "created_at": now(),
4     "freshness": file_freshness(path, workspace_root),
5 }

```

等到 `render_memory_text()` 再把这些摘要渲染给模型看时，它会重新算一次当前文件的 `freshness`。哈希对不上，旧摘要就不再显示。

```

1 current_freshness = file_freshness(path, workspace_root)
2 if summary.get("summary", "") and summary.get("freshness") == current_freshness:
3     summaries.append(...)

```

再加上 `write_file` 和 `patch_file` 之后会主动失效旧摘要：

```

1 elif name in {"write_file", "patch_file"}:
2     self.memory.invalidate_file_summary(canonical_path)

```

这样记忆层至少能避开一个很常见的问题，文件已经变了，模型还在拿旧摘要继续推理。

新事件怎么同时进入历史和记忆

这部分设计在 `pico/runtime.py` 的 `update_memory_after_tool()`。

它的主线很简单：

1. 工具结果先完整写进 `history`
2. 只有少量高价值结果会继续沉淀进 `memory`

现在跟记忆绑定最紧的是这三类工具：

- `read_file`
- `write_file`
- `patch_file`

规则也很明确：

- `read_file` 会记路径、生成摘要、更新 `file_summaries`，再追加一条 `note`
- `write_file` 会记路径，同时让旧摘要失效
- `patch_file` 会记路径，同时让旧摘要失效

代码就是这一段：

```
1 if name in {"read_file", "write_file", "patch_file"}:
2     self.memory.remember_file(canonical_path)
3 if name == "read_file":
4     summary = memorylib.summarize_read_result(result)
5     self.memory.set_file_summary(canonical_path, summary)
6     self.memory.append_note(summary, tags=(canonical_path,), source=canonical_path)
7 elif name in {"write_file", "patch_file"}:
8     self.memory.invalidate_file_summary(canonical_path)
```

这个设计的意思很清楚，记忆不是凭空总结出来的，它跟工具链路绑在一起。

这些记忆下一轮是怎么被用到的

到了下一轮，`ContextManager.build(user_message)` 会把 `memory` 分成两部分送回去：

- `memory`
- `relevant_memory`

`memory` 由 `render_memory_text()` 渲染，主要展示：

- 当前任务
- 最近文件
- 仍然新鲜的文件摘要
- 当前笔记数量

`relevant_memory` 则是从 `episodic_notes` 里按当前请求召回 top 3。

所以这一层真正有用的地方，不是东西存下来了，而是它们在下一轮 prompt 里会以两种不同形式回来：

- 一种是稳定的小仪表盘
- 一种是按当前问题拉出来的相关笔记

这套记忆现在是怎么被验证的

第一类是单测

`tests/test_memory.py` 现在已经覆盖了几件关键事：

- `task_summary` 和 `recent_files` 会按预期更新
- `episodic_notes` 的追加和召回是确定性的
- `file_summaries` 会走 canonical path
- 文件内容变化以后，旧摘要不会继续出现在 `render_memory_text()` 里

这些测试解决的是机制有没有按设计工作。

第二类是 memory on/off/irrelevant 实验

`pico/metrics.py` 里有 memory dependency experiment 和 large-scale memory experiment。

这两条实验线最关键的地方是，它们不是只做 on/off，对照组还加了 `memory_irrelevant`。

这很重要，因为它能回答一个更尖锐的问题，效果是不是只是因为 prompt 里多塞了点东西。

当前仓库里的大口径结果可以直接压成这几句：

- 在 12 个真实记忆依赖任务里，`memory_on` 的重复读取从 8 次降到 3 次
- 平均工具步数从 0.67 降到 0.25
- 正确率从 66.7% 提升到 100%
- `memory_irrelevant` 没有带来同样收益

目前设计的边界

`Pico` 这版记忆已经够用，但边界也很明确。

- 它现在主要擅长留住文件级和局部事实
- 它还没有显式维护更高层的计划状态

- 召回逻辑是轻量规则，不是语义检索
- assistant 的普通自然语言回复不会系统性地再提炼回 memory

所以它更像一个轻量工作记忆，不像完整的任务状态板。

这也是当前版本该有的边界。先把减少重复读取和跨轮复用做稳，比一开始就把记忆做成一个复杂黑盒更重要。

如果继续往下做，优先做什么

一，把任务状态字段补得更明确

后面可以继续加：

- `current_plan`
- `open_questions`
- `confirmed_findings`
- `blocked_on`
- `next_action`

这样模型恢复时，不只是知道刚读过哪些文件，还知道现在推进到哪一步了。

二，把更多高价值结论沉淀进 memory

现在 `episodic_notes` 跟 `read_file` 绑得比较紧。后面可以考虑把一些测试结论、shell 关键报错、用户新追加偏好也沉淀进去。

三，继续保持记忆可解释

如果后面真往更复杂的召回走，最需要保住的不是“更高级”，而是读者和开发者还能看懂这条记忆为什么被召回。

面试里怎么讲这一层

Pico 的记忆层没有一上来做成一个很重的会话知识库，我先解决的是重复读取和跨轮复用。现在记忆主要分成任务摘要、最近文件、文件摘要和会话笔记四层。文件摘要会带 freshness，只要文件变了，旧摘要就会失效；相关记忆召回也没有上重型语义检索，而是先用 tag、关键词、新旧顺序做轻量排序。这样记忆虽然很轻，但它能稳定回答两个问题：刚才读过什么，哪些结论下一轮还值得继续带着。

实验上这条线也有 on/off/irrelevant 对照，结果说明收益不是因为多塞了点上下文，而是结构化、相关的记忆真的减少了重复读取。

总结

Pico 的记忆层做的事情很直接，把跨轮还会继续有用的局部事实留下来，而且尽量保证这些事实不会过期复用。

它现在还不重，但已经把任务摘要、最近文件、文件摘要、跨轮笔记这几块接到了工具链和下一轮 prompt 里。对一个本地 coding harness 来说，这一层先把重复劳动降下来，比先做一个很大的记忆黑盒更重要。

06-上下文瘦身与输出管理设计

背景

问题从 prompt 能不能拼出来，变成了长链路任务里系统怎么把真正有用的上下文留下来。

代码任务一旦进入多轮循环，最先膨胀起来的通常不是用户输入，往往是别的东西：

- 工具输出
- 文件内容
- shell 日志
- 历史记录
- 相关记忆

这些内容有个共同点，都可能很长，而且价值分布很不均匀。有些内容对下一步很关键，有些只是调查过程里的副产品。如果系统不做控制，提示词很快就会被旧历史、长日志和重复信息占满。

这里主要给大家讲 **Pico** 怎么把上下文瘦身做成一条分层机制。

这篇要解决什么问题

问题可以压成一句话：

当 agent 在一轮任务里不断读文件、跑命令、积累历史时，runtime 怎么把最有用的上下文继续送给模型，又不让 prompt 被撑爆。

这里真正要回答的是四件事：

- 长输出最早在哪一层就开始收
- history 和 relevant memory 怎么被压缩
- 最终 prompt 超预算时，谁先让位
- 这套规则为什么比粗暴截断更稳

为什么不能靠粗暴截断

从工程角度看，粗暴截断的问题不在于它简单，而在于它不知道自己切掉的是什么。

代码仓库 agent 的上下文里，有些东西硬度很高：

- 当前用户请求
- 系统规则
- 仓库基线

有些东西可以压缩：

- 相关记忆
- 旧历史
- 重复读取
- 长日志

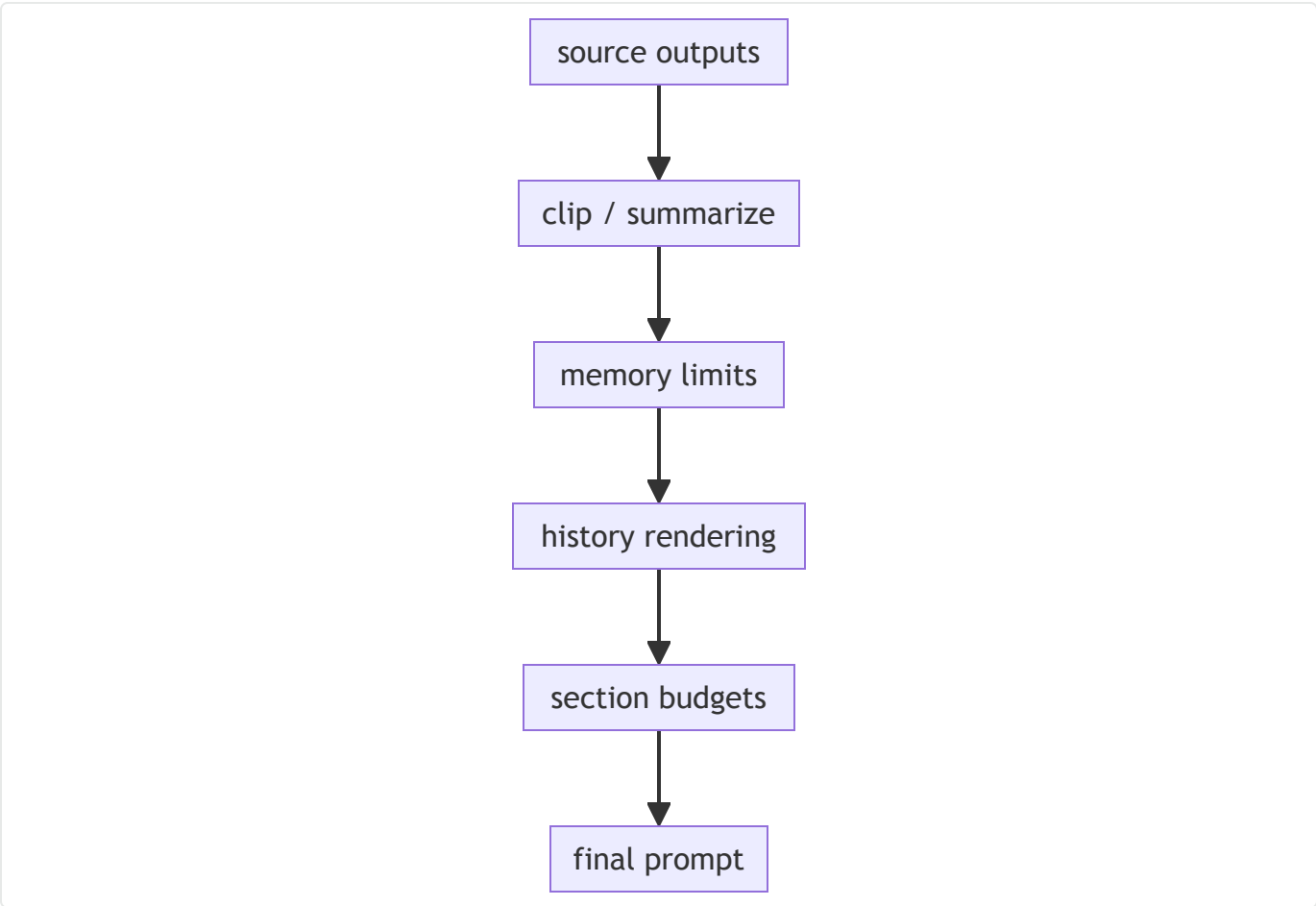
如果不分层，直接对整段 prompt 一刀切，结果经常会变成最该保留的东西被一起切坏。尤其是当前请求，一旦失真，后面所有优化都没意义。

Pico 这里的判断很明确，不走一把大剪刀，而是把瘦身拆成几层。

Pico 现在是怎么做上下文瘦身的

当前这套机制可以拆成四层。

1. 源头先裁长输出
2. 记忆层先做限额
3. 历史按新旧分层压缩
4. 最后按 section 预算再做一轮收缩



瘦身不只发生在 `ContextManager.build()` 里。很多长内容一进系统就已经开始被收了。

第一层，长输出先在入口处收

`Pico` 的第一道闸门，不在 `prompt` 组装阶段，而在内容刚进入系统的时候。

`pico/workspace.py` 里有一个通用的 `clip()`：

```
1 def clip(text, limit=MAX_TOOL_OUTPUT):
2     text = str(text)
3     if len(text) <= limit:
4         return text
5     return text[:limit] + f"\n...[truncated {len(text) - limit} chars]"
```

默认上限是 `MAX_TOOL_OUTPUT = 4000`。这意味着超长工具结果不会在系统内部无限流动。

当前这层裁剪至少出现在这些地方：

场景	上限
工具返回值统一裁剪	4000

工作区文档片段	1200
<code>git status --short</code>	1500
trace 里的工具结果摘要	500
<code>run_started</code> 里的用户请求摘要	300

除此之外，还有一层更细的压缩，`summarize_read_result()` 会把 `read_file` 的结果压成一句很短的摘要：

```
1 summary = " | ".join(lines[:3])
2 return clip(summary, limit)
```

这一步的作用很直接，不让完整文件内容直接冲进长期上下文。

第二层，memory 自己先限额

06 这一章不展开记忆设计细节，但它和瘦身是绑在一起的，所以这里必须提。

`pico/memory.py` 里现在有几组硬上限：

- `WORKING_FILE_LIMIT = 8`
- `EPISODIC_NOTE_LIMIT = 12`
- `FILE_SUMMARY_LIMIT = 6`

文本本身也会先裁长度：

- `task_summary` 裁到 300
- `episodic note` 裁到 500
- `file summary` 裁到 500

这层设计解决的是一个很实际的问题，记忆自己不能先长成第二份历史。它得保持小，才能真的帮下一轮做事。

第三层，Relevant memory 不全带，只带少量

到了 `ContextManager.build(user_message)`，相关记忆会先被召回，再被预算控制。

现在默认只召回 top 3：

```

1  selected_notes = self.agent.memory.retrieval_candidates(
2      user_message,
3      limit=RELEVANT_MEMORY_LIMIT,
4  )

```

`RELEVANT_MEMORY_LIMIT` 当前是 `3`。这里还有一个很实用的设计，每条 note 会平分这一段预算，不让一条超长笔记把其他笔记都挤掉。

代码在 `_render_relevant_memory()`：

```

1  per_note_budget = self._per_note_budget(budget, len(note_texts), header)
2  rendered_notes = [_tail_clip(text, per_note_budget) for text in note_texts]

```

这一步说明两件事。

- 相关记忆不会全带
- 带进去的几条也会继续被压短

第四层，history 按新旧分层

最容易膨胀的一层，其实是 transcript。

`ContextManager._render_history_section()` 当前的思路很清楚，它把历史拆成新旧两层来看：

- 最近窗口保留更多细节
- 更旧的历史压得更狠

最近窗口当前固定是 `6`：

```

1  recent_window = 6
2  recent_start = max(0, len(history) - recent_window)

```

这相当于一个很朴素的近因优先策略。越近的内容，越可能影响下一步动作，所以保留得更完整。

旧历史怎么压缩

旧历史不会原样回填。系统会把它重新渲染成更轻的摘要形式：

- 普通 `user` / `assistant` 消息只保留一行短文本
- 旧的 `read_file` 会按路径去重
- 旧的工具输出会被压成一行摘要

对应代码里，旧工具和最近工具的单条长度上限本身就不同：

```
1 line_limit = 900 if recent else 60
```

这个差异很重要。Pico 没有试图让所有历史都同等重要。

重复读取怎么折叠

旧历史里的重复 read_file 会按路径去重。

也就是说，系统会尽量保留我读过这个文件这件事，不会在旧历史里一遍遍重复我又读了一次。

记忆负责留短摘要，历史负责折叠重复过程。

最后一层，section 预算控制

上下文真的装箱，发生在 ContextManager.build() 。

当前 prompt 的 section 顺序是固定的：

- 1. prefix
- 2. memory
- 3. relevant_memory
- 4. history
- 5. current_request

默认预算也是固定的：

部分	默认预算
prefix	3600
memory	1600
relevant_memory	1200
history	5200
current_request	不裁剪

默认总预算是 12000 。

如果超预算，系统不会对整段 prompt 一刀切，而是按固定顺序收：

```
1  DEFAULT_REDUCTION_ORDER = ("relevant_memory", "history", "memory", "prefix"
    )
```

这条顺序很有代表性。平台已经把谁先让位写死了。

- 先让 `relevant_memory` 收
- 再让 `history` 收
- 再动 `memory`
- 最后才动 `prefix`

`current_request` 根本不进裁剪流程，它一直原样保留。

这就是 `Pico` 在上下文治理里的核心判断，**用户当前这句话最硬，不能失真**。

floor 现在怎么生效

`pico/context_manager.py` 里虽然定义了 `DEFAULT_SECTION_FLOORS`，但默认运行时用的 floor 计算来自 `_compute_section_floors()`：

```
1  floors = {
2      section: max(20, int(budget) // 4)
3      for section, budget in self.section_budgets.items()
4  }
```

也就是说，默认情况下，每个 section 最低会被压到自身预算的四分之一，除非显式传 `section_floors` 去覆盖。

这一层的设计怎么被验证

第一类是行为测试

`tests/test_context_manager.py` 现在已经覆盖了几件关键事：

- section 顺序固定
- 超预算时先裁 `relevant_memory` 再裁 `history`
- `Relevant memory` 最多只保留 3 条
- 当前用户请求会被完整保留
- 新的上下文比更旧的上下文更容易留下来

这些测试解决的是机制有没有按当前设计工作。

第二类是长上下文实验

`pico/metrics.py` 和现有指标文档里，已经有 `full` 和 `no_context_reduction` 的对照。

当前大口径结果可以直接压成这几句：

- 在 12 组长上下文实验配置下，平均 prompt 长度从 `6964` 压到 `5418`
- 平均压缩率 `18.01%`
- 最高压缩率 `35.63%`

目前没做什么

这套设计已经够用，但边界也很明确。

- 现在还是 char 级预算控制，还没做到 token-aware
- 规则能决定先裁哪一层，还不能真正理解哪一句最重要
- 重复检测主要集中在旧 `read_file` 这一类场景
- 最近窗口固定是 `6`，还没有更细的动态策略

如果继续往下做，优先怎么补

我会优先补三件事。

一，把预算从 char 往 token 再推进一步

现在这版已经够稳，但不同 provider 之间要做更公平的比较，token-aware 会更合适。

二，把历史保留策略再做细一点

现在是固定 recent window，后面可以继续区分失败事件、关键工具结果和普通历史。

三，把诊断视图再做清楚一点

现在 metadata 已经有 `budget_reductions`、`rendered_chars` 这些信息。后面还可以继续往前走，直接回答哪一层最常爆预算、哪一类内容最常被裁掉。

面试里怎么讲这一层

Pico 的上下文治理没有走粗暴截断这条路。我做的是分层瘦身。长输出先在入口处裁掉，记忆层自己先做数量和长度限制，历史记录按新旧分层，最后 `ContextManager` 再按 section 预算做一轮收缩。超预算时先裁 `relevant_memory`，再裁 `history`，再裁 `memory`，最后才动 `prefix`。`current_request` 永远不裁。这套设计的核心是该保留的东西尽量稳定留下来，不让旧日志和重复上下文把当前请求埋掉。

总结

`Pico` 的这一层设计，核心是把上下文瘦身做成一条分层机制。

长输出先在入口收，`memory` 自己先限额，`history` 按新旧分层，最后再做 section 预算控制。这样 prompt 会被收紧，同时结构和当前请求的完整性还在。对一个本地 coding harness 来说，这比单纯塞更多上下文更重要。

07-会话状态、运行工件与恢复机制设计

背景

本地 code agent 一旦开始处理真实仓库任务，很快就会碰到三件事：

- 跑到一半中断了，下一次怎么接着来
- 跑完以后结果不对，怎么回看过程
- 做 benchmark、做聚合、做面试展开时，证据从哪里拿

这三件事看起来像一类问题，实际上不是同一层。恢复现场需要的是还能继续工作的状态，复盘和审计需要的是可回看的运行证据。Pico 这一层做的，就是把这两类东西分开。

这篇要解决什么问题

问题可以压成一句话：

为什么 Pico 要把会话状态和 run 工件拆成两层，以及这两层各自解决什么问题。

这里的 run 工件，指的就是每次 ask() 跑完后单独落到 .pico/runs/<run_id>/ 下面的几份运行产物，主要是 task_state.json、trace.jsonl 和 report.json。

这里真正要回答的是四件事：

- session 在保存什么
- run artifact 在保存什么
- --resume 到底恢复了哪一层
- 为什么 task_state、trace、report 要分开写

如果这几件事讲不清楚，运行状态设计就很容易被看成反正都存成 JSON 了，看不出这层工程取舍。

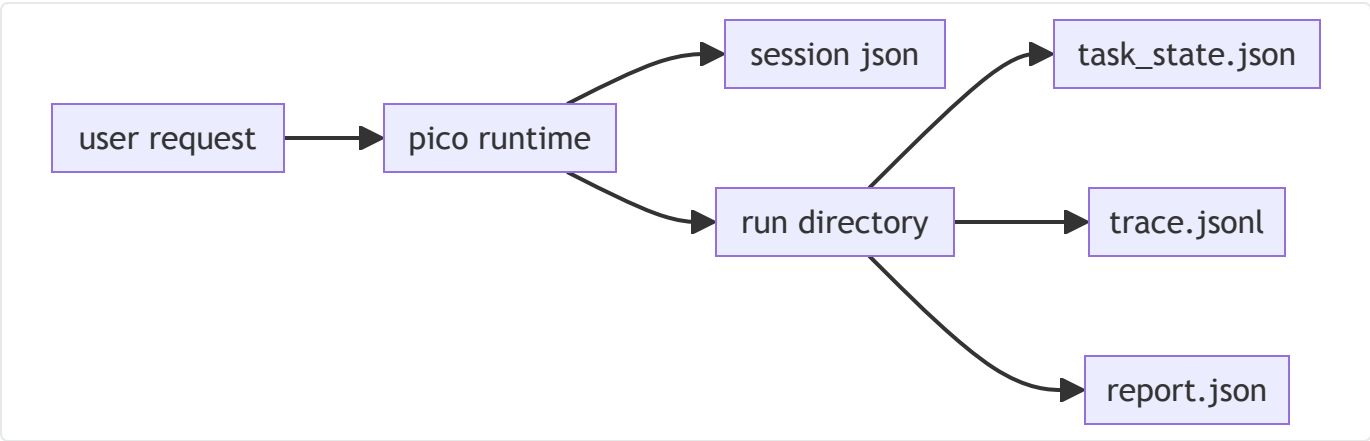
Pico 为什么要分两层

Pico 现在把运行状态分成两层。

- 第一层是 session
- 第二层是 run artifact

session 回答的是，以后还能不能继续干。

run artifact 回答的是，刚才到底发生了什么。



这张图里最重要的点就一句话，session 和 run artifact 虽然都在存状态，但回答的不是同一个问题。

session 这一层在存什么

session 持久化在 `pico/runtime.py` 的 `SessionStore` 。

路径很简单：

```
1 .pico/sessions/<session_id>.json
```

这个 JSON 里现在最关键的字段有这些：

字段	作用
<code>id</code>	会话 id
<code>created_at</code>	会话创建时间
<code>workspace_root</code>	当前工作区根目录
<code>history</code>	完整会话记录
<code>memory</code>	工作记忆

也就是说，session 里保存的是下一轮继续工作还会用到的状态。

`SessionStore` 的实现也很轻：

```

1 class SessionStore:
2     def path(self, session_id):
3         return self.root / f"{session_id}.json"
4
5     def save(self, session):
6         path = self.path(session["id"])
7         path.write_text(json.dumps(session, indent=2), encoding="utf-8")
8         return path
9
10    def load(self, session_id):
11        return json.loads(self.path(session_id).read_text(encoding="utf-8"))

```

这层没有数据库，也没有复杂索引。对 **Pico** 这个体量来说，这个取舍很实际，先把恢复做直接，先把存储做复杂更重要。

run artifact 这一层在存什么

每次 `ask()`，`RunStore.start_run()` 都会创建一个 run 目录：

```
1 .pico/runs/<run_id>/
```

当前核心工件有三类：

文件	作用
<code>task_state.json</code>	当前这轮运行的状态快照
<code>trace.jsonl</code>	运行过程中的逐事件时间线
<code>report.json</code>	这轮结束后的聚合摘要

对应代码在 `pico/run_store.py`：

```

1 def task_state_path(self, run_id):
2     return self.run_dir(run_id) / "task_state.json"
3
4 def trace_path(self, run_id):
5     return self.run_dir(run_id) / "trace.jsonl"
6
7 def report_path(self, run_id):
8     return self.run_dir(run_id) / "report.json"

```

这一层和 session 的区别很清楚，run artifact 负责的是审计、回放和聚合。

为什么每次 ask() 都要新开一个 run

`Pico.ask()` 一开始就会创建：

- 一个新的 `TaskState`
- 一个新的 `run_id`
- 一个新的 run 目录

主线在 `pico/runtime.py`：

```
1 task_state = TaskState.create(  
2     run_id=self.new_run_id(),  
3     task_id=self.new_task_id(),  
4     user_request=user_message,  
5 )  
6 self.current_task_state = task_state  
7 self.current_run_dir = self.run_store.start_run(task_state)
```

这一步很值，因为它把一次用户请求和一组独立工件绑定死了。

后面你做排查、看 trace、聚合 metrics，都不需要再从一整个会话里猜边界。run 的边界在文件系统上就是独立的。

task_state 为什么是快照

`pico/task_state.py` 里的 `TaskState` 保存的是这轮运行此刻的中心状态。

当前关键字段有这些：

- `status`
- `tool_steps`
- `attempts`
- `last_tool`
- `stop_reason`
- `final_answer`

它的职责是回答现在这轮跑到哪了。

比如：

- `attempts` 记录模型已经被调了多少轮
- `tool_steps` 记录真正执行了多少步工具
- `stop_reason` 记录最后为什么停

- `final_answer` 记录最后答案

这就是为什么 `task_state.json` 更像快照，不像日志。

你想先知道这轮为什么停，先看它就够了。

trace 为什么要单独用 jsonl

`RunStore.append_trace()` 现在是 JSONL 逐条追加，不是最后一次性写整份 trace。

```
1 with path.open("a", encoding="utf-8") as handle:
2     handle.write(json.dumps(event, sort_keys=True, ensure_ascii=True))
3     handle.write("\n")
```

这个做法很朴素，但很适合 agent 运行过程。

因为 trace 天生就是事件流，比如：

- `run_started`
- `prompt_built`
- `model_requested`
- `model_parsed`
- `tool_executed`
- `run_finished`

这类数据按 JSONL 写有几个好处：

- 运行中就能持续落盘
- 中途异常时，不容易整份 trace 都没了
- 后面做聚合时，顺着事件扫就行

所以 trace 这层负责回答的是，过程里发生过什么。

report 为什么还要单独写一份

有了 `task_state` 和 `trace`，为什么还要再写 `report.json`？

因为前两者都更像原材料。

- `task_state` 适合看当前状态
- `trace` 适合看过程细节

真正做聚合、benchmark、resume metrics 时，更顺手的是一份已经收过的结果摘要。

`build_report()` 当前会把这些关键字段写进去：

- `run_id`
- `task_id`
- `status`
- `stop_reason`
- `final_answer`
- `tool_steps`
- `attempts`
- `task_state`
- `prompt_metadata`

代码在 `pico/runtime.py`：

```
1  return {
2      "run_id": task_state.run_id,
3      "task_id": task_state.task_id,
4      "status": task_state.status,
5      "stop_reason": task_state.stop_reason,
6      "final_answer": task_state.final_answer,
7      "tool_steps": task_state.tool_steps,
8      "attempts": task_state.attempts,
9      "task_state": task_state.to_dict(),
10     "prompt_metadata": self.last_prompt_metadata,
11 }
```

所以 `report` 这层更像给聚合脚本和实验统计准备的成品层。

resume 到底恢复了什么

恢复走的是 `Pico.from_session()`，CLI 则通过：

- `--resume <session_id>`
- `--resume latest`

把这个入口接起来。

`build_agent()` 里的逻辑是：

```
1 session_id = args.resume
2 if session_id == "latest":
3     session_id = store.latest()
4 if session_id:
5     return Pico.from_session(...)
```

这里有个点很重要，恢复的时候做的是下面这几件事：

- 重新构造新的 `Pico`
- 重新注入当前 `model_client`
- 重新注入当前 `workspace`
- 把旧的 `session` 状态读回来

也就是说，`resume` 恢复的是会话状态，不是进程本身。

这个边界其实很干净。`session` 需要可序列化，`provider client` 和 `runtime` 对象本身不需要。

这一层现在怎么被验证

第一类是 run store 行为测试

`tests/test_run_store.py` 现在已经覆盖了：

- `start_run()` 会创建 `run` 目录和 `task_state.json`
- `append_trace()` 会按 JSONL 追加事件
- `write_report()` 会写出 `report.json`
- 缺失最终 `report` 的 `run` 目录也能被容忍

第二类是运行闭环测试

`tests/test_pico.py` 现在已经覆盖了：

- 一次 `ask()` 结束后会留下 `task_state.json`、`trace.jsonl`、`report.json`
- `task_id` 和 `run_id` 会分开
- `run_finished` 会出现在 `trace` 里
- `secret` 会在 `trace` 和 `report` 里被脱敏
- `session` 可以被保存再恢复

也就是说，这层状态闭环现在不是只停在设计图上，代码和测试都已经把关键行为钉住了。

这一层现在没做什么

- session 还是单文件 JSON，没有更复杂索引
- session 和 run 的关联关系还比较自然，没有单独抽一层索引
- run artifact 还没有 schema version
- resume 还没有更复杂的一致性检查

这些都不是 bug，更像后续可以继续往前走的点。

如果继续往下做，优先怎么补

我会优先补三件事。

一，给 run artifact 加 schema version

benchmark 这边已经有版本意识了。后面如果 trace 和 report 结构会继续演进，run artifact 也值得单独带版本。

二，把 session 和 run 的关系再写得更显式

现在已经能串起来，但如果以后要做跨会话分析，关联字段可以更明确一点。

三，给 resume 加更多一致性检查

如果后面 workspace 和 memory 结构变复杂，恢复时多做几项一致性检查会更稳。

面试里怎么讲这一层

Pico 这里把运行状态拆成了两层。session 负责保存还能继续工作的状态，比如 history 和 memory；run artifact 负责保存刚才这轮到底发生了什么，所以每次 `ask()` 都会单独写出 `task_state.json`、`trace.jsonl` 和 `report.json`。`task_state` 负责看现在跑到哪了，`trace` 负责看过程里发生了什么，`report` 负责给后面的聚合和实验统计提供摘要。这样恢复和复盘就不会混在一起，agent 也不只是当场跑完就结束。

总结

Pico 的这一层设计，核心是把还能继续工作的状态和能回看的证据分开。

session 负责延续会话，run artifact 负责记录单轮执行。这样一来，agent 不只会跑，也更容易恢复、更容易复盘、更容易拿去做 benchmark 和面试展开。

08-评测框架与实验方法设计

背景

如果一个 agent 项目只有模块介绍，没有评测方法，最后很容易掉进一个很尴尬的状态。你能展示它做成过几次事情，但你说不清这些成功是偶然还是稳定；你也能讲自己做了上下文治理、记忆、工具治理这些模块，但你很难回答它们到底有没有带来可测的收益。

对面试来说，这个问题更现实。面试官真正想听的，往往不只是你做了哪些模块，还包括：

- 你怎么知道它跑对了
- 你怎么比较不同机制的收益
- 你怎么避免自己给自己打高分

这篇主要讲的是，`Pico` 怎么把评测做成一条能落地、能复盘、能聚合的证据链。

这篇要解决什么问题

问题可以压成一句话：

agent 项目里，怎么把我觉得它还行变成一套有任务合同、有判定标准、有失败解释的评测方法。

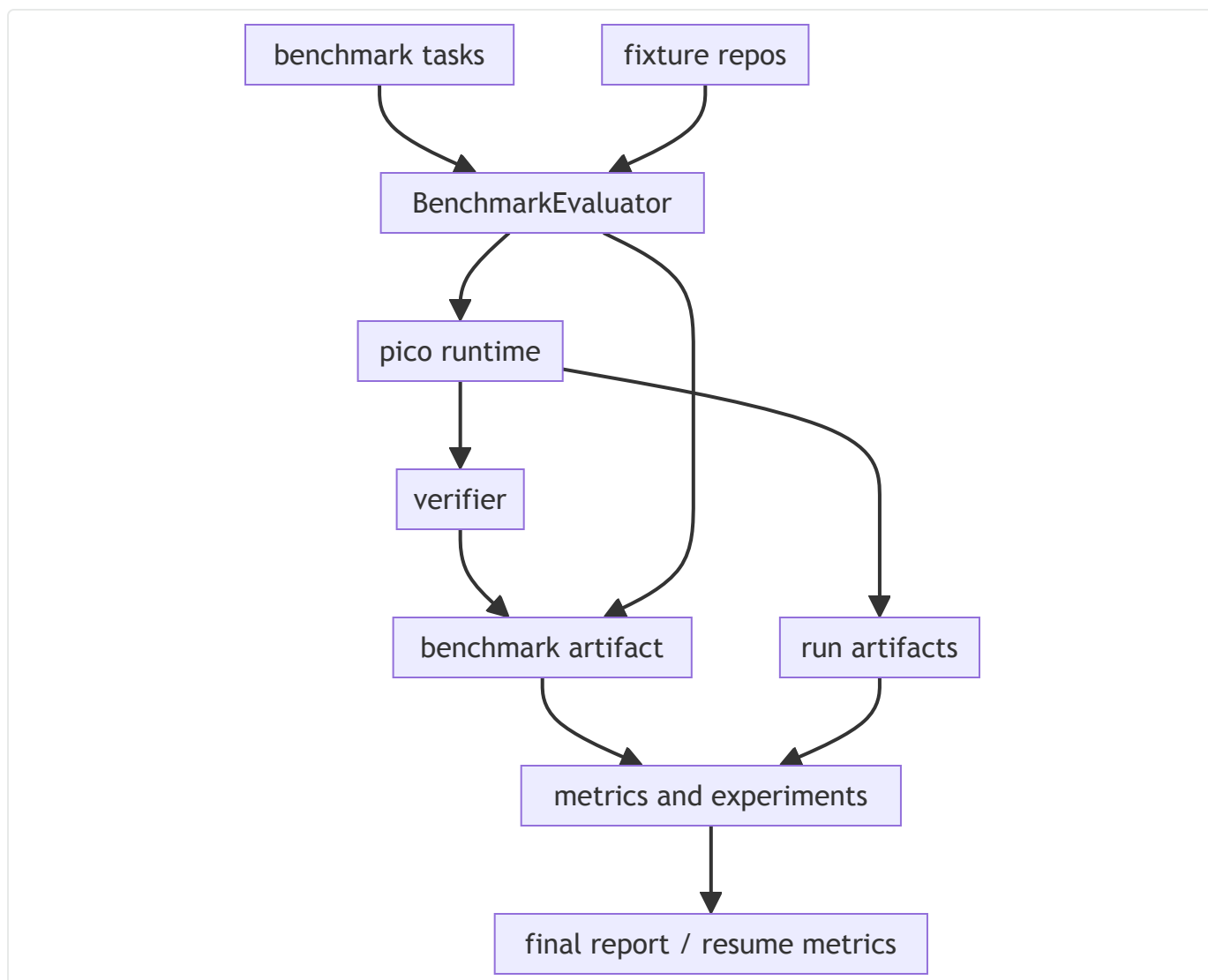
这里真正要回答的是四件事：

- benchmark 任务是怎么被定义成可执行合同的
- 单条任务怎么被判定通过或失败
- 运行过程里的工件怎么继续被聚合成指标
- 为什么不同机制不能都拿同一个大指标去证明

Pico 的评测链路长什么样

当前 `Pico` 的评测链路可以拆成三层。

1. 固定 benchmark 任务集
2. `BenchmarkEvaluator`
3. `metrics.py` 里的聚合和实验层



这张图最重要的地方是，**Pico** 不是只算一个 pass rate。它前面有任务合同，中间有执行器，后面还有工件聚合和实验层。这样最后的数字才有上下文。

benchmark 任务先被定义成合同

固定任务定义在 [benchmarks/coding_tasks.json](#)。

现在一条任务至少长这样：

```

1  {
2      "id": "readme_intro_locked",
3      "prompt": "In the README fixture, replace the placeholder opening sentence with 'This fixture is a locked benchmark workspace.'",
4      "fixture_repo": "tests/fixtures/bench_repo_readme",
5      "allowed_tools": ["read_file", "patch_file"],
6      "step_budget": 4,
7      "expected_artifact": "README.md opening sentence is locked benchmark workspace text",
8      "verifier": "python3 -c \"...\"",
9      "category": "documentation"
10 }

```

这几个字段一起出现以后，任务就不再只是请你改个 README，它会变成一个有约束、有环境、有验证方法的执行单元。

这层合同现在至少规定了：

- 任务 id
- 用户 prompt
- fixture repo
- 允许工具
- 步数预算
- 期望产物
- verifier
- 类别

这就是 `validate_benchmark()` 在做的事情，把自然语言任务拉成一条明确合同。

为什么 evaluator 要复制 fixture

`BenchmarkEvaluator.run_task()` 每次执行任务，都会先把 fixture repo 复制到新的工作目录，再在副本里启动 agent。

这一步很重要，因为 benchmark 的目标不只是跑通一次，它还得保证同一份任务能被反复测，而且环境一致。

主线代码就是这样：

```
1  fixture_source = self.repo_root / task["fixture_repo"]
2  fixture_copy_root = self.workspace_root / task["id"] / fixture_source.name
3  shutil.copytree(fixture_source, fixture_copy_root)
4
5  workspace = WorkspaceContext.build(
6      fixture_copy_root,
7      repo_root_override=fixture_copy_root,
8  )
```

这套做法解决的是任务污染问题。前一轮改过的文件，不应该影响后一轮的判定。

scripted baseline 为什么很重要

当前 evaluator 默认优先走 `FakeModelClient` 加 `SCRIPTED_MODEL_OUTPUTS`。

这层设计很值，因为它先把 harness 自己的正确性和真实模型波动拆开了。

如果一上来就绑真实模型，很多问题会混在一起：

- 是 runtime 有问题
- 还是模型随机波动
- 是 verifier 写得有问题
- 还是任务合同本身不稳

`FakeModelClient` 的存在，让 benchmark 能先回答一件更基础的事，给定固定输出，运行时、工具链、verifier、artifact 这条链能不能稳定跑通。

所以 09 这一层不能只看真实 provider。scripted deterministic baseline 本身就是评测方法的一部分。

一条任务到底怎么判定通过

`BenchmarkEvaluator.run_task()` 跑完以后，不会只看模型有没有回一句 Done，它会看几件条件的交集。

代码就在这里：

```

1  within_budget = task_state.tool_steps <= int(task["step_budget"])
2  verifier_passed = verifier.returncode == 0
3  non_failure_stop_reason = task_state.stop_reason == STOP_REASON_FINAL_ANSWER_RETURNED
4  passed = within_budget and verifier_passed and expected_artifact_exists and non_failure_stop_reason

```

这四个条件缺一个，这条任务都不算 pass：

- 没超预算
- verifier 通过
- 期望工件存在
- stop reason 正常

这样做的好处很直接，agent 说自己做完了，不代表真的算完成。评测层看的是一组显式条件。

failure category 为什么要单独留下来

评测如果只留一个 pass / fail，很快就会不够用。

当前 evaluator 在失败时还会进一步归类：

- missing_artifact
- budget_exceeded
- verifier_failed
- failure_stop_reason

归类逻辑在 `_failure_category()`：

```

1  if not expected_artifact_exists:
2      return "missing_artifact"
3  if not within_budget:
4      return "budget_exceeded"
5  if not verifier_passed:
6      return "verifier_failed"
7  if not non_failure_stop_reason:
8      return "failure_stop_reason"

```

这一层很关键，因为它会把失败继续拆成为什么没过。后面做回归、做面试表达、做不同版本对比，才不会停留在一个模糊失败上。

benchmark artifact 为什么不只是 summary

`BenchmarkEvaluator.run()` 最后会写出完整 artifact，不只是一个 summary。

它里面除了通过率，还会带：

- commit 和 branch
- benchmark 来源
- fixture snapshot id
- model name / version
- decoding 参数
- timezone 和 locale
- 每条 task 的完整 row

这里我觉得很重要的一点，是 `fixture_snapshot_id`。它会把 fixture 文件内容一起做哈希。

这说明 `Pico` 不是只在算结果，也在尽量保住复现上下文。后面你看到一组 benchmark 数字，至少知道它是在哪份任务、哪套 fixture、哪次代码状态下跑出来的。

metrics 层在做什么

如果 evaluator 解决的是单条任务怎么判定，那 `pico/metrics.py` 解决的就是多条结果怎么聚起来，以及不同机制怎么被比较。

它现在主要做三类事：

- 聚合 benchmark artifact
- 聚合 run artifact
- 跑 feature ablation 和实验

比如：

- `aggregate_benchmark_artifact()` 会聚 pass rate、平均步数、失败类别分布
- `aggregate_run_artifacts()` 会聚 attempts、tool_steps、cache、时长、工具状态、安全事件
- memory/context/security/provider 这些实验会继续往简历和报告口径走

这层的价值很直接，评测不再只是一份 benchmark JSON，而是能继续长成指标和报告。

为什么不同机制不能都用同一个大指标

这一点在 `Pico` 里其实做得很对，不是所有模块都硬拿成功率去证明。

举几个例子：

- 上下文治理更适合看 `prompt_chars`、压缩率
- 记忆层更适合看重复读取和 follow-up 工具步数
- 安全治理更适合看拦截事件和错误码分布
- provider 对照才更适合看 `pass_rate / avg_attempts / avg_tool_steps`

也就是说，`Pico` 的评测层不会只产出一个大指标，它会尽量让指标贴机制本身。

这一点很重要。什么都用 pass rate 去证明，最后反而讲不清楚每个设计到底带来了什么。

run artifact 为什么也要进评测链

只讲 benchmark 还不够。

因为 `Pico` 还有一类很重要的输入，不只 benchmark artifact，还有运行时自己的工件：

- `task_state.json`
- `trace.jsonl`
- `report.json`

`aggregate_run_artifacts()` 会把这些工件继续聚成：

- `avg_tool_steps`
- `avg_attempts`
- `cache_hit_rate`
- `tool_status_counts`
- `security_event_counts`
- `stop_reason_counts`

这层做完以后，评测就不再只是任务过没过，还会继续回答它是怎么过的，或者为什么会停在那里。

provider 实验为什么也在这一层

`run_provider_experiments()` 现在已经接进来了，而且处理得很工程化。

它不是只在 provider 成功时给结果，失败时也会结构化保留：

- `completed`
- `blocked`

- `error`

这点很重要，因为 provider 对照实验最怕只在成功的时候记录，失败就当没发生过。

当前 provider runner 会让 GPT 和 Claude 走同一套 benchmark 任务、同一套 artifact 输出和汇总逻辑。这样最后比较的就不只是接口有没有接通，而是同一评测框架下的结果。

这一层现在怎么被验证

这层代码现在至少有三类锚点。

第一类是 evaluator 测试

`tests/test_evaluator.py` 已经覆盖了：

- benchmark schema 验证
- fixture copy 和独立 run 目录
- 成功判定逻辑
- reproducibility 元数据
- summary 聚合

第二类是 metrics 测试

`tests/test_metrics.py` 已经覆盖了：

- benchmark artifact 聚合
- run artifact 聚合
- memory/context/security/provider 结果合并
- blocked provider 状态

第三类是结果文档和 artifacts

现在仓库里已经有：

- `artifacts/benchmark-v1.json`
- `artifacts/provider-experiments.json`
- `artifacts/final-resume-metrics.json`
- 以及对应的 metrics 报告文档

所以 09 现在不是只有设计说明，代码、测试和产物三层都已经在了。

这一层现在没做什么

Pico 的评测层已经够站住，但边界也很明确。

- benchmark 任务还不算多
- 任务类型还偏固定、偏可控
- 真实 provider 对照还受外部账号和服务可用性影响
- 现在更多是均值和类别统计，还没有更细的方差和稳定性视角

这些都不是问题被忽略了。当前阶段先把合同和证据链立住，更重要。

如果继续往下做，优先补什么

我会优先补三件事。

一，把 benchmark 任务类型再拉宽一点

现在链路已经成立，后面最值得补的是更贴近真实仓库排查和修改的任务，不只是固定 patch 类任务。

二，把 synthetic 和 real provider 结果边界继续讲清楚

这层现在代码里已经分开了，后面报告和文档还可以把口径再收严一点，避免读者把不同实验混着看。

三，把稳定性视角补进来

后面如果实验规模继续扩大，均值之外再补方差、波动范围和重复性指标，会更完整。

面试里怎么讲这一层

Pico 的评测层会把任务合同、执行器、运行工件和聚合层接成一条证据链。benchmark 任务先明确 fixture、允许工具、步数预算、期望产物和 verifier；BenchmarkEvaluator 再在干净副本里跑任务，把 pass / fail 和 failure category 结构化记下来；后面的 metrics.py 继续把 benchmark artifact 和 run artifact 聚起来，去回答上下文治理、记忆、安全、provider 这些机制到底有没有收益。这样最后的数字不会只是一轮演示结果，而是一套带上下文的评测口径。

总结

Pico 的这一层设计，核心是把评测做成一条证据链。

前面有 benchmark 合同，中间有 evaluator 执行器，后面有 run artifact 聚合和 feature 实验。这样你最后看到的不只是 pass rate，还有失败解释、过程指标和不同机制的收益方向。对一个本地 coding harness 来说，这一层会直接决定项目是不是能从系统描述走到可验证的工程样本。

【重要】90-针对项目描述的逐字面试话术

1. Harness相关

Agent Harness 架构设计：负责本地代码 agent 的整体设计与开发，统一模型接入、工具执行、会话状态、checkpoint 恢复和运行工件落盘流程，形成可复盘的执行链路；支持 2 类模型后端、7 类工具和 3 类运行工件。

1.1. 为什么把 Pico 定义成 harness? 你对harness的理解?

先从对harness的理解说起吧。

agent 负责决定下一步做什么

harness 负责规定agent在什么边界内做、怎么记录、怎么验证、什么时候才算真的完成。

对于agent工作的时候，harness其实就是围绕模型/agent，去补五类东西：

1. 它该怎么工作
2. 它现在做到哪了
3. 什么算完成
4. 它不能乱做到哪里去
5. 一次会话怎么开始、结束、交接

一个好的harness，最核心的标准是，用同样的模型，好的 Harness 能在同样的任务上表现更好，并且可以不经人为干预地运行得更久、更稳定

所以在设计pico之前，我看了anthropic的harness文章和网上相关的最佳实践，还包括OpenHarness这个项目，在设计的过程中，我把更多的时间花在了思考代码仓库里怎么不失控、运行时主循环怎么推进、prompt 怎么按结构拼起来，工具怎么显式注册和校验等问题，目的就是想让 agent 放进受控环境里跑的系统。

最后pico的设计上，它有 runtime，负责把用户一句话推进成一轮轮可控执行。

它有工具治理，不是看到工具调用就直接跑，而是先检查工具是否存在、参数是否合法、是不是重复调用、当前风险策略允不允许。

它有 memory 和 context manager，解决当前 prompt 里到底什么该进、什么不该进的问题。

它还有 run artifacts 和 benchmark，把过程和结果都留了下来，后面既能回放，也能评测，还能做不同机制的对照。

所以如果把这几层合起来看，我想解决的其实不是模型会不会写代码，而是能不能把一个 code agent 做成一个有状态、有边界、有验证、也有回归能力的工程系统。这也是为什么我最后会把 Pico 定义成 harness。

1.2. 讲一下从输入命令到agent执行完毕，整个pico到运行链路是怎么样

pico 从输入命令到执行结束，可以分成五层。

第一层是入口层。

用户在终端输入命令以后，系统先做**启动参数解析**，比如工作目录、模型后端、执行预算、审批策略这些。

这个阶段的目标是把用户的一条命令转换成一个完整的运行上下文。

这里会确定三件事：当前在哪个仓库里工作、这次运行用哪个模型、这次运行是接着旧 session 继续还是新开一次任务。

第二层是上下文构建层。

agent 真正开始推理前，不会直接拿用户原始请求去问模型，而是先把当前系统能提供的上下文整理成一个结构化输入。

这里我把上下文分成几种不同类型的数据：

- 稳定上下文，比如工具能力、系统规则、仓库概要
- 短期工作记忆，也就是当前任务相关的最近文件、最近摘要、临时笔记
- 相关长期记忆，也就是过去沉淀下来的稳定事实
- 会话历史，也就是前面几轮发生过什么
- 当前用户请求

这个设计的核心思想是，不同寿命的数据不能混成一团。稳定部分应该尽量可复用，短期部分应该围绕当前任务滚动更新，长期部分应该只在相关时被召回。

第三层是 agent 控制循环。（runtime）

这部分是整个系统的核心。

它不是一次性把 prompt 发给模型然后等答案，而是一个持续推进的闭环：

1. 构建当前轮上下文
2. 调模型做决策

3. 判断模型输出的是最终答案还是动作请求
4. 如果是动作请求，就去执行工具
5. 把执行结果写回系统状态
6. 进入下一轮

第四层是执行与状态更新层。

如果模型决定调用工具，系统不会直接照做，而是先过一层执行网关。

这层主要负责几件事：

- 校验动作是否合法
- 判断是不是重复动作
- 判断是不是高风险动作，需要不要审批
- 真正执行动作
- 把结果转换成系统下一轮可以消费的状态

这里最重要的设计点是，工具结果不会原样无限堆积，而是会被拆成两类：

- 一类进入会话历史，保证可回放
- 一类进入短期工作记忆，保证下一轮还能高效利用

比如读文件这种行为，更适合沉淀成最近读过哪些文件和这些文件的短摘要，而不是把整个文件内容一直带着走。

第五层是持久化与审计层。

当一轮任务结束以后，系统会把这次运行留下两种不同性质的数据。

第一种是 **可恢复状态**。

也就是 session 级别的数据，它服务的是“下次还能接着干活”。

第二种是 **运行工件**。

也就是 task state、trace、report 这类单次运行记录，它服务的是“解释这次到底发生了什么”。

它让 agent 系统不只是能跑，还能复盘、调试、评估。

最后我总结一下

这五层设计分别解决了不同的问题：

- 入口层解决“怎么把命令变成运行现场”
- 上下文构建层解决“怎么把不同寿命的数据组织成模型可消费的输入”
- 控制循环解决“怎么让 agent 持续推进，而不是一轮就结束”
- 执行与状态更新层解决“怎么安全调用工具，并且把结果沉淀成下一轮还能用的状态”
- 持久化与审计层解决“怎么让系统可恢复、可解释、可评估”

PS：对于面试的时候长回答，最好采用总分总的形式，最后收个尾，让面试官有印象，方便提问，你知道的，大家的注意力在AI时代已经慢慢消散了

1.3. 工具怎么治理

我在 Pico 里对工具的看法，不是模型需要什么能力，我就给它几个函数，而是把工具当成 agent 系统里最重要的风险入口来设计。因为模型最多只是提出一个动作意图，真正会对工作区产生影响的是工具执行这一步。

所以 Pico 的做法是，所有工具调用都必须先过一层统一的执行网关。

这层主要负责几件事。

第一，先检查工具是否合法，参数是不是完整、是不是越界。

第二，检查是不是重复调用。同一个动作刚做过又来一遍，很多时候说明 agent 已经开始出问题了。

第三，区分风险等级。像读文件、搜索这种偏只读的操作，风险低；写文件、打补丁、跑 shell 这种会改工作区的动作，风险高。高风险动作要受审批策略控制。

第四，执行结果也不能只分成功和失败两类。有些场景下命令虽然报错了，但工作区已经变了，这种半成功如果被系统当成普通失败忽略掉，后面就很危险。

在工具治理这里，pico真正解决的不是能不能调用，而是怎么让调用这件事不失控。

1.4. 会话状态、checkpoint 和运行工件怎么分层

agent 一旦做长链路任务，最容易乱的就是状态。

如果会话状态、恢复状态和运行日志全混在一起，短期也许还能跑，但后面一定会越来越难恢复、越来越难解释、也越来越难评测。

所以 Pico 里我把它拆成了几层，而且每层职责很明确。

第一层是会话状态。

它解决的是下次还能不能接着干。这里面放的是会话历史、工作记忆、长期记忆这些会跨轮延续的内容。

这一层服务的是任务连续性。

第二层是checkpoint。

这一层专门服务恢复。它保存的是如果现在上下文超了，或者运行被打断了，下次从哪继续的短状态。

这里面最关键的是当前目标、关键文件、当前卡点、下一步动作，以及这些状态对应的锚点现在还可不可以继续信。

它和普通会话状态不一样，它不是把所有过程都存下来，而是只保留恢复时必须信的那一份状态。

第三层是运行工件。

这一层不是为了恢复，而是为了回答这次到底发生了什么。

比如当前这轮停在什么状态、过程里发生过哪些关键事件、最后的结果和指标是什么，这些都应该作为单次运行工件保存下来。

它的价值在于复盘、调试和评测，而不是作为下次恢复的主入口。

pico的设计中很注意恢复和复盘不能混的思想。

恢复需要的是还能继续工作的状态，复盘需要的是能回看的过程证据。

如果把这两类东西混在一起，系统最后一定会陷入一个问题：该恢复的时候不知道该信哪份状态，该复盘的时候又不知道哪些是主状态、哪些只是过程日志。

1.5. 怎么复盘和评测，为什么说它是可复盘的执行链路

pico的复盘设计：

Pico 每次任务跑完以后，不只会留下最后答案，还会把这次运行拆成几层信息保存下来。最上面一层是这次任务最后停在什么状态，比如到底是正常完成了，还是卡在预算、恢复或者工具问题上。中间一层是过程事件，也就是这一轮 prompt 怎么组的、模型做了几轮决策、调了哪些工具、哪一步被拦住了。最后一层是结果摘要，也就是这次运行最终产物、关键指标和停止原因。

所以 Pico 的复盘不是回头翻一整段聊天记录，而是可以顺着这几层信息往回查。

比如这次结果不对，我可以先看它最后停在什么状态；如果状态正常，再往下看过程里有没有重复调用、有没有工具失败、有没有恢复判错；如果是上下文问题，还能继续看这轮 prompt 是怎么组出来的。也就是说，复盘时不是只能靠猜，而是能顺着运行留下来的状态和事件，一层层把问题定位回来。

再说评测。

Pico 的评测我没有做成一个总分，而是分层去看。最底下一层是固定任务集，用来保证 Harness 本身的合同稳定，比如工具能不能按规则跑、预算有没有超、验证能不能过。往上一层是对照实验，专门看某个机制到底有没有带来收益，比如上下文治理会不会真的把 prompt 压下来，记忆会不会减少重复读文件，恢复机制会不会误信旧状态。再往上一层，模型后端表现单独看，不和 Harness 结构收益混在一起。

这样做的好处是，复盘和评测能接起来。

复盘告诉我这次为什么出问题，评测告诉我这套机制整体上到底有没有用。

比如如果恢复实验里某类场景指标不好，我就可以回到具体运行工件去看，到底是旧状态没识别出来，还是恢复入口选错了。反过来，如果某次运行里发现异常，也可以看它是不是 benchmark 里已经暴露过的共性问题。

所以我说 Pico 是可复盘的执行链路，意思不是它有日志，而是一次运行结束以后，我既能把过程拆开讲清楚，也能把这些运行结果继续汇总成评测指标，最后知道问题出在哪一层、机制有没有真正生效。

2. 上下文治理

长上下文治理：设计分层上下文管理与预算裁剪机制，在 12 组长上下文配置里，将平均 prompt 长度从 7082 压到 5664，平均压缩率 16.19%，最高压缩率 33.28%，同时保证当前请求不被裁坏。

2.1. 上下文治理具体在治理什么

具体是在解决一个具体的问题：**多轮任务一长以后，agent 每一轮到底该看到什么，不该看到什么。**

因为 code agent 和普通聊天不一样，它不是回答一句话就结束了，而是在一个真实仓库里持续做判断。随着任务往下走，系统会不断积累不同类型的信息，比如工具能力、仓库背景、最近读过的文件、已经确认过的事实、过程里的历史记录，还有当前这一轮最新的用户请求。这些信息全都重要，但它们的重要程度和有效期是不一样的。

如果没有治理，最容易出现两个问题。

第一个问题是**上下文越来越胖**。很多旧信息其实只是某一轮临时有用，但它会一直留在后面的 prompt 里，最后把模型真正该关注的内容淹掉。

第二个问题是**信息混层**。稳定背景、当前任务状态、历史过程、相关记忆全混在一起，模型每次都像在一堆旧纸堆里翻东西。短期看好像“信息很全”，长期看反而更容易跑偏，因为它不知道这轮最该抓什么。

所以 Pico 这块的目标，不是追求让模型看得越多越好，而是让模型每一轮都尽量站在**当前最相关、最干净的一份上下文**上做判断。换句话说，我治理的不是长度本身，而是**上下文结构和优先级**。

最后这个设计会直接影响 agent 的三个能力。

第一，影响它会不会重复劳动；

第二，影响它会不会在长任务里抓错重点；

第三，影响它在预算不够的时候，是不是还能保住最关键的信息。

所以治理的不是 prompt 有多长，而是长任务里不同寿命、不同优先级的信息怎么在每一轮被正确组织起来。

2.2. 上下文是怎么分层的，为什么要这样分

Pico 里的上下文，我不是把所有东西拼成一段，而是故意拆成几层。

因为我觉得不同性质的信息，不应该混在一个平面上给模型看。

第一层是**稳定背景**。

这里放的是每一轮都需要的东西，比如 agent 的角色、工具怎么调用、当前仓库的一些稳定信息。这部

分的特点是变化慢、复用高，所以适合单独放着，不要跟每轮历史混在一起。

第二层是工作记忆。

这层放的是当前任务工作面上最常用的信息，比如任务摘要、最近接触过的文件、文件短摘要和少量过程笔记。它的特点是贴近当前任务，但也不是所有过程都要常驻，只留下一轮大概率还会继续用到的部分。

第三层是相关记忆。

这层不是所有记忆都进，而是针对当前这轮请求，挑出少量最相关的内容带回来。这样做的好处是，记忆不会整体撑大上下文，而是按需回来。

第四层是过程历史。

这层保留的是执行过程，包括前面几轮问答、工具调用和中间步骤。它当然有价值，但它更偏“过程证据”，不应该天然比当前任务状态优先级更高。

最后一层是当前请求。

这层我会单独放，而且必须完整保留。因为不管上面有多少背景、记忆和历史，这一轮模型最后到底要解决什么问题，还是由当前请求决定的。

这样分的原因是只有分层以后，系统才能在预算不够的时候做正确的取舍。比如相关记忆可以先收，历史可以压缩，但当前请求不能丢。再比如稳定背景适合复用，工作记忆适合滚动更新，过程历史适合按需压缩。这些动作都建立在“它们不是一类东西”这个前提上。

这其实是在做**信息职责分离**。如果不分层，后面所有事情都会变粗糙：裁剪会变得粗暴，召回会变得泛滥，恢复也会变得混乱。而一旦分层清楚，系统就知道哪些东西该常驻，哪些东西该滚动更新，哪些东西该在相关时再回来。

上下文分层的**目的**，不是为了结构好看，而是为了让系统知道每一类信息该怎么保留、怎么裁、怎么复用。

2.3. 预算裁剪是怎么工作的，为什么不会把重要信息裁坏

预算裁剪这块，是按优先级来裁剪。

Pico 的思路是：

先承认一个事实，prompt 是一个受预算约束的运行资源，不可能无限增长。那真正关键的问题就不是裁不裁，而是先裁谁、后裁谁、哪些绝对不能动。

所以我在设计上是有**一个固定顺序**的。

越偏补充性的内容，越早裁；越贴近当前任务主线的内容，越晚裁。

比如相关记忆可以先收，因为它是按需补充的；

过程历史也可以压缩，因为里面很多是旧过程，不一定每轮都要完整保留；

工作记忆再往后收，因为它已经更贴近当前任务；

稳定背景一般尽量保持稳定；

而当前请求是最后一层，而且必须完整保留。

也就是说，我做的不是把所有部分平均裁一点，而是一个比较明确的优先级裁剪。

这样系统即使在预算紧张的时候，也尽量还能保住两件最重要的事：

第一，模型知道这轮要解决什么；

第二，模型手里还有足够的当前任务状态，不会直接退回到瞎猜。

重要信息不会裁坏：

严格一点说，不是永远不会，而是我专门把最危险的点拿出来测了。用实验去支持

这轮实验除了看压缩率，还看“当前请求有没有被裁坏”。最后这一项是 100%，也就是说，在这组上下文配置里，当前请求都被完整保住了。

Pico 不追求压缩到最极致，而是追求在不破坏当前任务语义的前提下，尽量把冗余部分收掉。

从 system design 的角度，这里体现的是一种有保底约束的降级策略。

预算不够不是异常，而是长任务里的常态。关键在于，系统有没有定义好：预算紧张时，哪部分可以收，哪部分必须保。Pico 的裁剪不是为了追求更短，而是为了在预算不够的时候，先收补充信息，把当前请求和任务主状态保下来。

2.4. 这 12 组长上下文配置是怎么设计出来的

这是按几个最直接影响上下文压力的变量组合出来的。我当时想的是，长上下文问题本质上不是一个单变量问题。

它至少受三类东西影响：

第一类是历史长度。

也就是这轮前面到底已经积累了多少过程信息。历史短的时候，上下文压力和历史长的时候完全不是一个问题。

第二类是记忆负载。

包括工作记忆和相关记忆里到底有多少内容要带回来。因为有的任务历史不长，但记忆层很重，照样会把 prompt 挤大。

第三类是当前请求长度。

有些请求很短，有些请求本身就很长，而且表达里会带更多约束。如果请求本身比较长，裁剪策略是不是还稳，这也是要单独测的。

所以我最后把这三类因素做组合。

大致上是：历史做几档、记忆负载做几档、请求长度做几档，最后组合出 12 组不同压力条件。

这样测出来的不是某个单点结果，而是这套上下文治理机制在不同上下文压力下的整体表现。

我之所以不用更多配置，是因为 benchmark 不是越多越好。

如果只是不断加样本，但变量结构不清楚，最后只能得到一堆零散结果，面试里很难讲。

12 组对我来说是一个比较合适的平衡：

一方面足够把主要变量覆盖到，另一方面结果还讲得清楚。

这 12 组配置把历史长度、记忆负载和请求长度这几个主要压力维度系统地覆盖到了。

2.5. 数字问题：平均压缩率 16.19% 看起来不算特别高，为什么你觉得它有价值

Pico 这套上下文治理追求的不是压得越狠越好，而是在不破坏当前任务语义的前提下，把冗余部分收掉。

如果我只追求压缩率，最简单的方法其实是更激进地裁历史、裁记忆，甚至把当前请求也压一下。这样数字会更漂亮，但 agent 在真实任务里很可能直接失真。

对本地 code agent 来说，这种高压压缩率没什么意义。

我更看重：

第一，这个 16.19% 不是单点极限，而是 12 组配置下的平均结果，说明这套机制不是只在个别场景好看。

第二，我这里还有一个更重要的保底条件，就是当前请求没有被裁坏。也就是说，这个压缩率不是拿任务主线去换来的，而是在保住当前任务语义之后得到的结果。

另外一个角度是，平均值本来就会被短配置拉下来。

如果上下文本来就不长，可压缩空间自然有限。真正更能说明机制上限的，是最长那几组配置下的结果。这也是为什么我会同时看平均压缩率和最高压缩率。最高到 33.28%，说明在上下文压力真正上来的时候，这套机制是能把冗余显著收掉的。

这组结果在我看来，更像一个健康的工程结果。

它没有为了追数字去牺牲稳定性，而是在长任务里稳定地释放了一部分上下文预算。

对 code agent 来说，这种收益比追一个特别夸张、但很脆弱的压缩率更有实际价值。

3. 记忆系统

结构化记忆系统：针对多轮任务里 agent 反复读同一文件、重复确认已知事实的问题，把任务摘要、文件摘要、过程笔记和相关记忆召回做了分层；在 12 个记忆依赖任务里，follow-up 阶段的重复读文件次数从 60 次降到 0 次，且不再需要额外工具调用去重新确认已经拿到的事实。

3.1. 结构化记忆指什么

是指把不同类型、不同寿命的信息拆开管理。

因为 code agent 在真实仓库里做多轮任务时，信息其实不是一种东西。

有些信息是当前任务马上还要继续用的，比如现在的任务目标、最近碰过哪些文件、这些文件大概讲了什么；

有些信息是过程里的短结论，比如这个方向试过了、不成立，或者这个问题已经确认过了；

还有些是更稳定的项目事实，比如项目约定、关键决策、依赖要求，这类东西不应该和当前任务状态混在一起。

如果这些信息不拆层，全部混成聊天历史或者一段自由摘要，后面会出两个问题。

第一，系统不知道哪些内容应该常驻，哪些内容只是暂时有用，最后 prompt 会越来越乱。

第二，恢复、召回、裁剪这些动作都没法做精细判断，因为所有东西都长在一层里。

所以 Pico 里我把记忆分成了几层去处理。

最靠近当前任务的是工作记忆，里面放任务摘要、最近接触过的文件、文件短摘要和少量过程笔记；

更稳定一点的事实，单独作为长期记忆沉淀下来；

而真正跟恢复强相关的当前状态，我不放在记忆层里，而是放在 checkpoint 里。

这样就把下一轮可能有用的信息和恢复时必须信的状态拆开了。

从 system design 角度看，这件事本质上是在做信息分层和职责分离。

不是所有记住的东西都应该放进同一个容器里。

Pico 现在做的，就是把记忆从模糊的上下文堆积变成“裁剪、可召回、可失效、可恢复配合的运行时部件”。

让 agent 在多轮任务里记得住、拿得准、也不会越记越乱。

3.2. 为什么记忆要分层，而不是把历史或摘要直接塞进 prompt

有很多上下文不等于上下文有用。

如果只是把历史记录或者一段大摘要不断塞回 prompt，短期看很省事，但任务一长，问题会越来越明显。

第一个问题是上下文膨胀。

历史记录里混了很多过程性信息，比如已经读过的文件、失败过的尝试、旧的判断、已经完成的步骤。这些东西留着当然有价值，但不是每一轮都该带进模型。你如果不分层，最后 agent 每轮都要重新读一遍自己刚做过的事，成本越来越高。

第二个问题是信息寿命不同。

比如最近看过 README 这种信息，可能下一轮还很有用；

但刚才这个补丁失败了这种信息，可能只在后面一两轮有用；

再比如这个项目默认是只读模式这种约定，它又更稳定。

如果全部塞进一层，你就没法判断这轮到底该带什么、不该带什么。

第三个问题是后面的系统能力会跟着受影响。

比如裁剪的时候，哪些该优先保留；

召回的时候，哪些该按相关性带回来；

恢复的时候，哪些状态能继续信；

这些判断都依赖信息本身是不是分了层。

如果没有分层，系统做不出细粒度判断，只能粗暴截断或者全量重读。

所以 Pico 里做的是先把信息拆成几层，然后再按当前任务的需要去组装。

稳定背景放一层，工作记忆放一层，相关召回放一层，历史过程再放一层，当前请求永远单独保留。

这样做的好处是，模型每一轮看到的就是这轮最该看到的信息。

从 system design 的角度看，这其实是在做预算管理 + 优先级管理。

因为 prompt 本身就是受预算约束的运行资源，你不可能无限往里面塞内容。

关键不是记没记，而是不同层的信息在预算不够时谁先保、谁先裁、谁按需召回。

3.3. 相关记忆召回是怎么做的，怎么决定这一轮带什么

相关记忆召回这块，我的设计重点没有放在做一个很重的语义检索系统上，而是先把这一轮最该带什这件事做清楚。

Pico 里，一轮新的请求进来以后，不是把所有记忆全塞回 prompt，而是先去思考：这次任务到底跟哪些已有信息最相关。

然后系统再从工作记忆和长期记忆里，挑少量最相关的内容带回来。

我现在的做法比较克制，主要还是基于几类比较透明的信号。

比如标签有没有直接命中，关键词有没有重叠，这条信息离当前任务远不远，它是最近才确认的，还是很久以前的旧结论。这样做的好处是可解释。后面如果我想知道为什么这一条记忆被召回了，我是能说清楚的，而不是只能说模型觉得它像。

这里我刻意没有一上来做很重的语义检索，原因有两个。

第一，Pico 现在的体量还没大到必须上向量检索那一步；

第二，对本地 harness 来说，可解释性很重要。

如果召回变成一个很黑盒的过程，虽然可能更聪明一点，但后面在调试、做 benchmark、做面试展开的时候，反而更难说清。

从上下文组织上看，我把它放在一个单独的相关记忆层里，而不是混到工作记忆和历史里。

这样每一轮 prompt 里就会形成一个很清楚的结构：

稳定背景、工作记忆、相关记忆、过程历史、当前请求。

也就是说，相关召回不是到处散着发生，而是在 prompt 组装时有一个明确入口。

3.4. 怎么保证记忆不过期，不会拿旧事实继续推理从而产生错误

对 code agent 来说，过期的记忆比没有记忆更危险。没有记住，最多就是多读一遍文件；但如果记住的是旧结论，还继续拿它往下推理，那后面整条链路都可能偏掉。

所以 Pico 里我专门做了一层记忆和真实工作区对齐的约束。最典型的就是文件摘要不会只存一句话，它还会绑定到具体文件，并且带一个新鲜度标记。这样下一轮系统在准备上下文时，不是直接把旧摘要拿出来，而是会先确认：这个文件现在还是不是当时那个状态。

如果文件已经变了，那旧摘要就不能继续当成当前事实。

这时候系统要么让它失效，要么重新读取、重新锚定。

这样做把记忆从静态缓存变成受工作区约束的缓存。

也就是说，记忆从来不是绝对真相，它只是对某一时刻真实状态的压缩表示。

这件事和恢复机制其实是能接上的。

因为恢复时你同样不能盲信旧状态，得先看这些锚点现在还成立不成立。

所以我在 Pico 里一直强调一个思路：

不管是记忆还是恢复，它们都不能脱离当前活跃的工作去单独存在。

系统最后真正该信的，永远是当前工作区的真实状态，而不是某一轮留下来的文本。

这里我做的有点类似**缓存失效机制**。很多系统容易在怎么记住上花很多力气，但真正让系统长期稳定的，往往是什么时候该承认旧信息已经不能再信了。Pico 这块的约束，就是先处理失效，再谈复用。

Pico 不是在追求记得越多越好，而是在追求记住的东西尽量别过期；因为对 code agent 来说，错用旧事实比重新读一遍文件更危险。

3.5. 你怎么证明这套记忆真的有用？为什么重复读文件能降到 0

我设计记忆实验时，重点看的不是总成功率，而是 follow-up 阶段两个更贴近执行过程的指标：

第一是重复读文件次数，第二是工具步数。

因为这两项更直接反映 agent 有没有把已经确认过的事实真正留下来。

实验方法上，我做的是对照。

同一批记忆依赖任务，分成三组跑：

一组开记忆，一组关记忆，还有一组保留记忆框架但塞无关内容。

这样就能区分：收益到底是来自多塞了一点上下文，还是来自结构化、相关的记忆真的在工作。

最后跑出来的结果是，开记忆之后，在 follow-up 阶段重复读文件从 60 次降到 0，而且不再需要额外工具调用去重新确认已经确认过的事实。

这个结果看起来会比较夸张，但我反而觉得它合理。因为这批任务本来就是专门设计成如果记住了，就不该再读第二遍的那种依赖任务。所以它不是说明 agent 在所有任务里都会这样，而是说明在依赖前面已确认事实的场景里，结构化记忆确实把重复劳动压掉了

它证明了两件事。

第一，记忆收益不是多带点上下文这么模糊，而是能具体体现在执行过程上；

第二，Pico 的记忆层不是为了让模型显得更聪明，是为了让 agent 在多轮任务里少绕弯路。

我验证记忆层时，重点不是看它让模型多答对了几题，主要看它有没有让 agent 少做已经做过的事。

对 code agent 来说，这才是记忆最直接、也最值钱的收益。

4. 任务恢复机制

任务恢复机制：设计 checkpoint / resume 机制，让 agent 在上下文超预算、中断恢复和 workspace 漂移场景下恢复任务状态，而不是重读整段聊天历史；覆盖 10 个恢复场景，workspace 漂移识别率 100%，且没有出现误信旧状态继续执行的情况。

4.1. 怎么设计的这些恢复场景的实验

我设计这些恢复场景的时候，首先思考了一个问题：恢复这件事最容易在哪些地方出错。

对code agent 来说，恢复真正危险的地方不是恢复失败，而是恢复错了还继续跑。所以我不是从任务类型出发去设计，而是从恢复风险出发去拆场景。最后我把恢复这块拆成了五类边界，每类做两个场景，一共十个。（数量不是最重要的，最重要是覆盖，当然你也可以多加一些实验，这也是没问题的）

我设计这些恢复场景时，不是为了把数量堆到 10 个，而是先反过来看：**恢复这件事最容易在哪些地方出错。**

因为对 code agent 来说，恢复最危险的不是恢复失败，而是**恢复错了还继续跑**。所以我不是按任务类型去列 case，而是按恢复风险去拆。最后我把恢复这块分成了五类边界，每类两个场景，一共十个。

第一类是基础 checkpoint 恢复。

主要验证系统在已经有 checkpoint 的情况下，能不能把任务状态带回来，直接继续推进，而不是重新开始。这里重点看两件事：目标能不能带回来，关键文件上下文能不能带回来。

第二类是部分状态过期。

也就是 checkpoint 不是完全坏了，但里面有些文件锚点已经不对了。这里我拆成单文件过期和多文件过期，重点看系统会不会重新校准，而不是继续静默沿用旧状态。

第三类是workspace 漂移。

这里测的不是单个文件，而是恢复现场是不是已经不是上一次那个现场了，比如工作区指纹变了，或者运行条件变了。重点看系统能不能把这种情况识别成 mismatch，而不是装作没事继续做。

第四类是checkpoint 自身不兼容。

也就是 schema mismatch。这里一类是 checkpoint 还在，但结构版本不兼容；另一类更极端，就是不光版本不对，也没有可继续信的恢复状态。这个就是那 10 个里唯一没有继续恢复成功的场景。我故意保留它，是为了验证系统在没有恢复基础的时候，会不会误把旧状态当成可用状态继续往下跑。

第五类是工具半成功后的恢复。

比如命令报错了，但工作区其实已经变了。这种情况最怕系统把它当普通失败忽略掉。所以我这里专门

测 shell 半成功和工具半成功两种情况，看恢复时能不能意识到“状态已经变了”。

这套设计最重要的是验证三件事：

1. 有 checkpoint 时，系统能不能接着做
2. 旧状态部分失效时，系统会不会重新校准
3. 没有恢复基础时，系统会不会误信旧状态继续执行

4.2. 恢复任务状态指什么

恢复任务状态，是指把这件任务还能继续做下去所需要的最小状态重新拿回来。

对 code agent 来说，真正重要的不是上一次聊过什么，而是现在做到哪了，下一步该往哪走。

所以我在 Pico 里，把恢复状态收成了几类比较明确的东西：当前目标是什么，哪些文件是这轮最关键的，当前有没有卡点，下一步建议动作是什么，以及这些状态对应的文件锚点现在还可不可以信。

换句话说，我想恢复的东西是一份还能继续工作的短状态。

这个设计的理念是，一旦任务变长，history 会越来越大，里面混了很多过程噪音，比如已经失败过的尝试、临时性讨论、旧版本结论。这些东西拿来复盘很有用，但拿来恢复主状态就很不稳定。

Pico 这边的做法是，session 里保留完整 history 方便审计，但真正恢复时，优先走 checkpoint，再结合 working memory 和 freshness 校验来决定哪些状态还能继续信。

（供大家理解参考：落到代码上，这条链路主要在 runtime.py 里。比如 evaluate_resume_state() 会先看当前 checkpoint，再看 key files 的 freshness，再看 runtime identity 有没有对不上；真正进入 prompt 的时候，ContextManager 不会把旧 prompt 直接重用，而是重新组装一份新的上下文，把 checkpoint 里的状态作为“Task checkpoint”那一段塞进去。）

最终把 agent 从我知道我们聊过很多东西切回我知道这件任务现在做到哪了，而且我知道接下来该怎么继续。

4.3. checkpoint是怎么设计的，和 memory 的区别是什么

checkpoint 在 Pico 里，本质上是恢复入口。专门解决一个问题：如果上下文太长了，或者运行被打断了，下次系统到底从哪继续。

所以我在设计 checkpoint 的时候，放进去的是恢复时真正要信的状态。

比如当前目标、当前卡点、下一步动作、关键文件，还有这些关键文件对应的锚点是不是还成立，再加上一些运行现场信息。这样下次恢复的时候，系统可以直接用一份已经整理过的短状态重新起步。

checkpoint和 memory 的区别，在于checkpoint 是为了恢复，memory 是为了复用。

memory 更像工作记忆，它保存的是下一轮大概率还会继续用到的内容，比如任务摘要、最近接触过的文件、文件短摘要、过程笔记，还有后面加的轻量长期记忆。它的作用是减少重复劳动，让 agent 不要反复读同一个文件、反复确认已经确认过的事实。

但 checkpoint 不是干这个的。checkpoint 承载的是当前任务主状态，也就是这轮恢复时必须信的那份状态。像当前目标、当前卡点、下一步动作，这些如果长期放到 memory 里，后面就会和 checkpoint 抢职责，系统会越来越乱。所以我在设计上故意把它们拆开：让 memory 管下一轮可能有用的信息，checkpoint 管恢复时必须信的状态

4.4. 为什么不直接重读聊天历史

最核心的原因是太贵而且不稳定

太贵在于：

一旦任务进入多轮执行，历史记录会很快变长。里面不只是用户的问题和模型回答，还会夹着很多工具调用结果、失败重试、中间判断和临时尝试。如果每次恢复都靠把整段历史重新读一遍，相当于每次都让模型重新消化一大段过程信息。对 code agent 来说，这个成本会越来越高，而且里面很多内容对接下来该怎么做其实没有直接帮助。

不稳在于：

历史记录的问题在于，它天然混着很多不同寿命的信息。有些是稳定约束，有些只是某一轮临时过程，有些甚至已经被后面的证据推翻了。如果恢复的时候还是靠模型从这些历史里自己重新提炼现在做到哪了，这件事本身就很脆弱。它可能恢复得慢，也可能恢复错，还可能重新把已经过期的信息当成当前状态继续往下推。

所以 Pico 里我做的判断是：

历史记录当然要保留，但它更适合做**复盘和解释**，不适合做**恢复主入口**。恢复的时候，系统优先信的是已经整理好的短状态，也就是 checkpoint，再结合工作记忆和新鲜度检查，判断哪些内容还能继续用。这样恢复就不是把旧过程再看一遍，而是站在已经整理好的状态上继续做。

这个设计其实和做系统很像。你不会拿一整份操作日志去当数据库主状态，你会用日志做追踪，但真正恢复服务时，还是要靠一份结构更清晰、职责更明确的状态数据。Pico 这里也是同样的思路。

4.5. workspace 漂移具体指什么，怎么检测

workspace 漂移，指的是旧状态对应的运行现场，和当前现场已经不一致了。

主要有两层。

第一层是文件层的不一致。

比如 checkpoint 里记着某几个关键文件的状态，但这些文件后来已经被改过了，那旧的摘要、旧的结论就不该继续直接信。这个时候系统要做的不是假装恢复成功，而是先把这些锚点重新校准。

第二层是运行现场的不一致。

比如当前工作目录变了、工作区整体指纹不一样了、模型或执行策略变了、只读模式和审批策略变了，类似这些变化都说明：虽然系统表面上还在同一个任务里，但它已经不是之前那个完全相同的运行现场了。这个时候如果还把旧状态当成完全可用，那就很危险。

所以 Pico 在恢复前不会只看有没有 checkpoint，而是会先做一致性检查。

简单说，就是先判断：

- 关键文件还对不对
- 当前工作区和之前是不是同一个现场
- 这份旧状态在今天这个现场里还能信多少

如果只是部分文件过期，那就进入部分可用的恢复分支；

如果工作区或运行条件已经对不上，那就进入现场不一致的分支。

4.6. 误信旧状态继续执行是什么意思，怎么检测到

误信旧状态就是本来有些旧状态已经不该继续信了，比如关键文件已经变了，或者工作区和原来的现场已经不一致了，或者恢复所依赖的结构本身已经不兼容了。这种情况下，系统本来应该先停下来重校准，或者明确走降级恢复。但如果它没有这么做，而是直接把旧状态当成完全没问题，继续沿着那条路径往下执行，这就叫误信旧状态。

这是我在pico恢复机制里最在意的一类错误。

这类错误它不像报错那样显眼。系统表面上看起来还在正常工作，但它其实已经站在一份旧状态上继续生成新动作了。对 code agent 来说，这比直接失败更麻烦。因为它可能会继续读错文件、改错位置、跑错命令，而且问题会越滚越大。

所以 Pico 在这块不只看恢复成功率，还会单独看一件事：

系统有没有把本来不该完全信的旧状态，当成完全可用状态继续往下跑。

检测思路也比较直接。

恢复时，系统会先判断这份旧状态到底属于哪种情况：

- 完全可用
- 部分过期
- 工作区不一致

- 结构不兼容
- 干脆没有可恢复状态

如果一份状态本来应该落到后面这些更保守的分支里，结果系统却把它当成“完全可用”继续执行了，那就说明它误信了旧状态。

5. 为什么要做工具治理，不做了危险的地方在哪

5.1. 为什么要做工具治理

模型本身最多只是给出一个动作意图，真正会对工作区、文件和环境产生影响的是工具执行层。你如果不在这一层加治理，前面 prompt 写得再漂亮，后面也很容易失控。

我自己理解，code agent 最危险的点主要有三类。

第一类是越界执行。

比如模型本来只该在当前仓库里读写文件，但它给了一个带 ../ 的路径，或者通过符号链接去碰工作区外的内容。如果工具层不拦，这就不是答错了，而是直接越界了。

第二类是高风险动作没有被约束。

像写文件、打补丁、跑 shell，这些动作一旦执行，影响就已经发生了。尤其是 shell，它不是一个简单工具，它等于把系统能力暴露给模型。如果没有审批策略和风险边界，模型只要一次判断失误，代价会很高。

第三类是系统自己不知道现在处在什么状态。

很多 agent 真正麻烦的地方不是这次命令报错了，而是命令报错了、工作区却已经变了，系统还当成普通失败继续往下跑。这种情况比直接失败更危险，因为它会在错误状态上继续生成新动作。

我做 Pico 的时候，工具治理不是围绕怎么让模型更方便地调工具去设计的，而是围绕怎么让工具执行变成一条有边界、有审计、能恢复的链路去设计的。

也就是说，模型负责提议动作，Harness 负责决定这个动作能不能执行、怎么执行、执行完以后系统该怎么看待它。

5.2. 工具怎么分风险等级，审批策略怎么设计

我在 Pico 里做工具分级的目的是为了解决这个问题：哪些动作系统可以默认放行，哪些动作必须更保守。

我大概是按有没有副作用和副作用有多重来分的。

第一类是低风险工具。

像读文件、列目录、查找内容，这些动作本身不会改工作区，也不会改运行环境，所以默认可以放得比较开。它们的主要问题不是风险高，而是可能被乱用，比如重复调用、无效查询，把系统拖进低效循环。

第二类是中高风险工具。

像写文件、打补丁，这类动作会直接改代码仓库。虽然影响范围比 shell 小，但它已经会改变工作区状态，所以我会把它们放进高风险动作管理里。

第三类其实是最高风险动作，也就是 shell。

因为 shell 不是一个单点能力，它本质上是把操作系统能力暴露出来。它可以读、写、删、跑命令，影响范围最大，所以我在设计上会把 shell 默认看成最需要审批和约束的一类。

审批策略这块，我做的不是所有高风险动作都弹窗，而是按运行模式去控制。

比如自动模式下，系统可以默认放行一部分受控动作；只读模式下，高风险写操作就应该直接被拦；而在需要人工确认的模式下，高风险动作要先过审批。

这样做的原因是，不同运行场景对风险容忍度不一样。你在本地做一次性实验，和在一个重要仓库里持续跑任务，边界肯定不同。

另外我觉得审批策略不能只和工具名绑定，还要和参数内容绑定。

同样是 shell，执行一个只读查询和执行一个会改文件的命令，风险完全不是一回事。

所以真正更稳的设计，是这个工具在这次参数下、在这次运行模式下危险。

工具分级的核心，是让系统知道：在当前这个运行模式和参数条件下，这个动作到底能不能安全放行。

5.3. 参数校验和工作区隔离怎么做，怎么防越界

本质上是在做一件事：把模型输出的动作意图，收紧成一个受控的、不会乱碰外部世界的执行请求。

参数校验我是这么做的：不把模型吐出来的工具调用直接当成可信输入。因为模型输出本质上还是文本，只是长得像结构化调用。Pico 里每个工具在执行前，都会先检查参数是不是完整、类型对不对、值是不是在合理范围内。

比如读文件必须给路径和范围，跑 shell 要有命令和超时时间，打补丁要能明确定位修改目标。这一步的意义是把很多模型看起来会调工具，但其实参数已经坏了的情况，拦在执行之前。

工作区隔离是为了保证 agent 再怎么工作，也只能在当前仓库边界内活动。

最典型的风险就是路径逃逸。比如模型给一个带上层路径的输入，或者借助符号链接去访问仓库外的内容。如果系统只是按字符串去拼路径，那很容易越界。

所以更稳的做法一定是：先把路径解析成真正的绝对路径，再去判断它是不是还落在当前工作区里面。

这样不管是 ../ 这种显式逃逸，还是符号链接这种隐式跳出去，最后都会被挡住。

我这里为什么这么强调“执行前校验，是因为很多问题如果放到执行后再看，就已经晚了。

比如路径已经越界了，文件已经被改了，再去复盘当然能发现，但损害已经发生了。

所以我在设计上更倾向于把约束尽量前移：

先校验、再执行；先确认动作还在工作区里，再让它真正落地。

5.4. 你写的 100% 通过率、100% 预算内完成率、100% verifier 通过率，到底怎么定义，证明了什么，不证明什么

第一，通过率。

这里说的不是模型看起来答对了这样的主观判断，而是固定回归任务在当前这套 Harness 下，最后满足了任务合同。也就是说，该生成的产物生成了，该改的地方改对了，最终状态也符合预期。所以这个通过率本质上是固定回归任务在当前运行时合同下有没有稳定跑通。

第二，预算内完成率。

这个指的是这些回归任务在设定的步数预算里能完成，没有无限循环，也没有因为反复试错把任务拖出预算。

这个指标我觉得很重要，因为它证明的不是模型强不强，而是这套运行链路会不会在固定任务上出错。

第三，verifier 通过率。

这个其实是最看重的一个口径。因为它意味着任务完成不是靠模型自己说 done，而是靠外部验证来判。

也就是说，系统最终是不是完成，不取决于模型自我感觉，而取决于结果和验证条件能不能对上。

这三个 100% 真正证明的，是固定回归任务里，Harness 合同是稳定的。

它说明工具执行链、预算控制、验证链路和结果判定在这套固定任务上没有回归。

如果面试官追问是不是任务太简单，可以这样说：

对，这层任务本来就不是拿来测模型上限的，它更像回归层，重点是检验 Harness 这套合同稳不稳。

真正去看上下文治理、记忆收益和恢复正确性，我会看另外几组对照实验，不会都靠这一层来证明。

6. 评测与审计闭环

评测与审计闭环：将评测拆成 harness regression、上下文治理、记忆收益和恢复正确性几层，分别验证运行时合同稳定性、模块收益和恢复边界，避免把模型能力、系统能力和运行观测混成一个总分；形成固定 benchmark、对照实验和运行工件聚合三类评测路径。

6.1. 具体做了什么

没有做一个很大的评测平台，而是把 任务定义、运行埋点和结果汇总这三层接起来了。

这样一次运行结束以后，我既能知道它过没过，也能知道它为什么过，或者为什么没过。

第一层是任务定义。

Pico 这边不是随便丢一句 prompt 去跑，而是先把任务收成一个比较固定的合同。这里面至少会定义几件事：这次允许用哪些工具、步数预算是多少、目标产物是什么、最后怎么验证。

所以后面完成这件事，不是模型自己说 done 就算了，而是要看目标产物和 verifier 能不能对上。

第二层是运行埋点。

这层我主要留了三类信息。

第一类是当前任务状态。

比如这次运行最后是什么状态，模型一共尝试了多少轮，真正执行了多少步工具，最后停下来的原因是什么。

这类信息回答的是：这次任务最后停在哪。

第二类是过程事件。

这里我会记录每一轮上下文是怎么组出来的、模型什么时候被调用、输出被解析成了什么、工具有没有真正执行、有没有被拦下来、执行后工作区有没有变化、有没有触发 checkpoint、最后是不是正常结束。

也就是说，这类埋点回答的是：这次运行中间到底发生了什么。

第三类是结果摘要。

比如最终答案、关键指标、这轮 prompt 的大小、恢复状态、有没有命中过记忆、有没有发生安全事件。

这类信息回答的是：这次结果最后长什么样，后面汇总时该抓哪些字段。

第三层是结果汇总。

Pico 这里我没有把所有结果压成一个总分，而是按问题拆开看。

如果我要看 Harness 合同稳不稳，我就看固定回归任务的通过率、预算内完成率和 verifier 通过率；

如果我要看上下文治理，我就看 prompt 平均长度、压缩率和当前请求有没有被裁坏；

如果我要看记忆收益，我就看重复读文件次数、follow-up 阶段的工具步数；

如果我要看恢复正确性，我就看恢复成功率、workspace 漂移识别率，以及有没有误信旧状态继续执行。

我在这里的设计理念是，不靠留了很多日志去做，而是知道该埋哪些点，这些点最后又能汇总成什么指标。

这样后面如果某次运行不对，我不是只能说模型这次不太稳定，而是可以往回分析：

到底是任务合同定义得有问题，还是中间工具调用被拦了，还是上下文裁剪出了问题，还是恢复时把旧状态信错了。

【更新中】91-面试题

项目介绍

一分钟项目介绍

【讲清楚 是什么，为什么，怎么做】

这是一个面向本地coding场景的轻量code agent。

我之所以会自己做一个 code agent，主要有两个很现实的原因。第一是可控性。平时我也会用 Claude Code 这类产品，但很多关键东西对用户来说其实是黑盒，比如 prompt 是怎么拼的、上下文是怎么组装的、工具在什么条件下会被调用，我很难真正判断它为什么这次做对了、下次又为什么会跑偏，某种程度上有点像抽卡。所以我想自己把这套东西搭起来，把这些关键环节都摊开看清楚。第二是隐私和部署要求。实验室有些项目本身有保密约束，不适合直接放到外部闭源服务里做，所以我也希望这套 agent 从工具到模型都能自己控制。我在项目里预留了接开源模型的能力，比如可以直接接实验室已经部署好的 Qwen，这样在需要保密的场景下也能真正用起来。

在设计上我没有把重点放在“接个模型、配点工具”这种 demo 上，而是重点去解决 coding agent 真正落地时最麻烦的几个问题，比如任务怎么稳定执行、长会话怎么不跑偏、高风险操作怎么控住、出了问题之后怎么恢复和复盘。围绕这些点，我做了显式任务状态、分层记忆和预算感知的上下文组装、工具审批机制，以及 session 和 run 两层持久化，最后还配了一套固定任务集和离线 evaluator 去做可复现评测。

从结果看，pico。上下文治理在 12 组配置里把平均 prompt 长度从 6964 压到 5418；记忆实验里，重复读文件次数从 8 次降到 3 次，任务正确率从 66.7% 提到 100%；安全场景里也能稳定拦住路径逃逸、无效参数和重复调用。项目虽然不大，但执行、治理、恢复和评测这条关键链路已经放进去了。

核心问题

1. 为什么把 Pico 定义成 harness？你对harness的理解？

先从对harness的理解说起吧。

agent 负责决定下一步做什么

harness 负责规定agent在什么边界内做、怎么记录、怎么验证、什么时候才算真的完成。

对于agent工作的时候，harness其实就是围绕模型/agent，去补五类东西：

1. 它该怎么工作
2. 它现在做到哪了
3. 什么算完成
4. 它不能乱做到哪里去
5. 一次会话怎么开始、结束、交接

一个好的harness，最核心的标准是，用同样的模型，好的 Harness 能在同样的任务上表现更好，并且可以不经人为干预地运行得更久、更稳定

所以在设计pico之前，我看了anthropic的harness文章和网上相关的最佳实践，还包括OpenHarness这个项目，在设计的过程中，我把更多的时间花在了思考代码仓库里怎么不失控、运行时主循环怎么推进、prompt 怎么按结构拼起来，工具怎么显式注册和校验等问题，目的就是想让 agent 放进受控环境里跑的系统。

最后pico的设计上，它有 runtime，负责把用户一句话推进成一轮轮可控执行。

它有工具治理，不是看到工具调用就直接跑，而是先检查工具是否存在、参数是否合法、是不是重复调用、当前风险策略允不允许。

它有 memory 和 context manager，解决当前 prompt 里到底什么该进、什么不该进的问题。

它还有 run artifacts 和 benchmark，把过程和结果都留了下来，后面既能回放，也能评测，还能做不同机制的对照。

所以如果把这几层合起来看，我想解决的其实不是模型会不会写代码，而是能不能把一个 code agent 做成一个有状态、有边界、有验证、也有回归能力的工程系统。这也是为什么我最后会把 Pico 定义成 harness。

2. 讲一下从输入命令到agent执行完毕，整个pico到运行链路是怎么样

`pico` 从输入命令到执行结束，可以分成五层。

第一层是入口层。

用户在终端输入命令以后，系统先做启动参数解析，比如工作目录、模型后端、执行预算、审批策略这些。

这个阶段的目标是把用户的一条命令转换成一个完整的运行上下文。

这里会确定三件事：当前在哪个仓库里工作、这次运行用哪个模型、这次运行是接着旧 session 继续还是新开一次任务。

第二层是上下文构建层。

agent 真正开始推理前，不会直接拿用户原始请求去问模型，而是先把当前系统能提供的上下文整理成一个结构化输入。

这里我把上下文分成几种不同类型的数据：

- 稳定上下文，比如工具能力、系统规则、仓库概要
- 短期工作记忆，也就是当前任务相关的最近文件、最近摘要、临时笔记
- 相关长期记忆，也就是过去沉淀下来的稳定事实
- 会话历史，也就是前面几轮发生过什么
- 当前用户请求

这个设计的核心思想是，不同寿命的数据不能混成一团。稳定部分应该尽量可复用，短期部分应该围绕当前任务滚动更新，长期部分应该只在相关时被召回。

第三层是 agent 控制循环。（runtime）

这部分是整个系统的核心。

它不是一次性把 prompt 发给模型然后等答案，而是一个持续推进的闭环：

1. 构建当前轮上下文
2. 调模型做决策
3. 判断模型输出的是最终答案还是动作请求
4. 如果是动作请求，就去执行工具
5. 把执行结果写回系统状态
6. 进入下一轮

第四层是执行与状态更新层。

如果模型决定调用工具，系统不会直接照做，而是先过一层执行网关。

这层主要负责几件事：

- 校验动作是否合法
- 判断是不是重复动作
- 判断是不是高风险动作，需要不要审批
- 真正执行动作
- 把结果转换成系统下一轮可以消费的状态

这里最重要的设计点是，工具结果不会原样无限堆积，而是会被拆成两类：

- 一类进入会话历史，保证可回放

- 一类进入短期工作记忆，保证下一轮还能高效利用

比如读文件这种行为，更适合沉淀成最近读过哪些文件和这些文件的短摘要，而不是把整个文件内容一直带着走。

第五层是持久化与审计层。

当一轮任务结束以后，系统会把这次运行留下两种不同性质的数据。

第一种是 **可恢复状态**。

也就是 session 级别的数据，它服务的是“下次还能接着干活”。

第二种是 **运行工件**。

也就是 task state、trace、report 这类单次运行记录，它服务的是“解释这次到底发生了什么”。

它让 agent 系统不只是能跑，还能复盘、调试、评估。

最后我总结一下

这五层设计分别解决了不同的问题：

- 入口层解决“怎么把命令变成运行现场”
- 上下文构建层解决“怎么把不同寿命的数据组织成模型可消费的输入”
- 控制循环解决“怎么让 agent 持续推进，而不是一轮就结束”
- 执行与状态更新层解决“怎么安全调用工具，并且把结果沉淀成下一轮还能用的状态”
- 持久化与审计层解决“怎么让系统可恢复、可解释、可评估”

PS：对于面试的时候长回答，最好采用总分总的形式，最后收个尾，让面试官有印象，方便提问，你知道的，大家的注意力在AI时代已经慢慢消散了

3. Pico里面的上下文是怎么管理的

Pico上下文管理的核心思想是，把上下文当成一个有预算、有优先级的运行时状态来管理，这块的设计主要解决的问题是本地 coding agent 在多步任务里，**怎么让模型每一轮都看到一份够用、稳定、不过载的上下文**

第一层是稳定上下文，也就是 prefix。这部分不是会话历史，而是 agent 每一轮都要用到的固定背景。里面会有它是谁、工具怎么调用、当前仓库的一些稳定事实，比如 Git 状态、最近提交、白名单文档片段。这么做的原因很直接，本地 coding agent 和聊天不一样，它不是只回答一句话，它要在一个仓库里连续做判断。如果这些基础背景每一轮都跟着 history 一起拼进去，一方面 prompt 会越来越乱，另一方面很多其实没变的信息会被重复传输。Pico 把这部分单独拎出来，放在一个固定层里，目的就是让基础上下文稳定下来。这样后面 history 变了，这层也不会跟着抖，而且还能做 fingerprint 和缓存复用。

第二层是 working memory，也就是轻量工作记忆。这里我设计得比较克制。只保留几类下一轮大概率还会继续用到的状态：当前任务摘要、最近接触过的文件、这些文件的短摘要，还有少量 episodic

notes。核心思想是也尽量减少Agent的重复劳动。比如我刚读过哪个文件，这个文件前几行大概讲了什么，如果这些信息如果不留下来，下一轮就很容易再读一遍、再找一遍。working memory 的作用就是把这些高频、短期有用的信息留下来，把中间那些冗余过程留在 history 里，不让它们常驻。

第三层是 prompt 组装，也就是 ContextManager 这一层。真正发给模型的 prompt，不是把各种信息随手拼接一下，而是明确拆成五段：prefix、memory、relevant memory、history、current request。这个顺序本身就体现了设计判断。先给稳定规则和仓库背景，再给当前任务态，再给和这一轮请求相关的记忆，再给历史，最后把当前用户请求原样放在最末尾。这个顺序的安排是关注模型这一轮最先该看到什么，最后又该以什么收束。Pico 的做法很清楚，当前请求一定要压在最后，而且要完整保留，因为它是这轮决策的直接目标。

这五段都有预算。上下文超了以后，Pico 不是粗暴截断，而是按优先级收缩，先收 relevant memory，再收 history，然后才动 memory 和 prefix，current request 永远不裁剪。这里其实就是在处理一个很实际的问题：上下文不够的时候，哪些信息可以缩，哪些信息不能丢。Pico 的做法很明确，补充信息可以先收，当前这一轮要解决的问题必须完整保留。

最后我总结一下，Pico 的上下文管理就是把稳定背景、短期工作记忆、相关记忆和过程历史分开，再用预算和优先级把它们组织好，让模型每一轮看到的都不是最多的信息，而是当前最该看到的信息。

4. Pico的记忆设计是怎么样的

Pico 的记忆设计，核心思想是只记真正有用的东西，并且把记忆分层。它要解决的其实是两件事，一件是怎么把当前任务真正需要的信息留下来，让这件事能接着做；另一件是怎么把长期有效的项目知识沉淀下来，后面还能复用。同时还得控制一个问题，就是这些内容不能把上下文挤爆。

第一层是当前工作记忆，解决的是这件事怎么继续往下做。这里面主要放当前任务摘要、最近看过的文件、文件的短摘要，还有少量跨轮笔记。也就是说，它留下的是当前任务后面大概率还会继续用到的信息，而不是把整个过程原样背下来。在实现上，这几类内容都是有限量的，不会无限堆下去。

第二层是长期记忆，解决的是这个项目里哪些稳定事实值得留下来。比如用户偏好、项目约定、关键决策、依赖要求、反复出现的坑，这些会单独落到 .pico/memory 下面。Pico 不是什么都存。像临时尝试、这一轮失败、当前报错这种短期信息，它不会轻易升成长期记忆。也就是说，我们明确把过程信息和长期知识分开了，不让长期记忆变成另一份冗长日志。

再往下一层看，Pico 也不是把这些记忆一股脑全喂给模型，而是会先挑和当前问题最相关的那几条，再结合上下文预算去组装。重点不是让模型记得越多越好，而是让它每一轮都尽量站在真实仓库状态上做判断。比如读完文件之后，会提炼一个短摘要；如果后面这个文件被改了，旧摘要还会失效，避免模型继续拿过期信息推理。很多 agent 会记很多内容，但不太处理这些内容是不是已经过期。Pico 在这块是有意识做了约束的。

我总结一下，Pico 的记忆设计，不是在追求记忆越大越强，而是在追求记忆够用、相关，而且不过期。我的思路其实也很明确，就是先用一个比较轻的设计，把该覆盖的需求先覆盖住。

5. 为什么不直接用现成 agent framework

第一，依赖更少，行为更可控，出了问题更容易直接定位到 runtime 和 tool 层。

第二，这个项目的编排逻辑并不复杂，还没复杂到必须引入图编排框架。

第三，我希望 prompt、工具协议、审批策略、trace 格式都能完全按自己的方式定义，避免为了适配框架抽象而增加额外复杂度。等后面任务流变复杂，比如多节点状态机、可视化编排、长期 workflow 复用明显增多，再引入 LangGraph 这类框架会更合适。

6. 指标问题

7. 后续改进问题

7.1. 这个项目的harness能怎么进行改进

第一步，我想针对中间过程的 harness 做改进。现在 pico 的评测重点还是最后文件有没有改对、校验脚本有没有过、步数有没有超，这当然重要，因为它保证结果是干净的、能复现的。但真实场景里，Agent 最容易出问题的地方，往往不是最后那一下，而是中间怎么走的。它可能来回读一堆没用的文件，可能重复调同一个工具，可能明明权限不够还一直乱试，甚至可能这次是运气好碰巧改对了。这样的结果，就算最后过了，也不算一个真正靠谱的系统。所以我觉得下一步可以把评测目标从只看最终结果往前推，开始把中间过程也纳入评测，比如把无效重复调用、失败后的重试策略、工具使用质量、路径选择是否合理这些都沉淀成可量化指标。这样以后评测的就不只是有没有做成，还包括是不是用一条靠谱的过程做成了。

第二步，我会补这套系统的边界管理，重点解决 agent 什么能做、什么不能做的问题。现在 pico 已经有一些安全意识了，比如区分高风险工具、有只读模式、对环境变量也做了筛选。但如果从一个更完整的 Agent harness 来看，这一层还可以再独立一点。我的想法是，把这部分单独做成一套规则系统，明确管理几件事：哪些工具默认允许，哪些必须确认；哪些路径不能碰；哪些命令绝对不能跑；不同运行模式下允许的动作边界是什么。这样一来，安全就不只是零散地散在几个判断里，而是会变成一套清楚、可配置、也方便扩展的规则层。后面不管是切换运行模式、做权限实验，还是排查越界行为，都会更清楚。

第三步，我会针对可观察性去提高 tracing 能力。现在 pico 对一次运行，其实已经分成了三层信息来保存：一层是当前任务状态，回答的是现在停在哪；一层是运行过程里的事件流，回答的是中间发生了什么

么；还有一层是最后的总结报告，回答的是这次运行最终结果怎么样。这个分层思路本身是对的，说明它已经不只是简单打日志了。

但现在的问题是，这些信息在工程实现上还是有点散。有些埋点在事件流里，有些落在最终报告里，还有一些要靠后面的统计逻辑再拼出来。这样带来的问题就是，这套系统现在是能观察，但还没有到方便分析的程度。比如我想看为什么这次比上次更慢，为什么某一轮上下文裁剪之后效果变差了，或者为什么某一类任务里工具失败变多了，很多时候还得人工把几份信息对起来看。**所以我觉得下一步可以把关键节点进一步标准化，尽量都沉淀成结构化事件，同时把字段定义统一下来。**这样后面不管是查性能退化，还是研究某种策略为什么有效，成本都会低很多。好的 harness 不只是能跑起来，还应该能把自己为什么这样跑讲清楚。

我总结一下，pico 下一步我想做的，是把这套系统从能把任务跑完往能把过程管住、把边界守住、把原因讲清楚再推进一步。也就是结果上不只看做没做成，过程上要知道它是怎么做成的，边界上要明确它什么能做、什么不能做，分析上要能解释它为什么这次跑得好、为什么下次又退化了。

8. 主流agent框架改进的方向

9. code agent生成代码有什么挑战

code agent 的难点，不只是生成代码本身，而是在一个持续变化的真实系统里，稳定地做出可落地的修改。所以它本质上不只是一个文本生成问题，更像一个带状态的执行系统问题。

因为代码不是孤立文本，它永远挂在一个正在运行的工程系统上。一个函数怎么改，取决于当前仓库状态、已有实现约束、依赖版本、测试假设、文件之间的耦合关系，也取决于前面几步工具调用拿回来的信息是不是准确。

所以第一个挑战，是怎么把系统状态表示清楚。agent 到底看到了什么，哪些是当前任务最相关的信息，哪些是长期约束，哪些只是这一轮临时拿到的观察，这件事比生成本身更重要。Pico 在这块做得比较克制，它没有把整段对话历史都塞回去，而是把工作区基线、当前工作记忆、长期记忆和历史记录拆开，再按预算组装给模型。重点不是让模型记得越多越好，而是尽量让它站在真实仓库状态上做判断，而不是站在自己的猜测上做判断。

第二个挑战，是修改动作要尽量准确。很多代码 agent 出错，不是不会写，而是改不准，改错了位置，或者一次改多了不该改的地方。Pico 里 patch_file 要求旧文本必须唯一命中，本质上就是在解决这个问题。因为一旦修改目标不明确，后面的验证都会变得很脆弱。换句话说，它是在把一个比较模糊的生成动作，尽量收紧成一个确定的修改动作，不让不确定性在执行阶段继续放大。

第三个挑战，是多步执行过程本身要稳。代码 agent 不是一次性生成，而是一个循环过程，先读，再判断，再执行，然后根据结果继续往下走。真正容易失控的地方，往往不是模型不会写，而是它会重复调用同一个工具，拿过期信息继续推理，或者还没验证就提前说自己做完了。Pico 在运行时里会记录尝试

次数、工具调用步数和停止原因，也会拦住重复的相同调用。这里体现的是一个很重要的系统设计思路：agent 不只是能力问题，运行过程本身也需要治理。

最后一个挑战，是验证闭环。模型说自己做完了，不代表系统层面真的完成了。真正的完成，至少要回答几件事：产物是不是在，修改是不是落到了预期文件，验证脚本过没过，步数有没有超，停下来的原因是不是正常。Pico 不把完成交给模型自己定义，而是交给验证、基准任务和运行时状态一起判断。

最后我总结一下，code agent工程化上困难的地方不在把代码写出来，而在把代码生成放进一个有状态、有边界、能验证的执行系统里。Pico 现在做的，就是沿着这条线，把状态表示、修改准确性、运行过程治理和验证闭环一点点补起来。

10. 如何保证AI代码生成的质量

92-claude code相关

你有没有看过claude code的源码？你从他的设计中有学到什么内容吗？pico里面有相关的设计吗？

可以从以下几点给面试官分享：

第一点，agent loop 本质上应该被当成状态机来设计，不是普通 REPL。Claude Code 会显式管理继续原因、终止原因、恢复路径、上下文压缩和失败降级。这个思路对我影响很大，因为它把问题从模型回答质量拉回到了运行时可靠性。pico 现在也在沿着这个方向做，我们有 ask() 这一层主循环，有 TaskState，有 checkpoint 和 resume 状态，也会把一次运行的 trace、report、task_state 落盘。也就是说，我们不是只保留聊天记录，而是在保留一个可恢复、可复盘的运行现场。

第二点，工具系统一定要 fail-closed，而不是靠提示词让模型自觉。Claude Code 的工具不是简单的 schema 加 description，而是有统一契约、严格校验、权限链、并发安全判定和结果预算。它背后的原则是判断不出来的时候，默认按危险处理，而不是按安全处理。这个思路我在 pico 里也很认同。我们现在的工具是显式白名单注册，参数会严格校验，patch_file 要求精确命中一次，风险工具要经过审批，delegate 出去的子 agent 默认是只读的。这些设计看起来没那么炫，但我觉得这才是 agent 工程能力真正站得住的地方，因为它决定了系统在边界情况里会怎么表现。

第三点，上下文管理不是拼 prompt，而是做预算。Claude Code 很强的一点，是它把上下文当成受限资源来调度。什么该稳定、什么该后移、什么该压缩、什么不能碰，它都有很明确意识。这让我更明确了一件事：prompt engineering 到后面其实会变成 context engineering。pico 现在已经吸收了这条思路的一部分。我们的 ContextManager 不是把所有历史糊成一段字符串，而是拆成 prefix、memory、relevant_memory、history、current_request 这几段，每段有预算和收缩顺序，而且永远不裁当前请求。我们还有 prefix_hash、workspace_fingerprint、tool_signature 这些东西，目的也是让稳定前缀尽量稳定，动态部分尽量被控制住。

第四点，是我特别认同的一条，复杂任务要先对齐，再执行。Claude Code 的 plan mode 很有代表性，它在真正动手之前，会先把权限收成只读，先探索、先写计划、先拿批准，再进入执行。我觉得这件事非常重要，因为 coding agent 最大的风险，很多时候不是写坏代码，而是认真写了不该写的东西。这个点 pico 还没有完整实现，我不会夸大，但我们已经有前置积木，比如审批模式、只读 delegate、checkpoint 里对当前目标和下一步的显式记录。对我来说，这也是学习优秀项目时很关键的一点，不是去抄功能，而是看它到底在解决哪个根问题。Claude Code 解决的是意图对齐，我们接下来如果往前走，我会优先把真正的 plan mode 补出来，而不是先堆更多工具。

0417字节Agent开发一面

面经及黄同学准备的参考答案

1. 为什么要自己搞一个code agent? 这个过程中你觉得哪部分是比较有挑战的

参考回答:

为什么要做code agent：一个参考的包装环境，可以根据自己的背景和项目来

第一是可控性。平时我也会用 Claude Code 这类产品，但很多关键东西对用户来说其实是黑盒，比如 prompt 是怎么拼的、上下文是怎么组装的、工具在什么条件下会被调用，我很难真正判断它为什么这次做对了、下次又为什么会跑偏，某种程度上有点像抽卡。所以我想自己把这套东西搭起来，把这些关键环节都摊开看清楚。

第二是隐私和部署要求。实验室有些项目本身有保密约束，不适合直接放到外部闭源服务里做，所以我也希望这套 agent 从工具到模型都能自己控制。我在项目里预留了接开源模型的能力，比如可以直接接实验室已经部署好的 Qwen，这样在需要保密的场景下也能真正用起来。

哪部分比较有挑战

1. 上下文工程：context 是有限资源，怎么解决长仓库任务里逐渐偏离最初 spec，比如忘记约束、误用旧信息、重复操作、或者把局部修复做成全局破坏等上下文还没超窗时，模型对长历史的利用质量已经开始下降等问题。
2. 工具设计：agent 的效果取决于给它什么工具，而且哪怕只是 tool description 和 schema 的小改动，也会显著影响完成率，工具的粒度、装载方式的设计都是需要考虑的（可以看下这篇：https://www.anthropic.com/engineering/code-execution-with-mcp?utm_source=chatgpt.com）
3. harness 设计：怎么让 agent 跑起来之后，不至于失控、卡死、跑偏、忘事、无法追责。随着任务跨越多个 context window，光靠模型单轮推理已经不够，必须有一个外层 harness 去管理进度、状态、记忆、验证和恢复。（可以看下这篇：https://www.anthropic.com/engineering/harness-design-long-running-apps?utm_source=chatgpt.com）
4. 怎么做评价体系：只看任务是否pass/fail肯定是不够的
5. 安全、权限和可控性

2. 那你自己用的code agent，用过哪些，哪个最好用？有没有用过字节的trae，感觉怎么样？你的目前ai coding的工作流是怎么样？

参考回答：

可选：claude code/codex/cursor/trae/kimi code/opencode 等等

个人建议一定要回答到claude code，如果你做code agent连claude code都没用过相当于你写java没用过spring。

我用过claude code/codex/cursor/trae等，自己也订阅过claude code max和gpt pro，这几者我感觉claude code最好用，它的产品形态和思维能力都很强，这跟他的code agent设计以及模型的强大有关（在这个时候你就可以展示你的技术敏感度了），日常写代码，如果是比较复杂的问题，我会优先让claude分析。

codex的话，我觉得它更像一个工程师，比较能体现区别的就是codex和claude code的plan模式，前者只会吭哧吭哧的干活，后者会给我提供更多insights。所以我一般都是让claude code写prd/tech design doc，然后让codex编写代码和code review。

其他的话，我觉得比较有特点的是kimi code，它的文字能力我觉得是比较强的，如果是一些中文文档的撰写，我会优先用它。对于trae，我有尝试过在里面使用kimi 2.5，简单任务下还是不错的，但如果在复杂的编码任务下，它的表现还是不如claude code/codex，我觉得比较有影响的可能是模型的不同，claude我一般用opus4.6，codex用gpt5.4 xhigh。不过我也有在持续关注trae，最近很火的sbti人格测试，我也尝试过用trae solo实现，效果还是挺好的。

从我实际使用的角度来看，了解每个code agent的特点，清楚他的边界，再组合使用，会让我的任务解决变得高效。我的整个code agent介入下的工作流目前是：

claude code opus4.6写PRD -> executing plan-> codex编码/测试 -> 做code review -> 更新./AGENTS.md的bad case -> 再用Claude Code review -> codex进度管理，完成git commit 和 push ->

如果要举个例子来概括我用这些 code agent 的感受，我觉得自己更像一个工程团队的 TL。好的 TL 不靠微管理，也不会因为把任务交出去就总担心会搞砸。关键是他清楚每个人的能力边界，知道怎么把任务拆到合适的粒度，也知道怎么验收结果。这样既省力，也能把整个团队的力量用起来。

用 code agent 其实也是一样。你得了解每个 agent、每个模型擅长什么，不擅长什么，再把合适的任务分给合适的对象。分工对了，token 用得更省，事情也通常能办得更漂亮

怎么回答这类开放性问题？

1. **做垂类产品，应聘不同公司，一定要了解对方在这个垂类上的产品**，比如你做code agent，去应聘字节，肯定要看下trae。应聘腾讯，肯定要看下workbuddy.....
2. **自己要有一定的个人体验**，个人体验是区分是否真的使用过某类技术栈的最佳判断，面试官会根据你自己个人体验中延伸的个人思考来判读你是否有学习能力/独立思考能力
3. **日常学习工作时候要多沉淀sop，才能在复杂场景问题下快速响应**

3. 有没有观察过同一个模型在不同code agent的表现？你觉得为什么会有不同的表现？

参考答案：

这是个很有意思的问题，面试官用这种你日常使用场景里的小细节来判断你对code agent这个领域的敏感度。

我觉得同一个基础模型放在不同 code agent 里，表现差异往往非常明显，而且很多时候差异并不主要来自模型本身，而是来自 agent 框架对任务的拆解、上下文组织、工具调用、反馈闭环和执行约束的设计。

我一般会从几个层面去看这个差异。

第一，任务编排不一样。比如有的 agent 会先做任务理解、列计划、定位相关文件、再逐步修改；有的 agent 则更偏一次性生成。对于多文件修改、需要理解仓库结构的问题，

第二，上下文构造不一样。虽然底层模型一样，但不同 agent 给模型喂进去的信息不同，比如 system prompt、仓库摘要、检索到的代码片段、历史操作轨迹、错误日志、测试结果等。很多表现差异，本质上是输入分布不同，而不是模型能力突然变了。（这里可以举一些细节的例子，比如 claude code的CLAUDE.md和codex的AGENTS.md的差异）

第三，工具链和反馈机制不一样。code agent的工具设计很重要，搜索、grep、代码索引、运行测试、lint、执行 shell、读写文件等工具，各家是有差异的。一个 agent 如果能在改完代码后自动跑测试，并把失败栈回灌给模型继续修，就会比只生成代码不验证的 agent 明显更强。也就是说，agent 的闭环能力会放大或限制模型能力。

第四，决策策略不一样。比如一步改大还是小步快跑、遇到不确定时先查再改还是直接改、是否允许多轮反思、失败后是否回滚、是否做多候选采样和重排。这些策略都会影响最终结果的稳定性和成功率。

第五，约束和安全边界不一样。有些 agent 对改动范围、命令执行、依赖安装、网络访问限制更严格，这会让她在某些任务上显得保守，但也更可控。另一些 agent 更激进，短期通过率可能更高，但也更容易引入副作用。（codex的风控策略明显比claude code严格，claude code让他逆向一个软件他不会拒绝，但codex 99%的情况下会拒绝你）

所以我会认为：同一个模型在不同 code agent 上表现不同，是一个很正常的系统现象。**模型决定了上限，但 agent 设计决定了它能发挥出多少。**

如果面试官问你会怎么优化一个 code agent，可以参考这么个丝路：

第一，提高代码检索和上下文压缩质量，确保模型看到对的代码；

第二，增强执行后反馈闭环，把测试、traceback、静态检查结果结构化回灌；

第三，控制单次改动粒度，避免大范围误改，并给 agent 明确的停止条件和回滚策略。

4. Claude code源码泄漏了你有去看吗？你自己这个项目有没有参考它的设计

参考回答：

code agent在现在这个时间点完全避开不了claude code，特别是源码泄漏之后，大家需要好好准备。我之前也给大家准备了一些内容，可以自行参考

可以从以下几点给面试官分享：

第一点，agent loop 本质上应该被当成状态机来设计，不是普通 REPL。Claude Code 会显式管理继续原因、终止原因、恢复路径、上下文压缩和失败降级。这个思路对我影响很大，因为它把问题从模型回答质量拉回到了运行时可靠性。pico 现在也在沿着这个方向做，我们有 ask() 这一层主循环，有 TaskState，有 checkpoint 和 resume 状态，也会把一次运行的 trace、report、task_state 落盘。也就是说，我们不是只保留聊天记录，而是在保留一个可恢复、可复盘的运行现场。

第二点，工具系统一定要 fail-closed，而不是靠提示词让模型自觉。Claude Code 的工具不是简单的 schema 加 description，而是有统一契约、严格校验、权限链、并发安全判定和结果预算。它背后的原则是判断不出来的时候，默认按危险处理，而不是按安全处理。这个思路我在 pico 里也很认同。我们现在的工具是显式白名单注册，参数会严格校验，patch_file 要求精确命中一次，风险工具要经过审批，delegate 出去的子 agent 默认是只读的。这些设计看起来没那么炫，但我觉得这才是 agent 工程能力真正站得住的地方，因为它决定了系统在边界情况里会怎么表现。

第三点，上下文管理不是拼 prompt，而是做预算。Claude Code 很强的一点，是它把上下文当成受限资源来调度。什么该稳定、什么该后移、什么该压缩、什么不能碰，它都有很明确的意识。这让我更明确了一件事：prompt engineering 到后面其实会变成 context engineering。pico 现在已经吸收了这条思路的一部分。我们的 ContextManager 不是把所有历史糊成一段字符串，而是拆成 prefix、memory、relevant_memory、history、current_request 这几段，每段有预算和收缩顺序，而且永远不裁当前请求。我们还有 prefix_hash、workspace_fingerprint、tool_signature 这些东西，目的也是让稳定前缀尽量稳定，动态部分尽量被控制住。

第四点，是我特别认同的一条，复杂任务要先对齐，再执行。Claude Code 的 plan mode 很有代表性，它在真正动手之前，会先把权限收成只读，先探索、先写计划、先拿批准，再进入执行。我觉得这件事非常重要，因为 coding agent 最大的风险，很多时候不是写坏代码，而是认真写了不该写的东西。这个点 pico 还没有完整实现，我不会夸大，但我们已经有前置积木，比如审批模式、只读 delegate、checkpoint 里对当前目标和下一步的显式记录。对我来说，这也是学习优秀项目时很关键的一点，不是去抄功能，而是看它到底在解决哪个根问题。Claude Code 解决的是意图对齐，我们接下来如果往前走，我会优先把真正的 plan mode 补出来，而不是先堆更多工具。

5. Harness在你这个项目里面怎么体现的？你是怎么理解harness的？code agent的harness有哪些挑战？

参考答案：之前有写在面试逐字稿里了：

先从对harness的理解说起吧。

agent 负责决定下一步做什么

harness 负责规定agent在什么边界内做、怎么记录、怎么验证、什么时候才算真的完成。

对于agent工作的时候，harness其实就是围绕模型/agent，去补五类东西：

1. 它该怎么工作
2. 它现在做到哪了
3. 什么算完成
4. 它不能乱做到哪里去
5. 一次会话怎么开始、结束、交接

一个好的harness，最核心的标准是，用同样的模型，好的 Harness 能在同样的任务上表现更好，并且可以不经人为干预地运行得更久、更稳定

所以在设计pico之前，我看了anthropic的harness文章和网上相关的最佳实践，还包括OpenHarness这个项目，在设计的过程中，我把更多的时间花在了思考代码仓库里怎么不失控、运行时主循环怎么推进、prompt 怎么按结构拼起来，工具怎么显式注册和校验等问题，目的就是想让 agent 放进受控环境里跑的系统。

最后pico的设计上，它有 runtime，负责把用户一句话推进成一轮轮可控执行。

它有工具治理，不是看到工具调用就直接跑，而是先检查工具是否存在、参数是否合法、是不是重复调用、当前风险策略允不允许。

它有 memory 和 context manager，解决当前 prompt 里到底什么该进、什么不该进的问题。

它还有 run artifacts 和 benchmark，把过程和结果都留了下来，后面既能回放，也能评测，还能做不同机制的对照。

所以如果把这几层合起来看，我想解决的其实不是模型会不会写代码，而是能不能把一个 code agent 做成一个有状态、有边界、有验证、也有回归能力的工程系统。这也是为什么我最后会把 Pico 定义成 harness。

harness的挑战：

也是一个有意思的问题，harness这么新的概念，考察挑战，对校招生还是很有难度的

harness 设计的挑战，面试里可以讲成六个具体问题。

code agent 的核心难点，在把模型放进一个可控、可验证、可追责的执行系统里，harness 解决的就是这件事。

第一，任务生命周期怎么管。一个真实任务不是简单的 prompt 发出去就结束了，harness 必须定义清楚：什么时候开始执行，什么时候暂停等待外部条件，什么时候自动重试，什么时候判定失败并交还给人。这里的难点不在过程控制。**模型擅长局部推理，但不擅长长期、稳定地管理执行状态**，所以编排上必须在 harness 这一层完成。

第二，状态和记忆怎么管。长任务一定会遇到 context rot，也一定会出现上下文膨胀。harness 要决定哪些信息进入当前上下文，哪些压缩成 summary，哪些写入持久 memory，哪些应该直接丢弃。否则 agent 很容易做着做着忘掉最初约束，或者重复已经做过的步骤。

第三，验证闭环怎么做。harness 需要决定什么时候跑 lint、什么时候跑 typecheck、什么时候跑 unit test，什么时候需要 integration test，失败以后是继续自动修，还是停下来请求人工判断。没有 verifier，agent 只是会写代码；有 verifier，它才接近能交付结果。

第四，执行隔离和权限控制怎么做。只要 agent 能写文件、跑命令、访问网络，就一定会带来安全风险，包括误操作、越权操作和 prompt injection。harness 必须把自动化建立在 sandbox、approval、network control 和审计机制之上。不能先给足权限，再指望模型自律。应该先收紧边界，再允许它在边界内自主执行。

第五，并发和任务拆分怎么做。一个任务到底应该由单 agent 完成，还是拆成多个 subagent 并行处理。并行确实能提升速度，也能让不同 agent 分工处理搜索、修改、验证这些子问题，但代价是编排的成本更高，结果更难合并，一致性更难保证。harness 必须清楚定义拆分边界、上下文继承方式，以及最终怎么汇总结果。

第六，可观测性和可追责怎么做。真实系统里，你必须知道 agent 为什么改了这个文件，执行过哪些命令，在哪一步失败，是否触发过人工审批，这一版 patch 比上一版到底好在哪里。没有 trace、log、diff、grader 和 audit trail，你就没法调优 harness，也没法把失败归因到模型、工具、提示词还是编排逻辑。可观测性不是补充项，而是 harness 能不能持续演进的前提。

6. 讲一下你项目里面用户在终端发一个指令到agent完成所有任务的整个链路

逐字面试话术稿里有，我复制过来给大家参考一下，这个问题一定要有自己的理解。

pico 从输入命令到执行结束，可以分成五层。

第一层是入口层。

用户在终端输入命令以后，系统先做**启动参数解析**，比如工作目录、模型后端、执行预算、审批策略这些。

这个阶段的目标是把用户的一条命令转换成一个完整的运行上下文。

这里会确定三件事：当前在哪个仓库里工作、这次运行用哪个模型、这次运行是接着旧 session 继续还是新开一次任务。

第二层是上下文构建层。

agent 真正开始推理前，不会直接拿用户原始请求去问模型，而是先把当前系统能提供的上下文整理成一个结构化输入。

这里我把上下文分成几种不同类型的数据：

- 稳定上下文，比如工具能力、系统规则、仓库概要
- 短期工作记忆，也就是当前任务相关的最近文件、最近摘要、临时笔记
- 相关长期记忆，也就是过去沉淀下来的稳定事实
- 会话历史，也就是前面几轮发生过什么
- 当前用户请求

这个设计的核心思想是，不同寿命的数据不能混成一团。稳定部分应该尽量可复用，短期部分应该围绕当前任务滚动更新，长期部分应该只在相关时被召回。

第三层是 agent 控制循环。（runtime）

这部分是整个系统的核心。

它不是一次性把 prompt 发给模型然后等答案，而是一个持续推进的闭环：

1. 构建当前轮上下文
2. 调模型做决策
3. 判断模型输出的是最终答案还是动作请求
4. 如果是动作请求，就去执行工具
5. 把执行结果写回系统状态
6. 进入下一轮

第四层是执行与状态更新层。

如果模型决定调用工具，系统不会直接照做，而是先过**一层执行网关**。

这层主要负责几件事：

- 校验动作是否合法

- 判断是不是重复动作
- 判断是不是高风险动作，需要不要审批
- 真正执行动作
- 把结果转换成系统下一轮可以消费的状态

这里最重要的设计点是，工具结果不会原样无限堆积，而是会被拆成两类：

- 一类进入会话历史，保证可回放
- 一类进入短期工作记忆，保证下一轮还能高效利用

比如读文件这种行为，更适合沉淀成最近读过哪些文件和这些文件的短摘要，而不是把整个文件内容一直带着走。

第五层是持久化与审计层。

当一轮任务结束以后，系统会把这次运行留下两种不同性质的数据。

第一种是 **可恢复状态**。

也就是 session 级别的数据，它服务的是“下次还能接着干活”。

第二种是 **运行工件**。

也就是 task state、trace、report 这类单次运行记录，它服务的是“解释这次到底发生了什么”。

它让 agent 系统不只是能跑，还能复盘、调试、评估。

最后我总结一下

这五层设计分别解决了不同的问题：

- 入口层解决“怎么把命令变成运行现场”
- 上下文构建层解决“怎么把不同寿命的数据组织成模型可消费的输入”
- 控制循环解决“怎么让 agent 持续推进，而不是一轮就结束”
- 执行与状态更新层解决“怎么安全调用工具，并且把结果沉淀成下一轮还能用的状态”
- 持久化与审计层解决“怎么让系统可恢复、可解释、可评估”

PS：对于面试的时候长回答，最好采用总分总的形式，最后收个尾，让面试官有印象，方便提问，你知道的，大家的注意力在AI时代已经慢慢消散了

7. 怎么减少这个链路里面Agent的幻觉？

参考答案：

pico对于减少 Agent 幻觉不是做单点的优化，而是分层多体系的优化。

主要的思路是沿着整条 harness 链路，把幻觉分散到不同层去消化。

第一，入口层做任务收敛，减少目标歧义。

很多幻觉其实从一开始就埋下了，因为用户输入天然模糊。

所以在 pico 里，用户的一条命令不会直接送进模型，而是先被转换成明确的运行现场，比如当前仓库、执行边界、审批策略、是不是继续旧 session。这一步本质上是在减少模型自行补全任务定义的空间，让模型老实点。也就是说，先把问题收窄，再让模型行动。

第二，上下文层做信息治理，减少脏上下文。

很多时候幻觉的产生是因为模型在无中生有，而是它拿着过期信息、无关信息、冲突信息继续推理。

pico在这里的设计重点不是给模型干净、分层的（或者说更有结构的）上下文

稳定规则、短期工作记忆、相关长期记忆、会话历史、当前请求，这几类信息是分开管理的，而且当前请求优先级最高。

第三，控制循环依赖于证据逐步推进。

在 pico 里，一次任务是一个多轮 runtime。每一轮都会先重建当前上下文，再让模型做一个非常受限的选择：要么调用一个工具，要么返回最终答案。

让模型做一个非常受限的选择：要么调用一个工具，要么返回最终答案。模型在系统里会基于当前证据决定下一步动作。从设计上说，这里最关键的约束有两个。

第一，模型不能直接把自然语言变成外部操作，它必须先落成结构化动作，所以它要先决定“读文件、搜索、列目录、跑命令，还是结束”。

第二，工具执行结果不会停留在那一轮里，而是会回写到 history、working memory 和 task state，变成下一轮推理的输入。agent 每走一步，都会用真实执行结果刷新自己对当前仓库状态的理解。

在 pico 里，如果信息不够，合理路径不是猜一个最像的答案，而是继续读、搜、查；如果已经有足够证据，再收敛成最终的答案。这套控制循环其实是在系统层面把 agent 的默认行为从语言补全改成基于证据逐步推进。

第四，执行层做强约束，把认知幻觉隔离成动作前无法落地。

pico 在执行层有一个统一网关，负责校验动作是否合法、是否重复、是否越权、是否需要审批，然后才真正执行。即使模型内部出现了错误判断，它也不容易直接变成错误操作。把模型可能犯错这个事实，当成前提来设计，而不是当成例外。

第五，状态与恢复层保证世界模型持续对齐。

长任务里最容易出现的是状态幻觉，也就是外部世界已经变了，但 agent 还按旧状态继续推进。所以 pico 会把运行中的有效状态、可恢复状态、单次运行工件分开存，并且在恢复时检查当前工作区和旧状态是否还一致。

第六，可观测性把幻觉从玄学问题变成工程问题。

如果没有 trace、report、task state，你根本不知道这次错误到底是出在上下文污染、动作失败、状态漂移，还是模型自己瞎推理。所以 pico 会把过程、结果和关键状态变化都记录下来。只有可观测，幻觉问题才能被定位、分类、迭代优化。

总结一下

我对 pico 里减少 Agent 幻觉的理解是通过任务收敛、上下文治理、证据优先、执行隔离、状态对齐和可观测性，让错误很难一路穿透到最终结果。在系统设计层面上，就预先规避了部分幻觉。

8. Prompt是怎么构建的

参考答案

Prompt构建的核心目标不是信息越多越好，而是让模型在有限预算里，先看到边界，再看到当前现场，最后对齐到这轮请求。

第一层，是稳定前缀。

这一层先定义 agent 的工作边界。里面会放几类信息：

一类是 agent 自己的身份和输出协议，比如它必须返回工具调用还是 final answer；

一类是工具能力说明，比如有哪些工具、每个工具大概做什么、哪些是高风险动作；

还有一类是仓库的基础快照，比如当前工作目录、repo root、branch、git status、最近几条 commit，以及少量白名单文档的摘要。

所以前缀本质上像一份“运行手册 + 仓库导航包”。

它不是为了回答当前任务，而是先让模型知道自己处在什么环境、能做什么、不能做什么。

第二层，是工作记忆。

pico 不会把完整会话历史直接塞回去，而是先提炼一层 working memory。

这里保留的是当前任务摘要、最近接触过的文件、这些文件的短摘要、还有少量过程笔记。

比如它不会一直带着某个文件的全文，而是保留“最近读过这个文件，这个文件大概讲了什么”。

这样做的目的，是把上下文从“聊天记录”变成“任务现场”。

因为模型下一轮真正需要的，通常不是整段回放，而是当前正在处理什么、刚刚碰过哪些关键文件。

第三层，是相关记忆召回。

除了固定的 working memory，pico 还会围绕当前用户请求，再额外召回少量最相关的 note。

这一步不是把所有历史平均对待，而是看“这轮请求最可能需要哪几条过去的信息”。

而且它的召回方式不是特别黑盒的那种大检索系统，而是相对轻量、可解释的做法，主要根据 tag、关键词和新旧程度来选。

从 system design 角度看，这一层是在解决一个问题：

不是“历史有没有保存”，而是“保存下来的东西里，哪些值得重新进入当前决策”。

第四层，是压缩过的会话历史。

历史当然也会进 prompt，但不是原样堆。

pico 会更重视最近几轮，因为下一步最依赖刚刚发生的事情；更早的历史则做摘要和折叠，比如重复的旧读文件行为不会一直保留，能被文件摘要替代的就直接替代。

也就是说，它不是把 history 当成 transcript，而是把它当成一种可以压缩的状态来源。

这样做的好处是，prompt 不会被冗余历史淹没，同时模型还能保留“最近发生了什么”的连续性。

第五层，是当前用户请求。

这一层永远放在最后，而且优先级最高。

原因很简单：不管前面放了多少上下文，最终这轮推理还是要对齐到“现在要做什么”。

所以 pico 的设计不是把当前请求埋在一大堆背景信息里，而是让它成为 prompt 的最后落点。

然后如果面试官追问“那预算怎么控制”，你可以接这一段：

pico 的 prompt 组装是带预算意识的。

它不是拼完再硬截断，而是先给不同 section 分预算，然后超了再按优先级压缩。

压缩顺序大概体现了它的设计偏好：

先压相关记忆，再压历史，再压工作记忆，最稳定的前缀最后才动，而当前请求基本不动。

这说明它的设计假设很明确：

模型最不能丢的是规则边界和这轮任务本身，最可以牺牲的是一部分可替代的上下文细节。

最后你可以这样收尾：

所以我理解 pico 的 prompt 构建，本质上不是在“组一段长文本”，而是在做上下文分层治理。

前缀负责定义边界，工作记忆负责保留任务现场，相关召回负责把过去真正有用的信息拉回来，压缩历史负责控制体积，当前请求负责最后对齐。

它解决的核心问题不是信息不足，而是信息很多的时候，怎么让模型先看到最重要的东西。

9. 讲一下你的code agent的memory设计，有没有做提示词缓存？然后问了简

历史上的指标

Pico 的记忆设计核心是是让模型在多轮任务里别反复做已经做过的事。

因为对 code agent 来说，代码仓库里的信息其实不是一类东西。

像当前任务目标、最近看过哪些文件、这些文件的大意，这些是下一轮马上还要接着用的；

像这个方向试过了，不成立、刚才那个补丁只做了一半，先看 diff 再继续，这种更像 agent 执行过程里的备忘录；

再往上一层，还有项目约定、依赖事实、用户偏好这类更稳定的东西，有点像项目自己的长期规则。

它们的寿命不一样，如果全混在聊天历史里，或者压成一段大摘要，短期看省事，任务一长就会越来越乱。

所以我一开始就把它拆成了几层。

最靠近当前任务的是工作记忆。实现上，我在运行时的内存状态里专门留了几块：**任务摘要、最近接触的文件、文件短摘要、过程笔记**。比如每次新请求进来，系统会先把这一轮用户要干什么写成任务摘要；工具执行完以后，如果刚刚读过文件，不会把整段文件内容原样塞回去，而是抽一个很短的摘要存起来，同时把这个文件放进最近接触文件列表。这样下一轮模型看到的，就不是一大段历史回放，而是一个很小的工作面，能快速知道自己现在在做什么、最近碰过什么、刚刚读到了什么。

这里最关键的设计其实是文件短摘要，因为它直接解决了重复读文件的问题。我的做法把摘要和文件路径绑在一起，同时记录这个文件当时的内容指纹。你可以把它理解成一个带校验的缓存，不是单纯记住了就算完。下一轮组装上下文时，系统会先检查这个文件现在是不是还是上次那个版本。如果文件后来被改过了，这条旧摘要就不会继续带进提示词。反过来，写文件、打补丁之后，我也会主动把旧摘要作废。因为对 code agent 来说，没记住最多就是多读一遍文件；但如果记住的是旧结论，还拿它继续往下推，后面整条链路都可能都是错误的。所以我在这套设计里一直强调一件事，先解决什么时候不能再信旧信息，再谈怎么复用旧信息。

过程笔记这里，存的是**跨轮还有价值的短结论**，比如某个工具刚刚报错了、某次修改只是部分成功、下一步应该先检查哪里。它不是完整历史回放，更像是给下一轮留的一点提醒，只保留少量高价值信息。召回的时候，我也没有一上来就上很重的语义检索，而是先用比较透明的规则：先看标签有没有直接命中，再看关键词有没有重叠，最后看是不是最近刚确认的。这样做的好处是可解释。后面如果我要分析为什么这条记忆被带回来了，我能说清楚原因，而不是只能说模型觉得它相关。

再往上一层，我还做了**长期记忆**。因为有些内容不应该一直塞在当前会话的工作记忆里，比如项目约定、依赖事实、关键决策、用户稳定偏好。这些东西跟当前任务有关，但它们不属于这一轮结束就可以跟着会话一起消失的状态。所以我没有把它继续放在会话内存里，而是单独落到 `.pico/memory/` 下面，用一个索引文件加若干主题文件去存。这样工作记忆可以保持很小、很贴当前任务，长期事实单独沉淀，二者不会互相污染。真正要恢复运行时，我又把它和检查点分开。检查点里放的是当前任务状态、下一步、关键文件、工作区身份这些恢复时必须信的内容；记忆里放的是下一轮可能有用的信息。

上下文组装的时候，我也不是把所有东西一股脑塞进去，而是按层次放：稳定背景、工作记忆、相关记忆、过程历史、当前请求。预算不够时，会优先压缩相关记忆和旧历史，当前用户这一轮的新请求始终保留。旧历史里如果出现的是早就读过的文件，我还会尽量复用已经存下来的文件摘要，而不是再把完整工具输出塞进去。说得更直白一点，我不是在追求信息越多越好，而是在做上下文预算管理，尽量把这轮真正有用的东西留给模型，把重复的、过期的、没必要每轮都带的东西压掉。

提示词缓存这块，Pico 里也有实现，但它跟memory还是有区别的。记忆解决的是多轮任务里别重复确认已经拿到的事实；提示词缓存解决的是每一轮都会重复发送的那部分稳定内容，比如系统规则、工具说明、工作区摘要。我的做法是把这部分稳定内容单独做成一段稳定前缀，然后给它算一个哈希值。如果模型后端支持缓存，我就把这个哈希值当成缓存键发出去。也就是说，只要稳定前缀没变，后端就有机会复用之前那部分输入。这样可以节省重复传输和重复计算 token，

简历上的指标，对应的是我专门做的一组记忆依赖实验。实验一共设计了 12 类任务模板，每种设置重复跑 5 次，所以每种方案一共是 60 次 follow-up。对照组分三组：开记忆、关记忆、保留记忆框架但故意塞无关内容。最后结果很干净，开记忆以后，follow-up 阶段的重复读文件次数从 60 次降到 0 次；关记忆和塞无关内容，都是 60 次。与此同时，平均工具步数从 1.0 降到 0，平均尝试次数从 2.0 降到 1.0，正确率保持不变。这个结果说明是结构化记忆真的把前面已经确认过的事实记住了，所以 agent 在 follow-up 阶段不需要再调用工具去重新确认。

总结：

Pico 的记忆设计，本质上是在把有用的信息从完整的历史里拆出来，再给这些信息加上寿命管理和失效校验。这样 agent 在多轮任务里既记得住，又不会越记越乱。

10. 任务恢复是什么意思，怎么做的？

参考答案：

任务恢复主要解决 agent 上次做到一半停下了，这次要尽量接着那个任务状态继续做的问题

这里的难点是，Agent不能盲信旧状态。因为代码仓库是活的，文件会变，工作区会变，运行配置也可能变

如果只是把历史重新喂给模型，很容易带着已经过期的结论继续往下继续错误的进行。

所以 Pico 里做的恢复，本质上是恢复会话状态。它会重新构造一个新的 runtime，重新注入当前的模型客户端和当前工作区，然后把上次保存下来的 session 状态读回来。

面最核心的设计是checkpoint（检查点）。在pico中，检查点我把它做成了一份可恢复的短状态，里面会放当前目标、当前卡点、下一步、关键文件，以及这些关键文件当时对应的内容指纹。除此之外，我还会把一份运行身份一起存进去，比如当时的工作目录、模型、审批策略、只读开关、最大步数、功能开关、工作区指纹、工具签名这些。你可以把它理解成，系统不光记住上次做到哪了，还记住上次是在什么条件下做到这一步的。

在准备恢复时不会直接拿这个检查点开跑，而是先做一轮校验。会先清掉已经过期的文件摘要，然后检查两件事：一是关键文件现在是不是当时那个版本，二是当前运行环境和当时记录下来的运行身份是不是还对得上。比如文件内容变了，或者工作区指纹变了，或者模型、审批策略、功能开关变了，这些都会影响系统能不能继续信旧状态。

所以我在做恢复状态的时候不是简单的区分“能恢复”和“不能恢复”。

我这里分了几种。最理想的是一切都对得上，那就直接继续；

如果大部分还对，但有些关键文件已经变了，那就保留任务骨架，把过期的部分标出来，要求重新读取、重新确认；

如果工作区或者运行配置对不上，那就不能直接信旧检查点；

如果连检查点版本都不兼容，那也不能硬接。

还有一种情况，就是根本没有可用检查点，那就只能退回到普通会话状态去兜底。

对任务恢复来说其实大部份情况还是有一部分还能信，有一部分必须重新确认这样子。

恢复之后，系统还会把这份检查点显式放进下一轮 prompt。这样模型看到的不只是原始历史，而是结构化的恢复信息，比如当前目标、卡点、下一步、关键文件，还有这次恢复状态本身。这样模型接手的时候，不需要再从一大段历史里自己猜我上次停在哪，而是直接知道现在最该接着做什么。

我还自己设计了一个小心思，就是恢复和记忆是连在一起的。比如工作记忆里存过某个文件的短摘要，但如果恢复前发现这个文件已经变了，这条摘要会先失效，不会继续带进 prompt。也就是说，Pico 的恢复会先确认哪些旧信息还有效，再决定接多少。这点对 code agent 很重要，因为没记住，最多是多读一遍；但如果记住的是旧事实，还拿它继续推理，后面整条链路都会偏掉。（很危险的场景，需要和面试官强调）

11. 突然问了目前项目支不支持使用skill和mcp，如果要给他引入skill/mcp的功能，应该怎么设计？

参考回答：

目前这个项目还不支持skill 和 MCP。它现在更像一个比较轻量的 coding agent，核心能力是内置工具调用，工具集合是静态注册的，system prompt 也是运行时直接拼出来的。

所以它已经有 agent 的基本闭环了，但还没有做到能力可以被外部发现、加载和扩展这一步。

如果要把 skill 和 MCP 引进来，我会分别设计。

skill 更适合放在提示词和资源这一层，解决的是模型该怎么做；

MCP 更适合放在工具和能力提供这一层，解决的是模型能调用什么。

具体设计上，我会分多步设计。

第一步，先把现在的内置工具抽象成统一的 ToolProvider，把静态工具注册改成 provider registry。

第二步，在这个基础上加一个 SkillRegistry，负责发现、加载、匹配和注入 skill 文件，但 skill 本身不直接执行代码，它只影响 prompt 和执行策略。

第三步，再加一个 MCPToolProvider，把 MCP server 暴露出来的 tools 映射成系统内部统一的工具定义，这样现有的审批、trace、timeout、错误处理这些机制都还能复用，不需要重写一套。

这样做的好处是，skill 和 MCP 的职责是清楚的，核心 runtime 也不会被污染，而且整个演进路径是渐进的，不需要一上来重构成一个很重的平台。第一阶段我会先做 MCP tools，不会急着把 resources、prompts 这些能力一次性全接进来。因为对当前这个项目来说，先把外部工具接进来，收益最大，风险也最低。

总结一下：

目前项目还不支持 skill 和 MCP，它现在只有静态内置工具。要加的话，我会把 skill 放到 prompt/resource 层，把 MCP 放到 tool provider 层。落地顺序是先重构工具注册，再加 skill registry，最后接 MCP tool provider。这样既能复用现有的安全和执行链路，也不会把核心 runtime 做成一堆特判。

12. 现在你这个pico里面有什么工具，工具调用怎么做的，怎么加新工具

参考回复：

当前基础工具主要有 6 个：list_files、read_file、search 负责看目录、读文件、搜字符串；run_shell、write_file、patch_file 负责跑命令、写文件、精确改文件。除此之外，还有一个受限的 delegate，可以把一个小调查任务交给只读的子 agent 去做。

Pico 现在大概是 6 个基础工具，加 1 个受限委派工具。

工具调用这条链路，我做的是模型提申请，runtime 做安检，然后才执行。

模型在 prompt 里先看到工具清单、参数格式和调用示例，返回的是结构化的工具调用。

runtime 拿到模型输出后，会先解析成调工具还是直接结束，如果是调工具，不会直接跳到底层函数，而是统一先走 run_tool()。这里会按顺序做几层检查：先看工具是不是存在，再做参数校验，再拦连续重复调用，高风险工具再走审批，最后才真正执行。执行完以后，结果会写回 history、trace、report；如果涉及读写文件，还会顺手更新 memory，所以后面的上下文、恢复、审计都能接上。

这里在设计上我没有那么看重工具数量，更多考虑能不能有合适的边界。比如所有文件类工具都会先做路径解析，确保路径不能逃出当前工作区；patch_file 不是模糊修改，而是要求 old_text 精确命中且只能出现一次，这样修改行为才是确定的；run_shell 也不是完全放开，它有超时限制，工作目录固定在仓库根目录，环境变量也走 allowlist，不会把父进程环境整包带进去。还有一层很实用的保护，就是如果模型连续两次发起完全一样的工具调用，runtime 会直接拦掉，避免它在没有新信息的情况下空转。

如果要加一个新工具，可以顺着目前的链路加：

第一步，在 pico/tools.py 里把这个工具加进工具定义表，写清楚参数格式、风险等级和说明；

第二步，补一个调用示例，让模型知道该怎么正确发起调用；

第三步，在参数校验逻辑里把这类工具的输入边界写死；

第四步，写真正的执行函数；第五步，把它挂到 runner 映射里，让工具注册表自动把定义和执行绑起来。

这样接完以后，现有的审批、审计、重复调用拦截、路径边界这些护栏会自动接上。如果这个工具会影响工作区或者会产生对后续推理有价值的结果，我再补 memory 更新和测试。

总结：

Pico 的工具层本质上是显式工具表 + 统一执行闸口 + 风险护栏。设计的重点在它每次调工具时，harness这套有没有先把边界守住。

13. 详细问了下怎么做的评测，怎么设计的benchmark

参考答案，逐字稿里有，我建议自己看一下，然后看能不能补点benchmark

评测的思路：先想清楚系统里哪些点必须被观察，后面这些点又该汇总成什么指标。

我先把 Pico 自己最核心的几类问题收成一套轻量但可复现的 benchmark。把每个任务都先写成固定格式，跑完以后不光知道过没过，还知道为什么过，或者卡在了哪。

对于任务定义上，Pico 这边不是随便丢一句 prompt 去跑，而是先把任务写成固定格式。里面会明确任务目标、允许用哪些工具、步数预算、希望工作区最后变成什么状态、以及最后怎么验收。比如有的任务要求把 README 里某一行精确改掉，有的任务要求在预算内完成一次补丁修改，还有的任务要求遇到非法参数后恢复到正确路径继续做。这样任务完成不靠模型自己说完成了，而是看最后的工作区状态和 verifier 能不能对上。code agent 评测和普通问答不一样，很多时候真正要测的是它有没有把仓库改成我想要的样子。

benchmark的设计上，我按系统问题分层设计。大概分四类。

第一类是固定回归任务，主要看 harness 稳不稳。比如路径越界以后能不能被拦住，非法 patch 参数之后能不能恢复，重复读同一个文件会不会被识别成坏循环。这一层更像是在测 runtime 的基本执行合同有没有守住，不是在测模型灵感。

第二类是上下文相关的压力场景。这个不是靠真实任务自然出现，而是我会主动注入更长的 history、更多的 note、不同长度的 request，看 prompt 变厚以后，上下文裁剪会不会把当前请求裁坏，或者把真正重要的信息挤掉。因为这类问题平时看单次 demo 很难看出来，必须做受控场景。

第三类是记忆依赖任务。这类任务我会故意拆成两段，前一段先读文件、确认事实，后一段再问它刚才那个事实是什么或者基于刚才那个约束继续做。这样就能很直接地测出，结构化记忆到底有没有真的接住前面已经确认过的信息，而不是模型碰巧答对。像我现在简历上写的重复读文件从 60 次降到 0，就是这类任务跑出来的。

第四类是恢复相关任务。这里测的是几类具体情况，比如 checkpoint 还有效的时候能不能继续，关键文件已经变了的时候会不会先重读再继续，工作区身份对不上时会不会误信旧状态，旧版本 checkpoint 会不会被错当成可用状态。主要测试系统有没有把旧状态信错，因为对 code agent 来说，误信旧状态比重新读一遍更危险。

这些 benchmark 任务跑起来之后，我会留三层运行信息。

第一层是任务状态，比如这次运行最终是什么状态、模型尝试了多少轮、真正执行了多少步工具、最后是正常结束、步数打满，还是重试过多停下来的。这层回答的是 Agent 最后停在哪。

第二层是过程事件，也就是事件流。我会记录每一轮 prompt 是怎么组出来的、模型什么时候被调用、输出被解析成了什么、工具有没有真正执行、有没有被 runtime 拦下来、执行后工作区有没有变化、有没有触发 checkpoint。这层回答的是中间到底发生了什么。

第三层是结果摘要，里面放最终答案、关键指标、prompt 大小、恢复状态、安全事件这些后面方便聚合的字段。这层回答的是最后该拿什么做统计。

最后做结果汇总的时候，是按问题拆指标。比如看固定回归任务，我就看通过率、预算内完成率、verifier 通过率；看上下文治理，我就看 prompt 长度、压缩率、当前请求有没有被裁坏；看记忆收益，我就看重复读文件次数和 follow-up 阶段的工具步数；看恢复正确性，我就看恢复成功率、工作区漂移识别率、以及有没有误接受旧状态。这样拆开的好处是，一旦某次运行不对，我可以很清晰的知道哪里出了问题，到底是任务固定格式本身有问题，还是工具调用被拦了，还是上下文裁剪出了问题，还是恢复时把旧状态信错了。

14. 有没有用过claude code的 /btw指令，让我来设计到pico里面要怎么实现

老实说我也只是用过没想过，用AI分析了下claude code的源码给大家组织里下面这个答案：

可以把 /btw 理解成一个 side-channel question 机制，而不是一个普通命令。

它解决的是交互系统里的一个经典问题：主任务通常是长链路、有状态、可能还在持续执行，但用户经常会临时插入一个小问题。如果这类问题直接并入主执行链，就会污染任务状态、打断推理节奏，甚至破坏工具调用闭环。

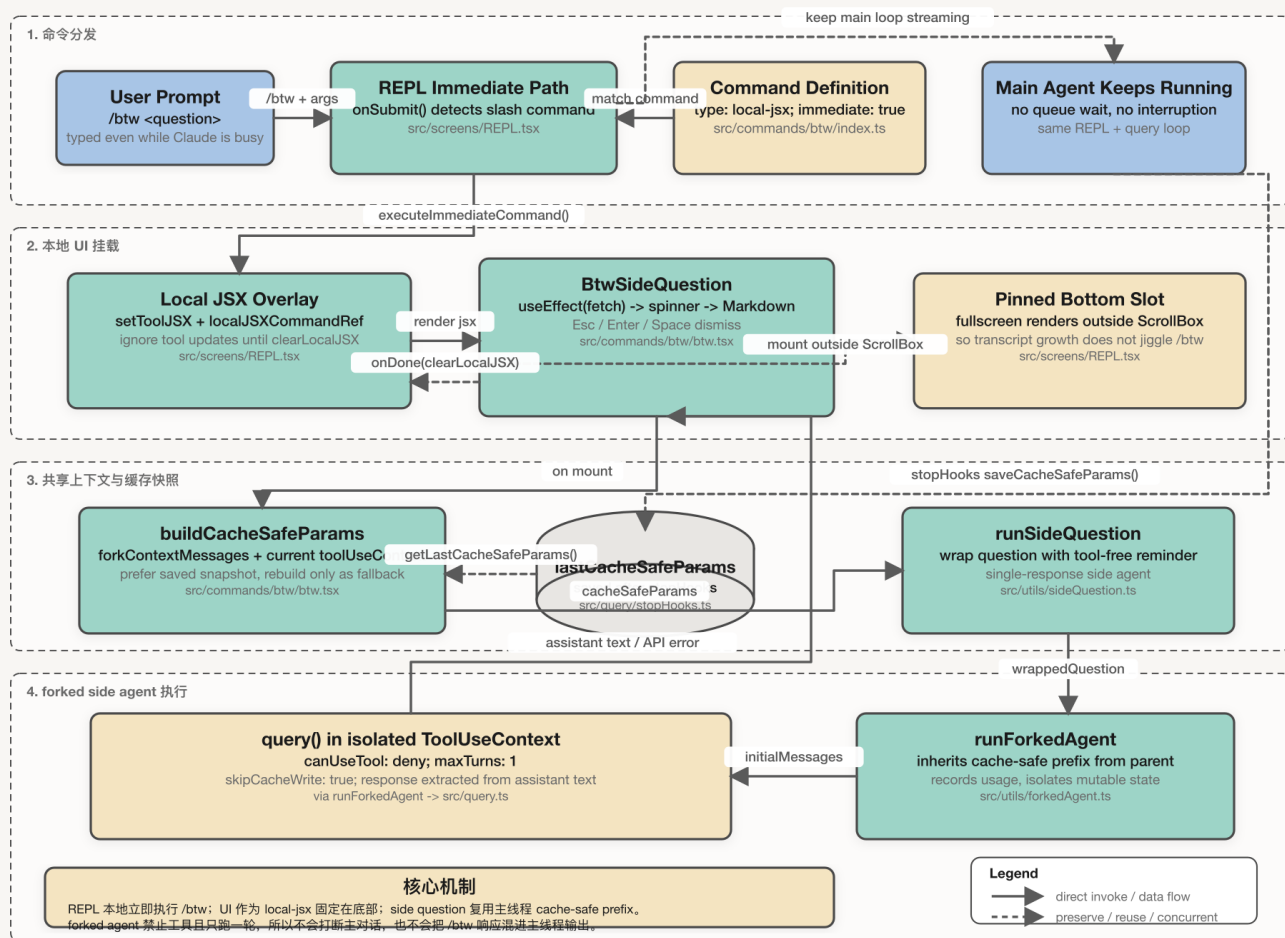
所以它比较好的设计方式，是把它做成一个和主任务并行、但共享上下文视图的轻量级会话。

这个子会话只继承必要上下文，不继承执行权；也就是说它可以读到当前任务的语义背景，但不能接管主任务、不能扩展成新的多轮工具流程。这样系统层面就同时满足了三个目标：主任务连续性不被破坏，用户获得低延迟答复，以及上下文复用带来的成本控制

pico 当前更像一个单线程、同步推进的 agent runtime：CLI 层负责少量本地命令，核心 runtime 负责一次 request 对应的一次控制循环。所以在这个架构下，最合适的 /btw 不是‘运行中插队’，而是‘受控旁路’。也就是在系统里新增一种 side-question 会话类型：它共享父会话的上下文快照和记忆层，但不写回主 session，不参与主 checkpoint，不拥有工具执行权限，生命周期也被严格限制在单次问答。这样设计的好处是最小侵入：不用把现有同步 runtime 重写成调度器，也不用引入复杂的并发一致性问题。面试里我会明确说，第一阶段目标是做对隔离边界和状态语义，而不是追求 UI 上看起来像真正并发。真正要做到‘主任务跑着时随时插入 side question’，那已经不是一个命令特性，而是调度模型升级，要引入前台命令通道、后台任务句柄、状态快照一致性和结果归属规则，这属于更高一层的系统演进。

Claude Code /btw 是怎么实现的

一个 immediate local-jsx 命令：主线程继续跑，旁路 fork 一个只回答一次、禁止用工具的 side agent



也让AI看整个设计生了张图

15. 整个项目中ai coding的占比，会用code agent辅助开发做些什么

自圆其说即可

推荐在下面几个方面说：

1. 测试
2. code review
3. benchmark

16. 你觉得一个好的code agent应该是什么样的，未来code agent的演进形势会怎么样

比较开放的问题，我抛砖引玉一下

1. 拉开差距的，往往不是模型，而是 harness。同一个模型在不同 code agent 里表现差很多，通常不是模型智商差很多，而是 runtime、tooling、guardrails 和 feedback loop 差很多。
2. 记忆要外部化，单靠上下文独木难支。比如 Claude Code 明确把 `CLAUDE.md` 和 auto memory 作为跨 session 的项目记忆，未来高质量 memory 更像 repo 里的工程规范、约定和经验沉淀，而不是一大段越来越长的聊天记录。
3. 形态最终会走向 agent team。未来工程师越来越像 supervisor，agent 更像可委派的执行单元。

17. 平常怎么学习新知识的

建议结合当前AI时代回答，可以给大家参考一下我怎么学东西的

可以突出以下几点：

第一，你有信息判断力；

第二，你会用 AI，但不依赖 AI；

第三，你重视输出闭环；

第四，你能快速学会并沉淀。

我平常学习新知识，比较像在做一个小型项目，而不是零散地看资料。一般我会先明确这次学习的目标，比如我是为了补齐某个技术点，还是为了最后写一篇文章、做一个项目，或者解决一个具体问题。目标定下来以后，我会先去搭一个高质量的资料池，优先看近几年的论文、官方博客、核心代码仓库、技术负责人公开分享，以及高校课程这类一手材料，而不是先看大量二手总结。

接下来我会做一轮快速筛选。自己能直接看懂的部分先通读一遍，建立整体框架；比较难的部分再借助 AI 帮我做解释、翻译或者辅助拆解，

当我对这个领域有一个整体认识之后，我会很快开始做输出，比如写大纲、写技术笔记、做分享，做成slide。因为我自己很相信，真正能留下来的不是你看了多少，而是你能不能把它讲清楚、写出来、用出来。输出的过程其实也是检验自己有没有真的理解的过程。

最后我会再做一轮复盘和压缩，把啰嗦、不严谨或者逻辑不连贯的地方修掉，必要时再查漏补缺。所以对我来说，学习新知识不是输入很多信息，而是一个收集—筛选—理解—实践—输出—迭代的闭环。

比如我前段时间为了系统研究大模型后训练流程，就专门搭了一个学习仓库，把论文、博客、课程、代码仓库都结构化整理进去，先做初筛，再结合 AI 和代码阅读补理解，最后把内容整理成一篇面向非专业读者也能看懂的markdown + 组会ppt。这个过程中我会发现，很多一开始以为自己懂的地方，真正写的时候其实还讲不明白，反过来又会逼着我继续补齐。这个方式对我来说效果一直比较好。

建议复盘方向及学习资料

1. 多看claude code的设计：<https://github.com/Rito-w/claude-code>
2. 多看claude code怎么用的，从产品的角度去思考怎么改进自己的code agent：
<https://code.claude.com/docs/zh-CN/best-practices>
3. 多了解harness，一定要熟悉概念及看一些最佳实践：<https://zhanghandong.github.io/harness-engineering-from-cc-to-ai-coding/part7/ch25.html>

AI Coding常见面试题

1.如何设计一个高质量的 Skill？需要考虑哪些要素？ (阿里)

我理解的高质量 Skill，是把某一类稳定、可复用的任务封装成一个能力单元。

它应该满足几个条件：**触发清楚、职责单一、输入输出明确、步骤稳定、边界明确、可测试可迭代。**

具体来说，我会从六个方面设计。

第一是**触发条件**。Skill 要明确什么时候应该被调用，什么时候不应该被调用。比如“生成单元测试 Skill”只在用户要求补测试、提高覆盖率、修复测试失败时触发，不应该在用户只是问业务逻辑时触发，否则会和代码解释、代码重构类 Skill 混淆。

第二是**单一职责**。一个 Skill 最好只解决一类问题，不要既做需求分析，又写代码，又做测试，又做部署。比如“根据接口文档生成前端 API Client”可以是一个 Skill，“修复 TypeScript 类型错误”可以是另一个 Skill。职责越清楚，模型越容易稳定执行。

第三是**指令清晰度**。Skill 内部的步骤要可执行，不能只写抽象原则。比如不要只写让模型帮你生成高质量代码，而是写成：先阅读已有同类文件，提取代码风格；再生成实现；然后补充必要测试；最后运行 lint 和 test，并汇报修改点。这种步骤对模型更友好。

第四是**上下文和资源引用**。Skill 要告诉模型优先使用哪些信息，比如项目 README、接口 schema、现有代码示例、组件库规范、测试命令等。如果有脚本、模板、工具函数，优先让模型调用这些资源，而不是凭空生成。

第五是**边界定义**。Skill 要明确不能做什么。比如不得修改公共 API、不得引入新依赖、不能猜测业务规则、如果缺少字段定义，需要先标记 TODO 或提问。这可以避免 AI coding 里常见的幻觉式实现。

第六是**可测试性和可迭代性**。一个 Skill 的好坏不能只靠感觉，应该能通过 eval 或实际指标衡量，比如任务完成率、一次通过测试率、代码 review 修改次数、是否遵守项目规范、是否产生多余改动。后续根据失败案例不断补充反例和示例。

第七是**要有示例**，就是在设计 skill 的时候，需要给出几个归纳出来的示例，尽量覆盖你的 skill 边界。示例是最有用的，至于什么场景触发边界，那些都很随缘，很多模型都会忽略一部分上下文，但是我发现它们对示例都非常执着。

总结：好的 Skill 是一个面向具体问题的、可复用的、边界清晰的、能被评估和持续优化的 Agent 能力。

每一点能结合实际例子说明更佳

比如我要设计一个“为已有函数补充单元测试”的 Skill。

它的触发条件是：用户明确要求补测试、提高覆盖率，或者 CI 因测试缺失而失败。

它的输入包括：目标函数路径、已有测试目录、项目测试框架，比如 Jest、Vitest、Pytest，以及已有测试样例。

它的执行步骤可以是：

先阅读目标函数和相邻测试文件，学习项目里的测试风格；然后分析函数的正常路径、异常路径、边界条件；再生成最小必要测试用例；最后运行对应测试命令，如果失败就根据报错修复。

它的边界是：不修改业务代码，除非发现明显 bug 并向用户说明；不为了让测试通过而降低断言强度；不引入新的测试框架。

它的评估方式是：测试是否通过，覆盖率是否提升，测试是否覆盖边界条件，是否符合项目已有风格。

2.什么样的prompt 算得上优秀的prompt？（阿里）

我理解的优秀 Prompt，是能让模型在明确目标、充分上下文、清晰约束和可验证标准下稳定完成任务。

在 AI coding 场景里，一个高质量 Prompt 通常包含六个要素。

第一是目标明确。要告诉模型最终要完成什么，而不是只描述一个模糊方向。比如不要只说“优化这个接口”，而是说“将订单查询接口的 P95 延迟从 800ms 降到 300ms 以内，优先从数据库查询和缓存命中率角度分析”。

第二是上下文充分。模型需要知道相关文件、错误日志、接口定义、已有实现、测试方式、业务约束。OpenAI Codex 的最佳实践也把好的任务描述拆成 Goal、Context、Constraints、Done when，也就是目标、上下文、约束和完成标准。这样可以减少模型猜测，让产出更容易 review。

第三是约束清楚。比如不能修改公共 API，不能引入新依赖，不能改变数据库 schema，不能降低测试断言强度，必须符合项目现有代码风格。AI coding 的很多问题不是“不会写代码”，而是“写过头了”或者“擅自改了不该改的东西”。

第四是输出格式明确。比如要求输出“修改方案、涉及文件、风险点、测试结果”，或者要求先给 plan，不要直接改代码。对于复杂任务，我会让模型先分析和规划，再实现。Cursor 的 agent 实践也强调先规划、再编码，并让 agent 先研究代码库、提出问题、生成包含文件路径和代码引用的计划。

第五是验收标准明确。比如“通过 npm test 和 npm run lint”，“新增测试覆盖空数组、非法参数、超时重试三类 case”，“修复后错误日志不再出现 NullPointerException”。没有验收标准，模型只能按语言概率完成；有验收标准，模型才能围绕可验证目标迭代。

第六是允许澄清和拒绝猜测。优秀 Prompt 不应该逼模型在信息不足时瞎写，而应该要求它在缺少关键字段、业务规则或接口契约时先提问，或者标记假设。

举个例子。

不好的 Prompt 是：

“帮我修一下这个登录问题。”

更好的 Prompt 是：

“请修复用户登录后偶尔被重定向回登录页的问题。相关文件是 `authMiddleware.ts`、`sessionService.ts` 和 `login.e2e.spec.ts`。现象是：用户登录成功后，刷新页面有 10% 概率丢失 session。约束：不要修改登录接口返回结构，不要引入新依赖，不要降低现有测试断言。请先分析可能原因和修改计划，经确认后再改代码。完成标准是：补充一个能稳定复现该问题的回归测试，修复后该测试和现有 auth 测试全部通过，并说明风险点。”

总结：优秀 Prompt 的本质是把“人的意图”转换成“模型可执行、可约束、可验证的任务说明”。

3.skill和prompt的区别（字节、腾讯）

普通 prompt 更偏一次性任务描述，而 Skill 更像一个可复用的操作规程。Prompt 解决的是**这次怎么让模型做好**，Skill 解决的是**以后遇到同类任务，怎么稳定做好**。所以 Skill 更强调触发条件、任务边界、资源引用、执行流程和评估闭环。

两者可以从几个维度区分。

从使用频率看，Prompt 适合一次性、低频、临时任务；Skill 适合高频、稳定、可复用任务。

从稳定性看，Prompt 依赖用户当次描述是否完整；Skill 会把沉淀过的流程、约束、资源和反例固化下来，减少每次重新解释。

从内容看，Prompt 通常是一段自然语言；Skill 可以包含 markdown 指令、脚本、模板、示例文件、测试命令和外部工具调用说明。

从触发方式看，Prompt 通常由用户显式输入；Skill 可以由模型根据任务描述自动判断是否适用，也可以由用户显式调用。

从工程管理看，Prompt 更像个人技巧；Skill 更像团队资产。它可以被版本管理、评测、复盘和持续优化。

举个例子。

如果我只是说：“帮我给 `calculatePrice` 写单元测试”，这是 Prompt。

如果我沉淀了一个“为已有函数补充单元测试”的 Skill，里面规定：

“触发条件：用户要求补测试、提高覆盖率、修复测试缺失。执行步骤：先阅读目标函数和相邻测试，识别正常路径、异常路径、边界条件；再按项目现有测试风格生成最小必要用例；最后运行指定测试命

令。边界：不修改业务代码，不降低断言强度，不引入新测试框架。评估：测试通过、覆盖率提升、边界条件覆盖、风格一致。”

这就是 Skill。

一句话总结：Prompt 是指挥模型完成一次任务；Skill 是把一类任务沉淀成可复用、可测试、可迭代的 Agent 能力。

5.AI 生成代码时，如何避免幻觉？（京东）

让你来设计一个AI Coding的工作流，你会怎么设计？（字节）

第一步，**明确任务边界**。先判断这是 bug fix、重构、补测试、性能优化，还是新功能。不同任务给 AI 的上下文和约束不同。

第二步，**让 AI 先读代码和出 plan**。对于复杂任务，不直接让它改代码，而是要求它先说明涉及哪些文件、准备怎么改、有哪些风险、需要哪些测试。OpenAI Codex 的最佳实践也建议复杂、模糊或难描述的任务先规划，再进入实现。

第三步，**小步修改**。尽量让 AI 一次只处理一个明确目标，不把需求分析、架构调整、代码实现、测试补充、性能优化全部塞到一次任务里。

第四步，**让 AI 自己跑验证**。包括单测、lint、类型检查、构建、相关 e2e 测试。Cursor 的 agent 实践也强调要给 agent 可验证目标，例如类型系统、lint 和测试，因为 agent 需要明确知道什么结果算正确。

第五步，**人做最终 review**。重点看 diff 是否超范围、业务语义是否正确、异常路径是否完整、是否引入安全问题、是否破坏兼容性。

第六步，**用 CI 和灰度兜底**。AI 生成代码也必须走正常工程流程，不能因为是 AI 写的就绕过 review、测试和发布管控。

6.如果让你用AI做code review，你会怎么做？（京东）

具体会分以下几步：

第一，我会给它足够上下文，包括 PR 需求背景、diff、相关测试、接口定义、项目规范和 CI 结果。因为只看局部代码很容易误判。

第二，我会先让它总结这次 PR 改了什么，影响哪些模块，有没有改接口、数据库、消息格式、缓存 key 或配置。这样可以确认它是否理解了变更范围。

第三，我会让它按 checklist 做 review，重点看功能正确性、边界条件、异常处理、兼容性、性能、并发幂等、事务一致性、安全风险和测试质量。比如在电商场景里，我会重点让它检查订单状态流转、库存扣减、支付回调、优惠金额计算、MQ 重复消费这些风险点。

第四，我会要求 AI 的 review comment 必须具体，包含问题位置、风险原因、影响范围、修改建议和验证方式，而不是泛泛地说这里可能有问题。

第五，AI 提出的结论我不会直接采信，而是作为候选问题。最终还是由人根据业务语义和架构约束判断。AI 更适合提高 review 覆盖率，帮我发现边界 case、重复代码、安全问题和测试遗漏，但不能替代人对业务正确性的判断。

如果面试官让你写 prompt，可以是这样的：

```
▼ Markdown |
1  请你作为代码审查助手 review 下面这个 PR。
2
3  背景：
4  - 需求目标：修复用户重复提交订单时可能产生两笔订单的问题。
5  - 业务约束：同一个 userId + skuId + requestId 在 5 分钟内只能创建一笔订单。
6  - 重点关注：并发、幂等、事务一致性、库存扣减、MQ 重复消费、测试覆盖。
7
8  输入：
9  - PR diff: ...
10 - 相关文件: ...
11 - 已有测试: ...
12 - CI 结果: ...
13
14 请按以下格式输出：
15 1. PR 改动总结
16 2. 高风险文件和原因
17 3. Blocking issues
18 4. Non-blocking suggestions
19 5. 建议补充的测试用例
20 6. 需要向作者确认的问题
21
22 要求：
23 - 不要泛泛而谈。
24 - 每个问题必须说明文件位置、原因、影响和建议。
25 - 不要把风格问题上升为 blocking。
26 - 如果证据不足，请明确说“需要确认”，不要直接下结论。
```

尝试过用AI写测试吗？ 如何避免AI“为了通过而写测试”？

我用过 AI 写测试，主要用于补单元测试、回归测试、接口测试、边界 case 和异常路径测试。但我不会直接让 AI “提高覆盖率”，因为这样容易生成弱测试。我的做法是先让 AI 阅读目标函数、接口契约和已有测试风格，再输出测试计划，列清楚正常路径、异常路径、边界条件、mock 边界和关键断言。对于 bug fix，我会要求 AI 先写一个修复前失败、修复后通过的回归测试，避免它先改代码再补一个形式化测试。为了防止 AI 为了通过而写测试，我会明确禁止删除或 skip 失败测试、弱化断言、mock 掉核心逻辑、只断言非空、盲目更新 snapshot、为了测试通过修改业务语义。每个测试 case 都要说明验证的业务行为和关键断言。最后由人重点 review 断言是否有效、是否覆盖边界和异常路径、是否符合项目风格。

如果要让你写个prompt，可以参考这个：

```
▼ Markdown |
1  请为下面这个函数补充单元测试。
2
3  目标：
4  - 验证业务行为，而不是实现细节。
5  - 覆盖正常路径、异常路径和边界条件。
6  - 如果是 bug fix，请先写能复现问题的失败测试。
7
8  输入：
9  - 目标函数：
10 - 类型定义：
11 - 业务规则：
12 - 已有测试样例：
13 - 测试框架：
14
15 要求：
16 1. 先输出测试计划，列出每个 case 的目的和关键断言。
17 2. 再生成测试代码。
18 3. 不允许删除、skip 或弱化已有测试。
19 4. 不允许 mock 被测函数本身或核心业务逻辑。
20 5. 不允许只使用 toBeTruthy、not.toBeNull 这类弱断言。
21 6. 每个测试必须验证明确的业务结果。
22 7. 如需 mock 外部依赖，请说明 mock 边界。
23 8. 最后说明新增测试覆盖了哪些风险。
```

如果让你用AI工具排查一个线上问题你会怎么排查

AI 在排查线上问题时，可以考虑它主要可以做六件事：

第一，整理脱敏后的日志，提取高频异常、最早出现时间、影响接口、错误堆栈和共同请求特征；

第二，结合 trace 和代码分析调用链，判断是入口服务、下游依赖、缓存、数据库还是 MQ 出现问题；

第三，对比最近发布、配置变更、开关变更、数据库变更，形成故障时间线；

第四，按“现象、假设、证据、验证方式”生成排查假设；

第五，辅助写只读 SQL，统计异常订单、错误码、状态分布等；

第六，总结监控指标，比如错误率、P95、连接池、慢查询、MQ 堆积等，帮助定位可疑模块。

但 AI 给出的根因只能作为候选假设，最终必须通过日志、监控、trace、代码 diff 和复现验证。生产修复仍然要走回滚、限流、降级、灰度和复盘流程。